

OpenShift v4 Infrastructure Day to day activities

Get one to one assistance for OpenShift Hands on labs (50 labs).
WhatsApp Dhinesh +91 9444410227 and get started today!

<https://assistedcloud.com/>

Node Management (Masters & Workers)

1. What command would you use to see the status of all nodes in the cluster?

Command: `oc get nodes -o wide`

Description: This is a fundamental command for cluster administrators.

- ✓ `oc get nodes` lists all registered nodes (control plane and workers).
- ✓ The STATUS column shows if a node is Ready (healthy and schedulable), NotReady (unhealthy or unreachable), or Ready,SchedulingDisabled (healthy but cordoned).
- ✓ Adding `-o wide` provides additional valuable information, including the node's internal and external IP addresses, OS image, kernel version, and container runtime version, giving a richer snapshot of the node's state and configuration.

2. How can you determine the Kubelet version running on a specific node?

Command: `oc get node <node_name> -o jsonpath='{.status.nodeInfo.kubeletVersion}'`

Description: The Kubelet is the primary agent running on each node that registers the node with the API server and manages pods and containers. Knowing its version is crucial for:

- ✓ **Compatibility:** Ensuring the node's Kubelet version aligns with the control plane version, especially during or after upgrades.
- ✓ **Troubleshooting:** Identifying if issues might be related to known bugs in a specific Kubelet version.
- ✓ This command directly queries the node's status information reported to the API server and extracts the specific kubeletVersion field.

3. Explain the difference between a node's Capacity and Allocatable resources. How do you check them?

Command to Check: `oc describe node <node_name> | grep -E 'Capacity|Allocatable'`

Description:

- ✓ Capacity: Represents the *total* amount of resources (CPU, memory, ephemeral-storage, pods) physically available on the node hardware. It's the raw capability of the machine.
- ✓ Allocatable: Represents the amount of resources *available for user pods* to consume. It is calculated by subtracting resources reserved for the operating system, the container runtime (CRI-O), and the Kubelet itself from the total Capacity.
- ✓ Why the difference matters: The Kubernetes scheduler uses the Allocatable value when deciding where to place pods. Understanding this difference is key for accurate capacity planning and troubleshooting situations where pods won't schedule even if Capacity seems sufficient. The `oc describe node` command displays both values clearly.

4. How would you check the current CPU and memory usage of a specific node?

Command: `oc adm top node <node_name>`

Description: This command provides a real-time snapshot of the actual resource *consumption* on the specified node. It relies on the metrics-server component being deployed and healthy in the cluster (which it usually is by default in OCP 4).

- ✓ It shows CPU usage (in cores/millicores) and memory usage (in bytes, typically MiB or GiB).
- ✓ It also shows the percentage of the node's *allocatable* resources being used.
- ✓ This is essential for identifying nodes under heavy load or diagnosing performance bottlenecks.

5. What is the purpose of cordoning a node, and what command achieves this?

Command: `oc adm cordon <node_name>`

Purpose: Cordoning marks a node as *unschedulable*. This means the Kubernetes scheduler will *not* place any *new* pods onto this node.

Use Case: It's the first step when preparing a node for maintenance (like patching, hardware changes, or rebooting). It prevents new work from landing on the node while allowing existing pods to continue running without disruption until they are deliberately drained or terminate naturally.

6. What is the command to allow scheduling back onto a cordoned node?

Command: `oc adm uncordon <node_name>`

Purpose: This command removes the unschedulable taint added by the cordon command.

Use Case: After node maintenance is complete and the node is verified to be healthy, uncordoning makes it available again for the scheduler to place new pods onto it, bringing it back into full service within the cluster.

7. Describe the process and command for safely draining a node for maintenance. What precautions should be taken?

Command: `oc adm drain <node_name> --ignore-daemonsets --delete-emptydir-data`

Process: Draining automates the process of safely removing workloads before node maintenance. It performs two main actions:

- ✓ Cordon the node (marks it unschedulable).
- ✓ Evicts (gracefully terminates and reschedules) all regular pods running on the node. It respects PodDisruptionBudgets (PDBs), ensuring application availability isn't compromised below configured levels.

Command Flags:

- ✓ `--ignore-daemonsets`: DaemonSet pods are meant to run on specific (or all) nodes and are managed differently; they are not evicted by drain. This flag tells drain to proceed even though DaemonSet pods will remain.
- ✓ `--delete-emptydir-data`: Pods using emptyDir volumes will lose their data when evicted (as emptyDir is tied to the pod lifecycle on that specific node). This flag confirms you understand and accept this data loss.

Precautions:

- ✓ PodDisruptionBudgets (PDBs): Ensure critical applications have PDBs configured correctly *before* draining. A PDB that's too restrictive (e.g., requiring 100% availability) can block the drain indefinitely.
- ✓ Stateful Workloads: Understand how stateful applications handle termination and rescheduling. Ensure data is persisted correctly (using Persistent Volumes) or that the application can gracefully handle leader election changes or instance restarts.
- ✓ Cluster Capacity: Verify there is enough capacity on other nodes to accommodate the pods being evicted from the drained node.
- ✓ Drain Timeout: Drains can take time, especially if pods have long termination grace periods. Monitor the process.

8. How can you tell if a node is currently unschedulable?

Methods:

- ✓ `oc get nodes`: Look at the STATUS column. If it includes SchedulingDisabled, the node is cordoned/unschedulable (e.g., Ready,SchedulingDisabled).
- ✓ `oc describe node <node_name>`: Look for the Taints section. A cordoned node will have a taint like `node.kubernetes.io/unschedulable:NoSchedule`.
- ✓ `oc get node <node_name> -o jsonpath='{.spec.unschedulable}'`: This command specifically checks the unschedulable field in the node's spec. If it returns true, the node is cordoned.

9. Why might you add labels to nodes? Provide the command to add a label.

Command: `oc label node <node_name> key=value` (e.g., `oc label node worker-gpu-1.example.com accelerator=nvidia-a100`)

Purpose: Labels are key-value pairs used to organize and categorize nodes. Common use cases include:

- ✓ Targeting Workloads: Using `nodeSelector` or `nodeAffinity` in pod specifications to ensure pods run only on nodes with specific hardware (like GPUs), geographic location (`region=east`), environment (`env=prod`), or specific storage capabilities.
- ✓ Applying Configurations: Targeting specific nodes for MachineConfigs or Tuned profiles.
- ✓ Inventory/Grouping: Simply organizing nodes for easier filtering and management (e.g., `role=infra`, `project=billing`).

10. How do you remove a label from a node using the oc command?

Command: `oc label node <node_name> key-` (Note the trailing hyphen after the key).

Description: This command removes the label identified by key from the specified node. This is used when a label is no longer relevant, was applied incorrectly, or the node's role/characteristic has changed.

11. Explain the concept of node taints and tolerations.

Concept: Taints and tolerations work together to control which pods can (or prefer not to) schedule onto specific nodes.

- ✓ Taint: Applied to a *node*. It acts as a "repellant" mark, indicating that pods generally shouldn't schedule there unless they explicitly tolerate the taint.
- ✓ Toleration: Applied to a *pod*. It indicates that the pod is "willing" to schedule onto nodes that have matching taints.

Effects: Taints have effects:

- ✓ NoSchedule: No *new* pods will be scheduled unless they tolerate the taint. Existing pods are unaffected. (Cordoning uses this).
- ✓ PreferNoSchedule: The scheduler will *try* not to schedule pods without the toleration onto the node, but it's not a strict requirement.
- ✓ NoExecute: New pods won't schedule, and *existing* pods running on the node *without* the toleration will be evicted. Often used for node conditions like NotReady or DiskPressure.

Use Case: Dedicating nodes for specific functions (e.g., infra workloads, specific hardware), ensuring pods only run on appropriate nodes, or automatically evicting pods from unhealthy nodes.

12. How would you add a NoSchedule taint to a node?

Command: `oc adm taint node <node_name> key=value:NoSchedule` (e.g., `oc adm taint node infra-node-1 role=infra:NoSchedule`)

Description: This command applies a taint with the specified key, value, and the NoSchedule effect. Any pod wanting to schedule on this node must now have a toleration in its spec for key=value (or just key if the operator is Exists). This is commonly used to reserve nodes for specific types of workloads that are configured with the necessary toleration.

13. How do you remove a specific taint from a node?

Command: `oc adm taint node <node_name> key:Effect-` (Note the trailing hyphen after the effect). (e.g., `oc adm taint node infra-node-1 role:NoSchedule-`)

Description: This removes the taint identified by the specified key and Effect. If multiple taints exist with the same key but different effects, only the one matching the specified effect is removed. Removing a NoSchedule taint makes the node generally available for scheduling again (assuming no other taints prevent it).

14. What information can you find using `oc describe node <node_name>`?

Description: This command provides a wealth of detailed information about a node's current state and configuration as known by the API server. Key sections include:

- ✓ Labels & Annotations: Metadata attached to the node.
- ✓ Taints: Any scheduling restrictions applied.
- ✓ CreationTimestamp: When the node object was created.
- ✓ Conditions: The node's health status (Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable) with reasons and transition times. Crucial for diagnosing NotReady states.
- ✓ Addresses: InternalIP, ExternalIP, Hostname.
- ✓ Capacity & Allocatable: Total vs. available resources (CPU, memory, pods, ephemeral-storage).
- ✓ System Info: OS Image, Kernel Version, Kubelet Version, CRI-O Version, Operating System, Architecture.
- ✓ Pods: A list of non-terminating pods currently scheduled on the node, including their resource requests/limits.
- ✓ Events: A chronological list of recent events related to the node (e.g., health checks, scheduling decisions, image pulls, volume mounts, reboots detected via Kubelet restarts). Essential for troubleshooting.

15. How do you find the internal and external IP addresses assigned to a node?

Commands:

- ✓ `oc get node <node_name> -o wide`: The INTERNAL-IP and EXTERNAL-IP columns show the primary addresses.
- ✓ `oc get node <node_name> -o jsonpath='{.status.addresses}'`: This provides a structured list of all addresses associated with the node, clearly identifying their type (InternalIP, ExternalIP, Hostname).
- ✓ `oc describe node <node_name>`: The Addresses section lists all types clearly.

Description: Nodes typically have an internal IP used for cluster communication and may have an external IP for access from outside the cluster network (common in cloud environments). Knowing these is vital for networking configuration and troubleshooting.

16. How can you check if the Kubelet on a node is healthy using the API?

Command: `oc get --raw /api/v1/nodes/<node_name>/proxy/healthz`

Description: This command uses the API server as a proxy to directly access the Kubelet's /healthz endpoint on the specified node.

- ✓ If the Kubelet is running and healthy, it will typically return ok.
- ✓ If it fails or times out, it indicates a problem with the Kubelet process itself or network connectivity between the API server and the Kubelet on that node. This is a more direct health check than just relying on the Ready status, which involves other factors.

17. What is the recommended way to gain shell access to an RHCOS node for debugging?

Command: `oc debug node/<node_name>`

Description: This is the standard, supported method in OpenShift 4. It works by:

- ✓ Scheduling a new, privileged pod directly onto the target node.
- ✓ This pod mounts the node's host filesystem at /host within the pod.
- ✓ It automatically provides an interactive shell session within this pod.
- ✓ From inside the pod's shell, you can run `chroot /host` to enter the node's actual filesystem context.

Why preferred: It doesn't require managing SSH keys for nodes, leverages cluster authentication/authorization, and provides the necessary privileges within a temporary, managed container.

18. Once you have shell access via a debug pod, how would you check the node's disk usage?

Command: `chroot /host df -h`

Description:

- ✓ First, you use `chroot /host` to change the root filesystem context from the debug pod's filesystem to the node's actual host filesystem (mounted at `/host`).
- ✓ Then, you run standard Linux commands like `df -h` (disk free, human-readable) to see the usage statistics for the node's mounted filesystems (root partition, `/var`, etc.). This is crucial for diagnosing DiskPressure conditions.

19. How would you check the status of the kubelet or crio systemd services on an RHCOS node?

Commands (inside `oc debug node/... pod`):

- ✓ `chroot /host systemctl status kubelet`
- ✓ `chroot /host systemctl status crio`

Description: After using `chroot /host` within the debug pod, you can use standard `systemctl` commands to interact with the node's `systemd` services. `systemctl status <service>` checks if the service is active (running), enabled, and shows recent log entries, helping diagnose issues if these core components are not running correctly.

20. How do you identify which node a particular pod is currently scheduled on?

Command: `oc get pod <pod_name> -o wide -n <project_name>`

Description: The `-o wide` output format for `oc get pod` includes a `NODE` column that explicitly shows the name of the node where the pod is running. This is essential for correlating pod issues with potential node problems or for accessing node-specific information related to the pod.

21. What command lists all pods running on a specific node?

Command: `oc get pods --all-namespaces -o wide --field-selector spec.nodeName=<node_name>`

Description: This command filters the list of pods across all projects (`--all-namespaces`) to show only those whose `spec.nodeName` field matches the specified node. The `-o wide` output helps see details like pod IP and readiness status alongside the node name. It's useful for understanding the workload distribution on a node or identifying all potentially affected pods if a node has issues.

22. How can you check the version of the container runtime (CRI-O) on a node?

Command: `oc get node <node_name> -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'`

Description: Similar to checking the Kubelet version, this retrieves the specific version of the container runtime (CRI-O in OpenShift 4) reported by the node. This is useful for checking compatibility or known issues related to the runtime.

23. When might you need to force delete a pod, and what is the command? What are the risks?

Command: `oc delete pod <pod_name> -n <project_name> --grace-period=0 --force`

When Needed: This is a last resort used when a pod is stuck in the Terminating state indefinitely. This usually happens because the Kubelet on the node cannot successfully stop the container(s) within the pod, often due to unresponsive processes, storage issues (unmountable volumes), or network problems preventing cleanup.

Risks:

- ✓ Bypasses Graceful Shutdown: The container process does not receive a SIGTERM signal and has no chance to shut down cleanly. This can lead to data corruption or inconsistent state, especially for stateful applications.
- ✓ Resource Leaks: Resources held by the pod (like network endpoints or potentially mounted volumes on the node) might not be cleaned up correctly by the Kubelet, potentially requiring manual intervention or a node reboot later.
- ✓ StatefulSet Issues: Force deleting pods managed by a StatefulSet can violate its ordering and uniqueness guarantees, potentially leading to data inconsistencies or "split-brain" scenarios if not handled carefully. Use `oc delete pod ... --force --grace-period=0` *very cautiously*.

24. How are node certificates managed in OpenShift 4, and how can you check their status?

Management: Node (Kubelet server and client) certificates are managed automatically by the cluster.

1. When a node joins, its Kubelet generates a Certificate Signing Request (CSR).
2. The kube-controller-manager automatically approves CSRs for recognized nodes.
3. The kube-apiserver signs the certificate.
4. Certificates have a relatively short lifespan (e.g., 1 year). The Kubelet automatically requests renewal before expiration, repeating the CSR process.

Checking Status:

- ✓ Node Conditions: The primary indicator is the Ready condition of the node (`oc describe node <node_name>`). Certificate issues often cause the Kubelet to fail communication, leading to a NotReady status with relevant messages in the conditions or events.
- ✓ CSRs: You can list CSRs with `oc get csr`. Look for pending or failed requests related to nodes (`kubernetes.io/kubelet-serving` or `kubernetes.io/kube-apiserver-client-kubelet`). Usually, approved CSRs are cleaned up quickly.
- ✓ Kubelet Logs: If a node is NotReady, checking Kubelet logs (`oc debug node/..., chroot /host journalctl -u kubelet`) often reveals specific certificate errors (e.g., expired, unable to request).

25. In an IPI environment, how do you link a Node object back to its corresponding Machine object?

Command: `oc get node <node_name> -o jsonpath='{.metadata.annotations.machine\.openshift\.io/machine}'`

Description: In Installer-Provisioned Infrastructure (IPI) or environments where the Machine API Operator (MAO) manages nodes, each Node object has an annotation (machine.openshift.io/machine) that stores the name and namespace of the corresponding Machine custom resource. The Machine object represents the underlying infrastructure instance (e.g., EC2 instance, vSphere VM) and manages its lifecycle. This command extracts that annotation value.

26. How can you determine the MachineSet that manages a specific Machine object?

Command: First, get the Machine name (using the previous question's command if starting from a Node). Then:

`oc get machine <machine_name> -n openshift-machine-api -o jsonpath='{.metadata.ownerReferences[?(@.kind=="MachineSet")].name}'`

Description: Machine objects are typically created and managed by a MachineSet (analogous to how ReplicaSets manage Pods). The MachineSet defines the template (instance type, image, user data) and the desired number of replicas for a group of identical machines. A Machine object's metadata contains an ownerReferences field pointing to its controlling MachineSet. This command filters the owner references to find the one whose kind is MachineSet and extracts its name.

27. What's a key indicator that a node is managed by the Machine API Operator (MAO)?

Indicator: The presence of the machine.openshift.io/machine annotation on the Node object.

Command to Check: `oc get node <node_name> -o jsonpath='{.metadata.annotations.machine\.openshift\.io/machine}'`

Description: If this command returns a value (like openshift-machine-api/worker- LZNQ-machine-0), it strongly indicates the node's lifecycle is tied to a Machine object, which is managed by the Machine API Operator. Nodes provisioned manually outside of the Machine API (common in some UPI scenarios) will typically lack this specific annotation.

Cluster Operators & Health

28. How do you check the health status of all Cluster Operators?

Command: `oc get clusteroperator` or `oc get co`

Description: This is the primary command for a high-level cluster health check. Cluster Operators are controllers that manage specific components of OpenShift (like networking, authentication, registry, etc.). This command lists all operators and their current status (AVAILABLE, PROGRESSING, DEGRADED, UNKNOWN). A healthy cluster typically shows all operators as AVAILABLE=True, PROGRESSING=False, and DEGRADED=False.

29. Explain the meaning of the AVAILABLE, PROGRESSING, and DEGRADED statuses for a Cluster Operator.

Description: These statuses indicate the operator's ability to manage its component:

- ✓ AVAILABLE (True): The operator is running and the component it manages (its operand) is functional and available according to the operator's checks. This is the desired healthy state.
- ✓ PROGRESSING (True): The operator is actively working to deploy or update its managed component to a desired state. This is expected during cluster upgrades, configuration changes, or initial deployment. It should be temporary; if an operator stays PROGRESSING for an extended period, it might indicate a problem.
- ✓ DEGRADED (True): The operator is encountering errors that prevent it or its managed component from functioning correctly. The component might be unavailable or experiencing significant issues. This status requires immediate investigation as it indicates a problem with a core cluster function.

30. If a Cluster Operator is DEGRADED, what is the first command you would use to investigate?

Command: `oc describe clusteroperator <operator_name>` (e.g., `oc describe co authentication`)

Description: When an operator shows DEGRADED=True (or AVAILABLE=False, or is stuck PROGRESSING), this command is the crucial first step. It provides detailed information, including:

- ✓ Status Conditions: More granular details about *why* the operator is in its current state (e.g., specific error messages, failing checks).
- ✓ Related Objects: References to the operand resources it manages.
- ✓ Events: Recent events associated with the operator, which often contain specific error logs or failure reasons. This helps pinpoint the root cause of the degradation.

31. How can you find the specific version of a deployed Cluster Operator?

Command: `oc get clusteroperator <operator_name> -o jsonpath='{.status.versions[?(@.name=="operator")].version}'`

Description: Cluster Operators manage operands, and both the operator code and the operand code have versions. This command specifically extracts the version of the *operator* controller itself from the operator's status field. Knowing the operator version is useful for checking compatibility or identifying bugs specific to that operator release.

32. How would you typically find and view the logs for the pods managed by a specific Cluster Operator (e.g., the authentication operator)?

Process:

- ✓ Identify Namespace: Operators usually run in dedicated namespaces, often following the pattern `openshift-<operator-name>` (e.g., `openshift-authentication-operator` for the auth operator, `openshift-kube-apiserver-operator` for the Kube API operator). You can often find this in `oc describe co <operator_name>`.
- ✓ Identify Pods: List pods in that namespace, often filtering by a label related to the operator: `oc get pods -n openshift-<operator-name> -l <app=operator-name or similar label>`.
- ✓ View Logs: Use `oc logs` with the pod name and namespace: `oc logs <operator-pod-name> -n openshift-<operator-name> [-f]` (-f to follow logs).

Description: Operator logs contain detailed information about the operator's actions, decisions, and any errors encountered while managing its component. This is often necessary for deep troubleshooting when `oc describe co` doesn't provide enough detail.

33. What command shows the overall installed version of the OpenShift cluster?

Command: `oc get clusterversion`

Description: This command queries the ClusterVersion object, which is a singleton resource (version) that reports the currently installed OpenShift version, the desired version (if an upgrade is in progress), and the overall status of the cluster version/upgrade.

34. How can you view the cluster's update history?

Command: `oc get clusterversion version -o jsonpath='{.status.history}'`

Description: This extracts the history field from the ClusterVersion object's status. It provides a list of previous versions the cluster has run, the state (Completed or Partial), and the start/completion times for each version transition. This is useful for tracking the cluster's upgrade path and identifying when specific versions were installed.

35. How do you check if updates are available for your cluster?

Command: `oc adm upgrade`

Description: For clusters with internet connectivity (or access to a mirrored OpenShift Update Service), this command queries the update service based on the cluster's current version and configured channel. It reports the current version and lists any available updates (newer versions) within that channel, along with recommended update paths if applicable.

36. What is an update channel in OpenShift 4, and how do you check the currently configured channel?

Command to Check: `oc get clusterversion version -o jsonpath='{.spec.channel}'` (or `oc adm upgrade | grep Channel`)

Description: An update channel dictates the stream of OpenShift updates offered to a cluster. Common channels include:

- ✓ `stable-4.x`: Receives generally available (GA) Z-stream (patch) releases for the specified minor version (e.g., 4.14). Recommended for production.
- ✓ `fast-4.x`: Receives GA Z-stream releases slightly earlier than stable. Suitable for environments wanting faster access to patches.
- ✓ `candidate-4.x`: Provides access to pre-release versions (release candidates) for testing before GA. Not for production.
- ✓ `eus-4.x`: Extended Update Support channels, providing patches for specific minor versions for a longer period (requires EUS subscription).
- ✓ The channel determines the upgrade path and stability level of the versions presented to the cluster.

37. How would you assess the health of the core control plane components like the API server and etcd?

Process:

- ✓ **Check Cluster Operators:** The primary method. Check the status of kube-apiserver, etcd, kube-controller-manager, and kube-scheduler operators using `oc get co`. Any DEGRADED or non-AVAILABLE status needs investigation via `oc describe co`.
- ✓ **Check Component Pods:** List pods in the relevant namespaces (e.g., openshift-kube-apiserver, openshift-etcd, openshift-kube-controller-manager, openshift-kube-scheduler) to ensure they are Running and haven't restarted frequently. Check their logs if issues are suspected.
- ✓ **API Responsiveness:** Use basic `oc` commands (like `oc get nodes`) to gauge API server responsiveness. Check API server latency metrics in monitoring.
- ✓ **Etcd Health (Specific):** Use `oc describe co etcd` and potentially `etcdctl` commands (see next question).

Description: The control plane is the brain of the cluster. Assessing its health involves checking the operators managing these components, the component pods themselves, and observing overall API responsiveness.

38. What are the steps to check the health of the etcd cluster specifically?

Steps:

- ✓ Check Operator: `oc get co etcd`. If not Available/False/False, use `oc describe co etcd` to find errors.
- ✓ Check Pods: `oc get pods -n openshift-etcd`. Ensure all pods (typically 3 on the masters) are Running. Check for restarts.
- ✓ Check Endpoints: `oc get endpoints etcd-client -n openshift-etcd -o yaml`. Verify it lists endpoints for all etcd pods.
- ✓ (Advanced) `etcdctl`: If deeper investigation is needed:
 - Exec into an etcd pod: `oc rsh -n openshift-etcd etcd-<master_node_name>`
 - Run health check: `etcdctl endpoint health --cacert /etc/kubernetes/static-pod-resources/etcd-certs/ca.crt --cert /etc/kubernetes/static-pod-resources/etcd-certs/etcd-peer/etcd-peer.crt --key /etc/kubernetes/static-pod-resources/etcd-certs/etcd-peer/etcd-peer.key`
 - Check member list: `etcdctl member list -w table --cacert ... --cert ... --key ...`
- ✓ Check Metrics: Look at etcd performance metrics in monitoring (disk sync times, leader changes, proposal failures).

Description: Etcd health is critical as it stores all cluster state. Checking involves verifying the managing operator, the etcd pods themselves, and potentially using `etcdctl` for direct member status and health checks. Performance metrics are also key indicators.

39. How do you verify the status and health of the internal container image registry?

Steps:

- ✓ Check Operator: `oc get co image-registry`. If not healthy, `oc describe co image-registry`.
- ✓ Check Deployment: `oc get deployment image-registry -n openshift-image-registry`. Ensure it has the desired number of available replicas.
- ✓ Check Pods: `oc get pods -n openshift-image-registry`. Ensure pods are Running and check logs (`oc logs <registry_pod> -n openshift-image-registry`) for errors if needed.
- ✓ Check Storage: If using persistent storage, check the associated PVC status: `oc get pvc -n openshift-image-registry`.
- ✓ (Optional) Test Push/Pull: Try pushing/pulling a test image using `podman` or `docker` logged into the internal registry route (if exposed) or test image pulls within cluster pods.

Description: Ensures the cluster's built-in registry for storing application and S2I images is operational. Problems here affect builds and deployments.

40. How do you verify the status and health of the cluster's Ingress Controllers?

Steps:

- ✓ Check Operator: `oc get co ingress`. If not healthy, `oc describe co ingress`.
- ✓ Check Deployment: `oc get deployment router-default -n openshift-ingress` (or other names if using custom ingress controllers). Ensure available replicas match desired.
- ✓ Check Pods: `oc get pods -n openshift-ingress`. Ensure router pods are Running. Check logs (`oc logs <router_pod> -n openshift-ingress`) for errors (e.g., config reload issues, connection problems).
- ✓ Check Route Status: Check specific application routes (`oc get route <route_name>`) for errors or admission status.
- ✓ Test External Access: Try accessing an application via its Route URL from outside the cluster.

Description: Verifies the components responsible for routing external traffic to internal services are healthy. Problems impact application accessibility.

41. Where can you find the unique Cluster ID for your OpenShift installation?

Command: `oc get clusterversion version -o jsonpath='{.spec.clusterID}'`

Description: Retrieves the globally unique identifier assigned to the cluster during installation. This ID is used for various purposes, including telemetry reporting (if enabled) and identifying the cluster in Red Hat support systems and the OpenShift Cluster Manager portal.

42. How do you check the status of the cluster monitoring stack components (Prometheus, Grafana, Alertmanager)?

Steps:

- ✓ Check Operator: `oc get co monitoring`. If not healthy, `oc describe co monitoring`.
- ✓ Check Pods: List pods in the `openshift-monitoring` namespace. Specifically check for:
 1. Prometheus: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=prometheus`
 2. Alertmanager: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=alertmanager`
 3. Grafana: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=grafana`
 4. Other components like node-exporter, kube-state-metrics, prometheus-adapter.
- ✓ Check UIs: Access the Grafana and Alertmanager routes (`oc get routes -n openshift-monitoring`) to ensure they are responsive.

Description: Ensures the core components responsible for metrics collection, alerting, and visualization are running correctly.

43. If the cluster logging stack is installed, how do you check its overall health?

Steps:

- ✓ Check Operator: `oc get co logging` (if using the Red Hat OpenShift Logging operator). If not healthy, `oc describe co logging`.
- ✓ Check Pods: List pods in the `openshift-logging` namespace. Specifically check for:
 1. Elasticsearch: `oc get pods -n openshift-logging -l component=elasticsearch`
 2. Fluentd (DaemonSet): `oc get pods -n openshift-logging -l component=fluentd`
 3. Kibana: `oc get pods -n openshift-logging -l component=kibana`
- ✓ Check Elasticsearch Health: Use the `curl` command (from Q#135 in the previous doc) or check the Kibana Stack Management UI for cluster health (green/yellow/red).
- ✓ Check Kibana UI: Access the Kibana route (`oc get route kibana -n openshift-logging`) and verify logs are searchable and dashboards load.

Description: Verifies the end-to-end health of the log aggregation pipeline (collection, storage, visualization).

44. How can you generally determine which operator is responsible for managing a specific Custom Resource Definition (CRD)?

Methods:

- ✓ CRD Naming Convention: Often, the CRD's group name hints at the operator (e.g., `console.operator.openshift.io` is managed by the console operator, `etcds.operator.openshift.io` by the etcd operator).
- ✓ `oc describe crd <crd_name>`: While it doesn't explicitly list the operator, the description and related resources might provide clues.
- ✓ Operator Descriptions: Check the descriptions of installed operators (`oc describe co <operator_name>`) - they sometimes list the CRDs they manage.
- ✓ Operator YAML: Look at the Cluster Operator's deployment YAML (`oc get deployment -n openshift-<operator-name> -o yaml`) - the RBAC rules might show which CRDs it has permissions for.
- ✓ Documentation: Operator documentation usually lists the CRDs it introduces and manages.

Description: Understanding which operator controls a CRD is essential when troubleshooting issues related to Custom Resources (CRs) based on that CRD.

45. Under what circumstances might you temporarily set a Cluster Operator's managementState to Unmanaged, and what is the command?

- ✓ Command: `oc patch clusteroperator <operator_name> --type=merge -p '{"spec": {"managementState": "Unmanaged"}}'`
 - ✓ Circumstances: This is an advanced and potentially dangerous operation used *very* sparingly, typically only under the guidance of Red Hat support or during complex manual recovery procedures. Setting an operator to Unmanaged tells it to stop managing its components (operands).
1. Use Case Example: If an operator is misbehaving catastrophically and preventing cluster recovery, support *might* advise setting it to Unmanaged to allow manual intervention on its operands without the operator immediately reverting the changes.
- ✓ Risks: The operator will no longer ensure its components are in the desired state, enforce configurations, or perform updates. This can lead to configuration drift, instability, and prevent future automated management or upgrades until set back to Managed. Never do this unless explicitly instructed and fully understanding the consequences. To revert: `oc patch clusteroperator <operator_name> --type=merge -p '{"spec": {"managementState": "Managed"}}'`.

Storage (PV, PVC, StorageClass)

46. How do you list all Persistent Volume Claims (PVCs) within a specific project? What key information is shown?

Command: `oc get pvc -n <project_name>`

Description: This command retrieves all PVC objects within the specified project (namespace). Key information displayed typically includes:

- ✓ NAME: The name of the PVC.
- ✓ STATUS: The current state of the PVC (e.g., Pending, Bound, Lost). Bound means it's successfully linked to a PV. Pending means it's waiting for a suitable PV to be provisioned or found.
- ✓ VOLUME: The name of the Persistent Volume (PV) the PVC is bound to (if STATUS is Bound).
- ✓ CAPACITY: The amount of storage allocated to the PV bound to this PVC.
- ✓ ACCESS MODES: How the volume can be mounted (e.g., RWO - ReadWriteOnce, ROX - ReadOnlyMany, RWX - ReadWriteMany).
- ✓ STORAGECLASS: The StorageClass requested by the PVC, defining the type of storage.
- ✓ AGE: How long the PVC object has existed.

This overview is crucial for understanding application storage requests and their fulfillment status within a project.

47. What is the difference between a Persistent Volume (PV) and a Persistent Volume Claim (PVC)?

Description: These two objects form the core of Kubernetes/OpenShift persistent storage abstraction:

- ✓ Persistent Volume (PV): Represents a piece of *actual storage* in the cluster (e.g., an NFS share, a vSphere VMDK, an AWS EBS volume, a Ceph RBD volume). PVs are cluster resources, managed by administrators. They have a lifecycle independent of any individual pod and contain details about the storage capacity, access modes, and type. PVs can be provisioned statically (pre-created by an admin) or dynamically (created on-demand by a StorageClass provisioner).
- ✓ Persistent Volume Claim (PVC): Represents a *request for storage* by a user or application within a project. A PVC specifies the desired storage size, access modes, and optionally a specific StorageClass. It acts like a voucher that consumes the resources of a matching PV. Pods mount PVCs, not PVs directly.

Analogy: Think of PVs as the available lockers (storage) in a gym, and PVCs as the request slip a member (application) uses to get assigned a specific locker that meets their size requirements.

48. How can you check if a PVC is Bound or Pending? What does Pending usually indicate?

Command: `oc get pvc <pvc_name> -n <project_name>` (Check the STATUS column).

Description:

- ✓ Bound: This is the desired state. It means the PVC has successfully found and claimed a matching PV (either pre-existing or dynamically provisioned). The application can now use this PVC in its pods.
- ✓ Pending: This indicates the PVC's request cannot currently be fulfilled. Common reasons include:
 1. No Matching PV: For static provisioning, no available PV meets the PVC's requirements (size, access modes, labels).
 2. Dynamic Provisioning Issues: If using a StorageClass, the provisioner might be failing (check StorageClass, CSI driver pods), or there might be insufficient capacity in the underlying storage pool.
 3. StorageClass Not Found: The StorageClass specified in the PVC doesn't exist.
 4. Quota Limits: The project might have hit its storage quota limits.
- ✓ Troubleshooting a Pending PVC usually involves checking `oc describe pvc <pvc_name>` for events and verifying the availability and configuration of PVs and StorageClasses.

49. How do you find the name of the PV that a specific PVC is bound to?

Command: `oc get pvc <pvc_name> -n <project_name> -o jsonpath='{.spec.volumeName}'`

Description: Once a PVC is Bound, its `spec.volumeName` field holds the name of the specific PV it has claimed. This command extracts that field value directly. Knowing the PV name allows you to investigate the underlying storage volume using `oc describe pv <pv_name>`.

50. What details about a PVC can you find using `oc describe pvc`?

Command: `oc describe pvc <pvc_name> -n <project_name>`

Description: This command provides comprehensive details about a PVC, including:

- ✓ Name, Namespace, Labels, Annotations.
- ✓ Status: Current phase (Pending, Bound, Lost).
- ✓ Volume: Name of the bound PV (if Bound).
- ✓ Capacity: Requested and allocated capacity.
- ✓ Access Modes: Requested access modes.
- ✓ StorageClassName: The requested StorageClass.
- ✓ VolumeMode: Filesystem or Block.
- ✓ Mounted By: Lists pods currently mounting this PVC.
- ✓ Events: A crucial section showing a history of actions and potential errors related to the PVC's lifecycle (e.g., provisioning attempts, binding success/failure). Essential for troubleshooting.

51. How do you list all Persistent Volumes (PVs) in the cluster? What statuses can a PV have?

Command: `oc get pv`

Description: Lists all PV objects known to the cluster. Key statuses include:

- ✓ Available: The PV is ready and has not yet been claimed by any PVC.
- ✓ Bound: The PV has been successfully claimed by a PVC and is in use.
- ✓ Released: The PVC that was bound to this PV has been deleted, but the PV itself has not yet been reclaimed (its fate depends on the `persistentVolumeReclaimPolicy`). It's not available for a new PVC yet.
- ✓ Failed: The PV encountered an error during provisioning or operation.

52. What command provides detailed information about a PV, including its reclaim policy and source?

Command: `oc describe pv <pv_name>`

Description: Gives a full picture of the PV, including:

- ✓ Name, Labels, Annotations.
- ✓ Status: Current phase (Available, Bound, Released, Failed).
- ✓ Claim: The namespace/name of the PVC it's bound to (if Bound).
- ✓ Reclaim Policy: Retain, Delete, or Recycle (Recycle is deprecated). Determines what happens to the underlying storage when the PV is released.
- ✓ Access Modes: Supported access modes.
- ✓ Capacity: Total storage size.
- ✓ StorageClassName: The class it belongs to.
- ✓ Source: Crucially, describes the *actual underlying storage* (e.g., NFS server/path, vSphere volume path, AWS EBS Volume ID, Ceph details, CSI driver info).
- ✓ Events: History related to the PV's lifecycle.

53. How do you determine which PVC, if any, a specific PV is currently bound to?

Command: `oc get pv <pv_name> -o jsonpath='{.spec.claimRef}'`

Description: A Bound PV has a claimRef field in its spec that references the PVC claiming it. This command extracts that reference, which includes the PVC's name and namespace. If the PV is not Bound, this field will likely be null or empty.

54. What is a StorageClass in OpenShift/Kubernetes? How do you list available ones?

Command to List: `oc get storageclass` or `oc get sc`

Description: A StorageClass provides a way for administrators to define different "classes" or types of storage they offer. It acts as a template or blueprint for *dynamic provisioning*. When a PVC requests a specific StorageClass, the cluster uses the provisioner defined in that StorageClass to automatically create a matching PV and the underlying storage volume. Key elements defined in a StorageClass include:

- ✓ **provisioner:** Identifies the backend storage system plugin (e.g., `kubernetes.io/vsphere-volume`, `ebs.csi.aws.com`, `openshift-storage.rbd.csi.ceph.com`).
- ✓ **parameters:** Provisioner-specific options (e.g., disk type, filesystem type, encryption).
- ✓ **reclaimPolicy:** Default reclaim policy for PVs created using this class.
- ✓ **volumeBindingMode:** Immediate (provisioning happens right away) or WaitForFirstConsumer (provisioning waits until a pod using the PVC is scheduled, allowing for topology-aware provisioning).

55. How can you find out which provisioner is used by a specific StorageClass?

Command: `oc get sc <storageclass_name> -o jsonpath='{.provisioner}'`

Description: This command directly extracts the provisioner field from the StorageClass definition. This tells you which storage plugin (internal, or more commonly now, a CSI driver) is responsible for creating PVs based on this StorageClass. Knowing the provisioner is key to troubleshooting dynamic provisioning failures, as you'd then check the logs and status of the corresponding provisioner pods.

56. How does OpenShift determine which StorageClass to use if a PVC doesn't specify one? How do you identify the default?

Mechanism: If a PVC is created without explicitly setting the `storageClassName` field, OpenShift/Kubernetes will use the StorageClass marked as the *default* for the cluster. Only one StorageClass can be marked as default. If no default is set and the PVC doesn't specify a class, dynamic provisioning won't occur, and the PVC will only bind to a pre-existing PV that matches its requirements and *doesn't* have a StorageClass specified.

Command to Identify Default: `oc get storageclass -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-class=="true")].metadata.name}'`

Description: This command filters all StorageClasses to find the one with the specific annotation (`storageclass.kubernetes.io/is-default-class: "true"`) that designates it as the default.

57. What are Volume Snapshots, and how do you check if VolumeSnapshotClasses are available?

Command to Check: `oc get volumesnapshotclass`

Description:

- ✓ Volume Snapshots: Provide a Kubernetes-native way to create point-in-time snapshots of Persistent Volumes. This functionality relies on the underlying storage provider and its CSI driver supporting the snapshot feature. Users can create VolumeSnapshot objects referencing a PVC, and the CSI driver interacts with the storage system to create the actual snapshot. These snapshots can later be used to restore data or provision new volumes.
- ✓ VolumeSnapshotClass: Similar to StorageClass, this defines *classes* of snapshots. It specifies the CSI driver responsible for handling the snapshots and may include snapshot-specific parameters.
- ✓ The command `oc get volumesnapshotclass` lists the available classes, indicating if the snapshot feature is configured and available for use with corresponding CSI drivers.

58. How would you check the health of the pods belonging to a specific CSI storage driver?

Process:

- ✓ Identify Driver Name: Determine the name of the CSI provisioner (e.g., from the StorageClass using `oc get sc <sc_name> -o jsonpath='{.provisioner}'`).
- ✓ Find Namespace: CSI drivers usually run their components (controller pods, node daemonsets) in a dedicated namespace, often `openshift-cluster-csi-drivers` or a specific namespace like `openshift-storage` (for ODF) or one named after the driver.
- ✓ List Pods: Use `oc get pods -n <csi_driver_namespace>` and filter using labels associated with the driver (e.g., `app=<driver_name>`).
- ✓ Check Status: Ensure the controller pods and the node daemonset pods are Running without frequent restarts. Check their logs if issues are suspected.

Description: Container Storage Interface (CSI) drivers are the modern way Kubernetes interacts with storage systems. They typically consist of controller components (handling provisioning, attaching) and node components (handling mounting). Checking the health of these pods is crucial for troubleshooting any storage operation failures (provisioning, attaching, mounting, snapshots).

59. A user reports their PVC is stuck in Pending. How would you troubleshoot this?

Troubleshooting Steps:

- ✓ Describe PVC: `oc describe pvc <pvc_name> -n <project_name>`. Pay close attention to the Events section at the bottom. This often contains the specific error message (e.g., "no persistent volumes available for this claim", "storageclass not found", "provisioning failed").
- ✓ Check StorageClass:
 - Does the PVC specify a storageClassName? If so, verify it exists: `oc get sc <storageclass_name>`.
 - If no class is specified, is there a default StorageClass? `oc get sc` (look for (default)).
 - Describe the StorageClass: `oc describe sc <storageclass_name>`. Verify the provisioner is correct.
- ✓ Check Provisioner Health: Check the status of the pods for the CSI driver/provisioner associated with the StorageClass (see Q13). Look for errors in their logs.
- ✓ Check PV Availability (Static Provisioning): If not using dynamic provisioning, check if any Available PVs match the PVC's requirements (size, access modes): `oc get pv`.
- ✓ Check Quotas: Does the project have ResourceQuotas defined for storage? Check if limits have been reached: `oc describe resourcequota -n <project_name>`.
- ✓ Check Underlying Storage: Are there issues in the backend storage system itself (e.g., pool full, connectivity issues)?

60. A PV is showing a Failed status. How would you investigate?

Troubleshooting Steps:

1. Describe PV: `oc describe pv <pv_name>`. Look at the Events section and the Message field in the status for error details. This often indicates *why* it failed (e.g., provisioner error, invalid configuration, underlying storage issue).
2. Check Provisioner Logs: If dynamically provisioned, check the logs of the corresponding CSI driver/provisioner controller pods around the time the PV entered the Failed state.
3. Check Underlying Storage: Investigate the storage system directly using its management tools. Was the volume creation initiated? Did it encounter errors there?
4. Clean Up: Often, a Failed PV needs to be manually deleted (`oc delete pv <pv_name>`). Depending on the cause and reclaim policy, the underlying storage might also need manual cleanup. The associated PVC might need to be deleted and recreated.

61. When might you need to manually create a PV object?

Scenarios: Manual PV creation (Static Provisioning) is less common now with dynamic provisioning via StorageClasses, but it's still used when:

- ✓ Integrating Existing Storage: You have pre-existing storage volumes (e.g., NFS exports, iSCSI LUNs, cloud disks) that you want to make available to the cluster without using a provisioner.
- ✓ Unsupported Provisioner: The storage system doesn't have a dynamic provisioner (or CSI driver) available or configured.
- ✓ Fine-grained Control: You need absolute control over specific volume parameters or lifecycle that dynamic provisioning doesn't offer easily.

Process: You create a YAML manifest defining the PV, specifying its capacity, access modes, reclaim policy, and crucially, the details of the underlying storage source (e.g., NFS server/path, volume IDs). Then apply it with `oc apply -f my-pv.yaml`.

62. Explain the different Persistent Volume Reclaim Policies (Delete, Retain) and their implications.

Description: The `persistentVolumeReclaimPolicy` field on a PV dictates what happens to the *underlying storage volume* in the storage system when the PV becomes Released (i.e., after its bound PVC is deleted).

- ✓ Delete: When the PVC is deleted, the PV object is deleted, *and* OpenShift instructs the storage provisioner (if dynamic) or system admin (if static) to delete the actual storage volume in the backend (e.g., delete the EBS volume, delete the VMDK file, remove the NFS directory contents). Data is lost. This is common for dynamically provisioned volumes where the data lifecycle matches the PVC lifecycle.
- ✓ Retain: When the PVC is deleted, the PV status changes to Released, but the PV object *and the underlying storage volume* are kept. The data remains intact on the volume. An

administrator must manually clean up the PV object (`oc delete pv <pv_name>`) and decide what to do with the underlying storage volume (reuse it, delete it manually). This is safer for critical data, preventing accidental deletion, but requires manual cleanup.

- ✓ Recycle: (Deprecated) Attempted basic cleanup (`rm -rf /volume/*`). Not recommended and often unavailable.

63. How do you check the storage capacity defined for a specific PV?

Command: `oc get pv <pv_name> -o jsonpath='{.spec.capacity.storage}'`

Description: This command extracts the storage value from the `spec.capacity` field of the PV object, showing the total size of the storage volume represented by this PV (e.g., 10Gi, 1Ti).

64. How can you identify which running pods are currently using a specific PVC?

Methods:

1. `oc describe pvc <pvc_name> -n <project_name>`: Look for the Mounted By field near the top. It lists the names of the pods currently mounting this claim.
2. (Manual/Scripted): List all pods in the namespace (`oc get pods -n <project_name> -o yaml` or `-o json`) and inspect the `spec.volumes` section of each pod definition. Look for volumes of type `persistentVolumeClaim` where the `claimName` matches the PVC you're interested in.

Description: This is important for understanding which applications depend on a specific piece of storage, especially before attempting to delete a PVC or perform maintenance that might affect the volume.

Networking (SDN, Services, Routes, NetworkPolicy)

65. How do you check the health of the Cluster Network Operator?

Command: `oc get clusteroperator network` or `oc get co network`

Description: The Cluster Network Operator is responsible for deploying and managing the cluster's core networking components (like OpenShift SDN or OVN-Kubernetes). This command provides a quick health check. Look for `AVAILABLE=True`, `PROGRESSING=False`, `DEGRADED=False`. If the status is not healthy, use `oc describe co network` to get detailed error messages and events.

66. How can you identify whether the cluster is using OpenShift SDN or OVN-Kubernetes as its CNI plugin?

Command: `oc get network.config.openshift.io cluster -o jsonpath='{.spec.networkType}'`

Description: OpenShift 4 supports different network plugins (CNIs). This command queries the cluster-wide network configuration object and extracts the `networkType` field, which will explicitly state either `OpenShiftSDN` or `OVNKubernetes`. Knowing the CNI plugin is crucial as configuration, features, and troubleshooting steps differ between them.

67. How would you check the status of the main SDN/OVN pods running on the cluster nodes?

Commands:

- ✓ For OpenShift SDN: `oc get pods -n openshift-sdn`
- ✓ For OVN-Kubernetes: `oc get pods -n openshift-ovn-kubernetes`

Description: Both OpenShift SDN and OVN-Kubernetes run agent pods (as DaemonSets) on each node to manage pod networking interfaces, implement network policies, and handle traffic routing. These commands list the pods in their respective namespaces. You should check that all pods are in the Running state and have minimal restarts. Problems with these pods can cause network connectivity issues for application pods on the affected nodes.

68. What is a Kubernetes Service? How do you list services in a project?

Command to List: `oc get service -n <project_name>` or `oc get svc -n <project_name>`

Description: A Kubernetes Service is an abstraction that defines a logical set of Pods (usually determined by a label selector) and a policy by which to access them. Services provide a stable endpoint (ClusterIP, NodePort, or LoadBalancer IP) for accessing pods, even as pods are created, destroyed, or rescheduled. They act as an internal load balancer and service discovery mechanism within the cluster. Listing services shows these stable endpoints available within a project.

69. Explain the different Service types (ClusterIP, NodePort, LoadBalancer).

Description:

- ✓ **ClusterIP:** (Default type) Exposes the Service on an internal IP address within the cluster. This IP is only reachable *from within* the cluster. This is the most common type for internal service-to-service communication.
- ✓ **NodePort:** Exposes the Service on each Node's IP address at a static port (the NodePort). A ClusterIP Service (to which the NodePort routes) is automatically created. This allows external traffic to reach the Service by accessing <NodeIP>:<NodePort>. It's often used as a building block for external load balancers or for direct access during development/testing, but less common for production external access due to node IP management challenges.
- ✓ **LoadBalancer:** Exposes the Service externally using a cloud provider's load balancer (e.g., AWS ELB, Azure Load Balancer, GCP Load Balancer) or an on-premise solution like MetalLB. The cloud provider (or MetalLB) creates a load balancer, which then directs traffic to the Service's NodePorts (which are automatically created, along with the ClusterIP). This is the standard way to expose services directly to the internet in supported cloud/on-prem environments. The external IP address of the load balancer is populated in the Service's status.

70. How do you find the internal ClusterIP assigned to a Service?

Command: `oc get svc <service_name> -n <project_name> -o jsonpath='{.spec.clusterIP}'`

Description: This command extracts the stable internal IP address assigned to the Service. Pods within the cluster can use this IP (along with the service port) to reliably connect to the pods backing the Service. If the value is None, it might be a Headless Service.

71. What are Service Endpoints, and how do you check them for a specific Service?

Command to Check: `oc get endpoints <service_name> -n <project_name>` or `oc get ep <service_name> -n <project_name>`

Description: An Endpoints object holds the list of *actual* IP addresses and ports of the healthy Pods that match the Service's label selector. When a connection is made to a Service's ClusterIP, kube-proxy (or OVN) uses the information in the Endpoints object to route the traffic to one of the listed pod IPs. Checking endpoints is crucial for verifying that a Service is correctly selecting healthy backend pods.

72. What does it mean if a Service has no endpoints listed? How would you troubleshoot?

- ✓ **Meaning:** It means the Service selector is not matching any *currently running and ready* pods. Traffic sent to the Service's ClusterIP will fail because there are no backend pods to route to.
- ✓ **Troubleshooting Steps:**
 1. **Check Service Selector:** `oc get svc <service_name> -o jsonpath='{.spec.selector}'`. Note the labels the service is looking for.
 2. **Check Pod Labels:** List pods intended to be part of the service: `oc get pods -n <project_name> -l <key>=<value>` (using the selector labels). Do any pods exist with *exactly* these labels? Check for typos.
 3. **Check Pod Status:** Are the matching pods Running? Are they Ready? Services only include pods that are marked as ready (i.e., passing their readiness probes). Use `oc get pods <pod_name> -o wide` and `oc describe pod <pod_name>` to check status and readiness probe results.
 4. **Check Pod Namespace:** Ensure the pods and the Service are in the same namespace.
 5. **Check Readiness Probes:** If pods are Running but not Ready, investigate why their readiness probes are failing (`oc describe pod`, `oc logs`).

73. What is an OpenShift Route? How does it differ from a Kubernetes Ingress object?

Description:

- ✓ OpenShift Route (route.route.openshift.io): An OpenShift-specific resource that exposes a Service at a publicly accessible hostname (e.g., myapp.apps.mycluster.com). Routes are handled by the built-in OpenShift Ingress Controller (based on HAProxy). Routes offer features like TLS termination (including re-encryption), path-based routing, header manipulation, sticky sessions, and blue-green/canary deployment strategies directly within the Route definition.
- ✓ Kubernetes Ingress (networking.k8s.io/v1/ingress): The standard Kubernetes way to expose HTTP/HTTPS services. It acts as a specification for routing rules. An external Ingress *controller* (like Nginx Ingress, Traefik) is required to actually implement the rules defined in the Ingress object. OpenShift can run other Ingress controllers, but Routes are the native, integrated solution using the default OpenShift Ingress Controller.

Key Differences: Routes are tightly integrated with the OpenShift Ingress Controller and offer more built-in features out-of-the-box compared to the base Kubernetes Ingress specification, which relies more heavily on the capabilities of the specific controller implementation being used. OpenShift can automatically generate hostnames for Routes based on the cluster's ingress domain.

74. How do you list all Routes within a specific project?

Command: `oc get routes -n <project_name>`

Description: This command shows all the Route objects defined in the specified project, listing their names, assigned hostnames, the services they point to, ports, and TLS termination status.

75. How do you find the publicly accessible hostname generated for a Route?

Command: `oc get route <route_name> -n <project_name> -o jsonpath='{.spec.host}'`

Description: This extracts the host field from the Route's specification. This is the DNS hostname that external clients use to access the application exposed by this Route. DNS must be configured (often automatically via wildcard DNS for the *.apps domain) to point this hostname to the OpenShift router's public IP address.

76. How can you tell which Service a particular Route is directing traffic towards?

Command: `oc get route <route_name> -n <project_name> -o jsonpath='{.spec.to.name}'`

Description: A Route must target an internal Kubernetes Service. This command extracts the name of the target Service (spec.to.name) from the Route definition, showing where the Ingress Controller will forward incoming requests that match the Route's host/path.

77. How do you check the status of the Ingress Controller (router) pods?

Commands:

- ✓ `oc get pods -n openshift-ingress`
- ✓ `oc get deployment router-default -n openshift-ingress` (or other deployment names if customized)
- ✓ `oc get co ingress` (Checks the managing operator)

Description: The Ingress Controller runs as regular pods (typically managed by a Deployment) within the openshift-ingress namespace. Checking these pods ensures the router instances responsible for handling Route traffic are running and healthy. Checking the ingress Cluster Operator verifies the overall health of the ingress subsystem.

78. How would you typically find the public IP address used by the OpenShift router/Ingress Controller?

Method: This depends on how the router Service is exposed, which varies by platform:

- ✓ Cloud Provider (AWS, Azure, GCP etc.): The router service is usually of type LoadBalancer. Find its external IP: `oc get svc router-default -n openshift-ingress -o jsonpath='{.status.loadBalancer.ingress[0].ip}'` or `{.status.loadBalancer.ingress[0].hostname}`.
- ✓ Bare Metal (with MetalLB): Similar to cloud providers, check the LoadBalancer service: `oc get svc router-default -n openshift-ingress`. The external IP will be assigned from a MetalLB pool.
- ✓ vSphere/Other UPI: Often uses NodePort or external HAProxy/F5. You might check the NodePort service (`oc get svc router-default -n openshift-ingress`) and then find the public IPs of the worker nodes designated for ingress traffic, or check the configuration of the external load balancer VIP.

Description: This IP address is the external entry point for all Route traffic. DNS records for Route hostnames must resolve to this IP (or the IPs of the load balancer/nodes).

79. What is the purpose of Network Policies?

Purpose: Network Policies are Kubernetes resources that provide network segmentation and control traffic flow at the IP address or port level (OSI layer 3 or 4) between pods within an OpenShift cluster. They act like a distributed firewall for pods.

Function: By default, all pods within a project can communicate with each other. Network Policies allow administrators to define rules specifying which pods (based on labels) are allowed to connect to other pods, or which pods are allowed to receive incoming connections from specific sources (other pods, namespaces, or IP blocks) on particular ports/protocols. They are crucial for implementing security principles like zero-trust networking and least privilege.

80. How do you list all Network Policies applied within a project?

Command: `oc get networkpolicy -n <project_name> or oc get netpol -n <project_name>`

Description: This command shows all the NetworkPolicy objects currently defined within the specified project, giving an overview of the network segmentation rules in place.

81. How can you view the specific rules (selectors, ingress/egress rules) defined in a Network Policy?

Commands:

- ✓ `oc describe networkpolicy <policy_name> -n <project_name>`
- ✓ `oc get networkpolicy <policy_name> -n <project_name> -o yaml`

Description:

- ✓ `describe` provides a human-readable summary of the policy, showing the podSelector (which pods the policy applies to) and summarizing the ingress (incoming) and egress (outgoing) rules (which peers are allowed, on which ports/protocols).
- ✓ `get ... -o yaml` shows the full YAML definition, providing the exact structure and details of the selectors and rules, which is useful for precise understanding or debugging.

82. How would you implement a "default deny" network stance for a project?

Method: Apply a Network Policy that selects all pods in the namespace but allows no ingress traffic.

Example YAML:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {} # Selects all pods in the namespace
  policyTypes:
    - Ingress    # Applies only to ingress rules

    # Implicitly denies all ingress because the 'ingress' list is empty or omitted
```

Apply: `oc apply -f default-deny.yaml -n <project_name>`

Description: This policy selects every pod (`podSelector: {}`) and specifies it applies to Ingress. By *not* defining any ingress rules, it effectively blocks all incoming traffic to all pods from any source (within or outside the namespace), unless allowed by *other* more specific Network Policies. You would then create additional policies to explicitly allow necessary traffic (e.g., allow ingress from the router, allow ingress from specific app tiers).

83. Describe how you would test network connectivity between two specific pods in different projects.

Process:

1. Identify Pod IPs: Get the IP addresses of the source and destination pods: `oc get pod <pod_name> -n <namespace> -o wide`.
2. Exec into Source Pod: Start an interactive shell in the source pod: `oc exec <source_pod_name> -n <source_namespace> -it -- /bin/bash (or /bin/sh)`.
3. Install Test Tools (if needed): The base container image might not have tools like ping, curl, or telnet. You might need to install them temporarily (e.g., `yum install iputils curl telnet` on UBI) if possible, or use a debug container with these tools.
4. Test Connectivity:
 - ✓ Ping: `ping <destination_pod_ip>` (Tests basic ICMP reachability, might be blocked by policies even if TCP works).
 - ✓ Curl/Telnet: `curl -v <destination_pod_ip>:<destination_port>` or `telnet <destination_pod_ip> <destination_port>` (Tests TCP connectivity to a specific port the destination pod should be listening on).
5. Check Network Policies: If connectivity fails, check Network Policies in *both* the source and destination namespaces. Ensure an egress policy in the source namespace allows traffic *to* the destination pod/namespace/IP, AND an ingress policy in the destination namespace allows traffic *from* the source pod/namespace/IP on the required port.

84. What is an Egress IP in OpenShift, and how would you check if one is configured for a project?

Description: Egress IP allows you to assign a specific, predictable source IP address to traffic *originating from* pods within one or more designated projects when that traffic leaves the OpenShift cluster network (e.g., goes to the internet or legacy systems). This is often required by external firewalls that filter based on source IP. OpenShift automatically configures routing on the node hosting the Egress IP to NAT the outgoing traffic. High availability can be configured.

Commands to Check:

- ✓ `oc get egressip` (Cluster-scoped resource defining available Egress IPs and assignments).
- ✓ `oc get hostsubnet` (Check nodes for egressIPs assignment).
- ✓ `oc get netnamespace <project_name> -o yaml` (Check for egressIPs annotation assigned to the project).
- ✓ `oc get egressfirewall -n <project_name>` (Older mechanism, less common now).

85. How can you check the configured MTU for the cluster network?

Command (OVN-Kubernetes): `oc get network.config.openshift.io cluster -o jsonpath='{.status.networkType}{"\n"}{.status.clusterNetworkMTU}'`

Command (OpenShift SDN): Check the `network.config.openshift.io` object or the configuration of the `openshift-sdn pods/operator`. Often inferred or set via CNO config.

Description: Checks the Maximum Transmission Unit (packet size) configured for the pod network overlay. Mismatches between the overlay MTU and the underlying physical network MTU can cause packet fragmentation or loss, leading to performance degradation or connectivity failures.

86. How do you find the defined CIDR block for the Cluster Network (pod network)?

Command: `oc get network.config.openshift.io cluster -o jsonpath='{.spec.clusterNetwork[*].cidr}'`

Description: Retrieves the IP address range(s) from which pods within the cluster are assigned their IP addresses.

87. How do you find the defined CIDR block for the Service Network (ClusterIP range)?

Command: `oc get network.config.openshift.io cluster -o jsonpath='{.spec.serviceNetwork}'`

Description: Retrieves the IP address range from which Services of type ClusterIP are assigned their virtual IP addresses.

88. If using Multus for multiple networks, how would you check the status of its components?

Process:

1. Check CNO: The Cluster Network Operator manages Multus deployment. Check `oc get co network`.
2. Check DaemonSet: Multus typically runs as a DaemonSet to install the Multus CNI binary on nodes. Check pods in `openshift-multus` or `kube-system` (depending on version/config): `oc get pods -n openshift-multus` or `oc get ds -n openshift-multus`.
3. Check NetworkAttachmentDefinitions: List the custom resources that define the additional networks: `oc get network-attachment-definitions -A` or `oc get net-attach-def -A`.

Description: Multus allows pods to connect to multiple networks simultaneously (e.g., the default pod network plus an SR-IOV network). Checking involves verifying the Multus plugin deployment and the definitions of the additional networks.

89. How would you view the logs for the core OVN or SDN components on a specific node?

Method: Use `oc debug node/` to get shell access, then use `journalctl`.

Commands (inside `oc debug node/... pod`):

- ✓ OVN-Kubernetes: `chroot /host journalctl -u ovn-controller` (for OVN controller) or `chroot /host journalctl -u ovs-vsitchd / ovsdb-server` (for Open vSwitch).
- ✓ OpenShift SDN: `chroot /host journalctl -u atomic-openshift-node` (Older versions) or check the specific `openshift-sdn` container logs via `crictl` logs or `journalctl` filtering.

Description: Accessing the detailed logs of the node-level networking agents (OVN controller, OVS, or SDN agent) is essential for deep troubleshooting of pod networking issues, policy enforcement problems, or overlay network failures on that specific node.

90. How can you test DNS resolution using the cluster's internal DNS service from within a pod?

Method: Exec into a pod and use a DNS lookup tool (`dig`, `nslookup`) directed specifically at the cluster DNS service IP.

Commands (inside `oc exec <any_pod> ...`):

1. Find DNS Service IP: `CLUSTER_DNS_IP=$(oc get svc -n openshift-dns dns-default -o jsonpath='{.spec.clusterIP}')` (Run this outside the pod or pass it in).
2. Use `dig`: `dig @$CLUSTER_DNS_IP <service_name>.<project_name>.svc.cluster.local`
3. Use `nslookup`: `nslookup <service_name>.<project_name>.svc.cluster.local $CLUSTER_DNS_IP`

Description: This bypasses the pod's local `/etc/resolv.conf` settings and directly queries the CoreDNS service responsible for internal cluster name resolution. It helps isolate whether a DNS issue lies with the pod's configuration or the central DNS service itself. Ensure the pod's container image has `dig` or `nslookup` installed (often in `bind-utils` or `dnstools` packages).

Security (SCC, RBAC, Secrets, Certificates)

91. What are Security Context Constraints (SCCs) in OpenShift? Name a few standard ones.

Description: Security Context Constraints (SCCs) are OpenShift-specific resources that control the permissions and capabilities a pod can request or execute within the cluster. They act as a gatekeeper for security-sensitive settings defined in a pod's `securityContext` (e.g., running as root, using host network, specific Linux capabilities, volume types). When a pod is scheduled, OpenShift checks if the pod's requested security settings are allowed by any of the SCCs granted to the pod's Service Account. If no matching SCC allows the request, the pod will fail to start.

Standard SCCs:

- ✓ restricted (formerly restricted-v2): The most restrictive default SCC, applied to most regular users and service accounts. Disallows running as root, host access, privileged containers, etc.
- ✓ nonroot (formerly nonroot-v2): Requires pods to run with a non-root UID, but is slightly less restrictive than restricted in other areas.
- ✓ anyuid: Allows pods to run with any UID (including root/UID 0), but still restricts other privileged settings.
- ✓ privileged: The least restrictive SCC, granting almost all capabilities, including running privileged containers and accessing the host filesystem/network. Access is tightly controlled and usually reserved for cluster infrastructure pods.
- ✓ hostnetwork, hostmount-anyuid, hostaccess: Grant specific host access permissions.

92. How do you list all available SCCs in the cluster?

Command: `oc get scc`

Description: This command lists all the Security Context Constraint objects defined in the cluster, showing their names and some basic settings (like whether privileged containers are allowed, default add/drop capabilities). It provides an overview of the different security profiles available.

93. How can you view the specific permissions and settings defined within an SCC like restricted?

Command: `oc describe scc restricted` or `oc get scc restricted -o yaml`

Description:

- ✓ describe gives a human-readable summary of the SCC's settings, including allowed capabilities, volume types, SELinux context, RunAsUser strategy, FSGroup strategy, supplemental groups, priority, required drop capabilities, etc.
- ✓ get ... -o yaml provides the full YAML definition, showing the precise configuration of every field within the SCC object. This is useful for understanding the exact constraints it enforces.

94. How do you determine which SCCs a specific service account is allowed to use?

Command: `oc adm policy scc-subject-review -z <service_account_name> -n <project_name>`

Description: This command checks the RBAC permissions (Roles/ClusterRoles bound to the service account and its groups) and determines which SCCs the specified service account (-z <name>) in the given namespace (-n <namespace>) is authorized to use. OpenShift will try to validate a pod against the allowed SCCs in order of priority (most restrictive first usually).

Alternatively, you can check which users/groups can use a *specific* SCC: `oc adm policy who-can use scc <scc_name>`.

95. What is the command to grant a service account access to a specific SCC? Why should this be done cautiously?

Command: `oc adm policy add-scc-to-user <scc_name> -z <service_account_name> -n <project_name>`

Description: This command directly binds an SCC to a specific service account within a namespace.

Caution: Granting access to less restrictive SCCs (like `anyuid`, `hostaccess`, or especially privileged) significantly increases the potential security risk if a pod running under that service account is compromised. It bypasses many default security protections. This should only be done when absolutely necessary for the application's function and after carefully evaluating the security implications. Always grant the *least permissive* SCC that meets the pod's requirements.

96. Explain the concept of Role-Based Access Control (RBAC) in OpenShift/Kubernetes.

Concept: RBAC is the standard mechanism for controlling *who* (Users, Groups, Service Accounts - called "Subjects") can perform *what actions* (Verbs like `get`, `list`, `create`, `delete`, `patch`) on *which resources* (like pods, deployments, secrets, nodes) within the cluster or specific projects (namespaces).

Components: RBAC relies on four main object types:

- ✓ Role / ClusterRole: Define a set of permissions (rules combining verbs and resources). Roles are namespaced, ClusterRoles are cluster-wide.
- ✓ RoleBinding / ClusterRoleBinding: Grant the permissions defined in a Role or ClusterRole to specific Subjects. RoleBindings operate within a namespace, ClusterRoleBindings operate cluster-wide.

97. What is the difference between a Role and a ClusterRole?

Description:

- ✓ Role: Contains rules that grant permissions *within a specific namespace*. A Role can only grant access to namespaced resources (like pods, deployments, secrets within its namespace) or non-resource URLs within that namespace. It cannot grant access to cluster-scoped resources (like nodes, clusterroles, sccs) or resources in other namespaces.
- ✓ ClusterRole: Contains rules that can grant permissions *cluster-wide*. It can grant access to namespaced resources across *all* namespaces, cluster-scoped resources (like nodes, persistentvolumes, clusterroles, sccs), or non-resource URLs (`/healthz`, `/version`).

98. What is the difference between a RoleBinding and a ClusterRoleBinding?

Description:

- ✓ RoleBinding: Grants the permissions defined in a Role or a ClusterRole to Subjects (users, groups, service accounts) *within a specific namespace*. If a RoleBinding uses a ClusterRole, it only grants the permissions defined in that ClusterRole *for resources within the RoleBinding's namespace*.

- ✓ ClusterRoleBinding: Grants the permissions defined in a ClusterRole to Subjects *across the entire cluster*. This is used for granting cluster-wide permissions, like cluster administration or access to cluster-scoped resources.

99. How do you list all Roles defined within a specific project?

Command: `oc get roles -n <project_name>`

Description: This command retrieves all the Role objects that exist within the specified namespace (<project_name>). These define sets of permissions scoped to that project.

100. How do you list all ClusterRoles defined in the cluster?

Command: `oc get clusterroles`

Description: This command retrieves all ClusterRole objects defined cluster-wide. This includes default roles (like cluster-admin, admin, edit, view) and any custom cluster roles created by administrators or operators.

101. How can you inspect the specific API permissions (verbs, resources) granted by a Role or ClusterRole?

Command:

- ✓ `oc describe role <role_name> -n <project_name>`
- ✓ `oc describe clusterrole <clusterrole_name>`

Description: The describe command provides a human-readable summary of the rules section within the Role or ClusterRole. It lists the allowed API Resources (like pods, services, nodes), Non-Resource URLs, and the permitted Verbs (like get, list, watch, create, update, patch, delete) for each. This clearly shows what actions the role allows.

102. How do you check which users, groups, or service accounts are bound to a specific Role within a project?

Command: `oc get rolebinding -n <project_name> -o wide` (Inspect bindings referencing the role) or `oc describe rolebinding <rolebinding_name> -n <project_name>`

Description: You need to look at RoleBinding objects within the project.

- ✓ List all bindings (`oc get rolebindings -n <project_name>`) and find the ones where the ROLE column matches the Role you're interested in. The USER, GROUP, and SERVICE ACCOUNT columns (in `-o wide` or `describe`) show the bound subjects.
- ✓ Alternatively, if you know the binding name, describe it to see the Role it references and the Subjects it applies to.

103. How do you check which subjects are bound to a specific ClusterRole cluster-wide?

Command: `oc get clusterrolebinding -o wide` (Inspect bindings referencing the ClusterRole) or `oc describe clusterrolebinding <clusterrolebinding_name>`

Description: Similar to RoleBindings, you list ClusterRoleBinding objects (`oc get clusterrolebindings`) and find those referencing the ClusterRole in question (check the ROLE column or roleRef field in describe or YAML output). The Subjects section of the binding shows the users, groups, or service accounts granted those cluster-wide permissions.

104. How can you verify if a particular user has the permission to perform a specific action (e.g., delete pods) in a certain project?

Command: `oc auth can-i <verb> <resource> -n <project_name> --as <user_name>` (e.g., `oc auth can-i delete pods -n my-app-dev --as john.doe`)

Description: This is a direct authorization check. It simulates the action request as the specified user (`--as`) and tells you (yes or no) if their combined RBAC permissions allow them to perform that specific verb on that resource within the given namespace. It's very useful for quickly verifying permissions without needing to trace through all role bindings.

105. What command grants a user the standard edit role within a project?

Command: `oc policy add-role-to-user edit <user_name> -n <project_name>`

Description: This command creates or updates a RoleBinding within the specified project (`-n`). It binds the default edit ClusterRole (which allows modifying most standard application resources but not RBAC rules or quotas) to the specified user (`<user_name>`). This is a common way to give developers permissions to manage their applications within a project.

106. How do you remove a user's role binding from a project?

Command: `oc policy remove-role-from-user <role_name> <user_name> -n <project_name>`

Description: This command finds the RoleBinding that grants the specified `<role_name>` to the specified `<user_name>` within the project (`-n`) and removes that user from the binding's subjects list. If the user was the only subject, the binding might be deleted. This effectively revokes those specific permissions from the user within that project.

107. What is a Kubernetes Secret used for? How do you list secrets in a project?

Command to List: `oc get secrets -n <project_name>`

Description: A Secret is a Kubernetes object designed to store small amounts of sensitive data, such as passwords, OAuth tokens, SSH keys, TLS certificates, or API keys. Storing this information in Secrets is more secure and flexible than hardcoding it into pod definitions or container images. Secrets are stored (by default) base64 encoded in etcd, and potentially encrypted at rest if etcd encryption is enabled. Pods can access secrets as mounted volumes or environment variables. Listing secrets shows the available secret objects within a project.

108. How can you view the decoded data stored within a Secret? What precautions are needed?

Command: `oc get secret <secret_name> -n <project_name> -o jsonpath='{.data}' | jq 'map_values(@base64d)'` (Requires jq utility)

- ✓ Alternatively, get the YAML (-o yaml), copy a base64 encoded value, and decode it manually: `echo "<base64_encoded_value>" | base64 --decode`

Precautions:

- ✓ Permissions: Viewing secrets requires appropriate RBAC permissions (get verb on secrets).
- ✓ Sensitivity: The decoded data is sensitive! Avoid displaying it in shared terminals, logs, or scripts.
- ✓ Audit Logging: Accessing secrets is typically audited if audit logging is enabled. Be mindful that your access is likely recorded.
- ✓ Need-to-Know: Only view secret data if absolutely necessary for troubleshooting or configuration.

109. What is a Service Account? How do you list them in a project?

Command to List: `oc get serviceaccount -n <project_name>` or `oc get sa -n <project_name>`

Description: A Service Account provides an identity for *processes running inside pods* to interact with the Kubernetes API server or external services. When a pod needs to talk to the API (e.g., to list other pods, modify resources), it authenticates using the token associated with its Service Account. Each namespace has a default service account, but it's best practice to create dedicated service accounts for applications with specific RBAC permissions assigned (principle of least privilege).

110. How would you check the expiration date and issuer of the cluster's API server certificate?

Command: `echo | openssl s_client -connect $(oc whoami --show-server | cut -d':' -f 2 | sed 's/\\\\/\\/'):443 2>/dev/null | openssl x509 -noout -text | grep -E 'Issuer:|Not After'`

Description: This command connects to the Kubernetes API server's secure port (usually 443), retrieves its TLS certificate using `openssl s_client`, pipes the certificate details to `openssl x509` for parsing, and then filters the output to show the Issuer (who signed the certificate, often an internal CA) and the Not After field (the expiration date). Monitoring certificate expiration is crucial for cluster stability.

111. How would you check the expiration date and issuer of the default Ingress (router) certificate?

Command: `echo | openssl s_client -connect $(oc get route console -n openshift-console -o jsonpath='{.spec.host}' | sed 's/console-//'):443 -servername $(oc get route console -n openshift-console -o jsonpath='{.spec.host}') 2>/dev/null | openssl x509 -noout -text | grep -E 'Issuer:|Not After'` (Uses console route as an example to find the apps domain)

Description: This command finds the hostname of a known route (like the console route) to determine the base *.apps domain, connects to the Ingress Controller (router) on port 443 using that domain (important for SNI), retrieves the default TLS certificate presented by the router, and displays its Issuer and Expiration Date (Not After). This checks the validity of the certificate securing external application access via Routes.

112. What are Certificate Signing Requests (CSRs) used for in OpenShift, and how do you list them?

Command to List: `oc get csr`

Description: CSRs are the mechanism by which clients (primarily Kubelets on nodes) request TLS certificates from the cluster's internal Certificate Authority (managed by the kube-controller-manager). When a new node joins or an existing node needs to renew its certificate, its Kubelet creates a CSR object. The cluster then validates and approves (usually automatically for nodes) the CSR, and the certificate is issued. Listing CSRs shows pending, approved, or denied requests.

113. Under what circumstances might you need to manually approve a CSR? What is the command?

Command: `oc adm certificate approve <csr_name>`

Circumstances: Manual approval is generally *not* required for node Kubelet certificates in a standard OCP 4 installation, as this is handled by automated approvers. However, you *might* need manual approval if:

- ✓ Automated approval is misconfigured or failing.
- ✓ You are using CSRs for other custom purposes (less common).
- ✓ Troubleshooting certificate issuance problems under guidance from support.

Caution: Only approve CSRs you trust and understand. Approving a malicious CSR could compromise cluster security.

114. How can you check the configured audit policy for the Kubernetes API server?

- ✓ **Method:** Audit configuration is part of the API server's configuration, managed by the kube-apiserver-operator.
- ✓ **Command:** Check the apiserver/cluster resource: `oc get apiserver cluster -o jsonpath='{.spec.audit}'`. This shows the high-level policy profile (Default, WriteRequestBodies, AllRequestBodies, None).
- ✓ **For detailed policy:** The actual policy file might be referenced in the operator config or mounted directly into the API server pods. You might need to inspect the kube-apiserver-operator config or the static pod manifest on master nodes (`oc debug node/...`) to find the exact policy file path and content if a custom policy is used.
- ✓ **Description:** Audit logging records actions performed against the Kubernetes API. The audit policy defines *what* events are logged (e.g., metadata only, requests, responses) and at *what level* (e.g., metadata, request, requestResponse). Checking the policy helps understand the scope and detail of audit logging.

115. How would you typically find the location of API server audit logs on the master nodes?

Method: Requires access to the master nodes, usually via `oc debug node/<master_node_name>`.

Steps (inside debug pod):

1. `chroot /host`
2. Inspect the API server static pod manifest: `cat /etc/kubernetes/manifests/kube-apiserver-pod.yaml`
3. Look for flags like `--audit-log-path=`, `--audit-webhook-config-file=`, or `--audit-policy-file=`. The `--audit-log-path` flag directly specifies the log file location on the host node (e.g., `/var/log/kube-apiserver/audit.log`). If a webhook is used, logs are sent to a remote server defined in the webhook config file.

Description: Audit logs contain sensitive records of API activity. Finding their location on the master nodes (or the webhook configuration) is necessary for security analysis, compliance checks, or detailed troubleshooting.

116. How do you list the Identity Providers (IDPs) configured for cluster authentication?

Command: `oc get oauth cluster -o jsonpath='{.spec.identityProviders}'`

Description: This command queries the central OAuth configuration object and extracts the list of configured IDPs. This shows how users can log in to the cluster (e.g., `htpasswd`, `ldap`, `github`, `oidc`). Each entry in the list contains the name and configuration details for that specific IDP.

Monitoring & Logging

117. How do you typically access the cluster's central Grafana monitoring dashboards?

Method: Access via its Route.

Command to find URL: `oc get route grafana -n openshift-monitoring -o jsonpath='{\"https://\"}{.spec.host}{\"\\n\"}'`

Description: OpenShift includes a pre-configured Grafana instance providing dashboards for visualizing cluster and node metrics collected by Prometheus. This command finds the Route exposing the Grafana web UI. You access this URL in a browser and typically log in using your OpenShift credentials (via OAuth integration).

118. What is Alertmanager, and how do you access its UI?

Command to find URL: `oc get route alertmanager-main -n openshift-monitoring -o jsonpath='{\"https://\"}{.spec.host}{\"\\n\"}'`

Description: Alertmanager is a component of the monitoring stack responsible for handling alerts sent by Prometheus. It deduplicates, groups, and routes alerts to configured receivers (like email, Slack, PagerDuty). It also manages silencing (muting) alerts. The UI allows you to view currently firing alerts, check receiver configurations, and manage silences. Accessing its Route URL provides access to this UI.

To get the OpenShift Free downloads, please visit: <https://assistedcloud.com/free-downloads/>

119. How can you check which alerts are currently firing in the cluster?

Methods:

- ✓ Alertmanager UI: Access the Alertmanager UI (see previous question). The main page displays currently active (firing) alerts.
- ✓ Grafana Dashboards: Some default Grafana dashboards display firing alerts.
- ✓ Prometheus UI (Advanced): Access the Prometheus UI (prometheus-k8s service/route in openshift-monitoring) and navigate to the "Alerts" section.
- ✓ oc Command (Indirect): `oc get prometheusrules -A -o jsonpath='{range .items[*]}{.metadata.namespace}/{.metadata.name}\n}{range .spec.groups[*].rules[?(@.state=="firing")]}{ " ALERT "}{.alert}\n}{end}{end}'`
(This lists rules, not active alert instances precisely, but can indicate firing rules).
A more direct way is `oc get monitoring.coreos.com/alerts -A`.

Description: Identifying active alerts is crucial for proactive cluster management. Alertmanager is the primary tool for viewing and managing these active alerts.

120. Explain how you would temporarily silence a specific, known alert.

Method: Use the Alertmanager UI.

Process:

- ✓ Access the Alertmanager UI (`oc get route alertmanager-main -n openshift-monitoring`).
- ✓ Find the firing alert you want to silence.
- ✓ Click the "Silence" button associated with the alert or group.
- ✓ The UI will pre-fill matchers based on the alert's labels (e.g., `alertname="KubePodCrashLooping", namespace="my-app"`). Adjust matchers if needed to broaden or narrow the silence scope.
- ✓ Set a duration for the silence (e.g., 1 hour, 2 days). Add a comment explaining the reason.
- ✓ Click "Create".

Description: Silencing temporarily stops Alertmanager from sending notifications for alerts matching specific criteria. This is useful during planned maintenance, for known issues being addressed, or to reduce noise from flapping alerts while investigating the root cause. Silences are temporary and expire automatically.

121. How do you check the status of the core Prometheus pods responsible for cluster monitoring?

Command: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=prometheus`

Description: The core cluster monitoring relies on a highly available Prometheus deployment (typically 2 replicas: `prometheus-k8s-0` and `prometheus-k8s-1`). This command lists these specific pods within the `openshift-monitoring` namespace. Check that they are Running and have minimal restarts. These pods scrape metrics, evaluate alerting rules, and store time-series data.

122. How do you check the status of the Alertmanager pods?

Command: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=alertmanager`

Description: Alertmanager typically runs as a StatefulSet (e.g., `alertmanager-main-0`, `-1`, `-2`) for high availability. This command lists the Alertmanager pods. Ensure they are Running and stable. Problems here can prevent alert notifications from being delivered.

123. How do you check the status of the Grafana pods?

Command: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=grafana`

Description: This command lists the pod(s) running the Grafana web UI and backend. Ensure it's Running to allow users access to monitoring dashboards.

124. What is user workload monitoring, and how would you check the status of its components if enabled?

Description: User Workload Monitoring (UWM) is an optional feature in OpenShift that allows developers and application owners to monitor their own applications within their projects using the same Prometheus-based stack used for core cluster monitoring. It deploys a separate Prometheus instance (`prometheus-user-workload`) that discovers and scrapes metrics from user-defined `ServiceMonitor` and `PodMonitor` resources within allowed namespaces.

Command to Check Status: `oc get pods -n openshift-user-workload-monitoring`

Details: This command lists the pods specific to UWM, primarily the `prometheus-user-workload-*` pods and potentially `thanos-ruler-user-workload-*` pods if configured. Check that these are Running.

125. How can you verify if user workload monitoring is enabled for the cluster?

Method: Check the cluster monitoring configuration ConfigMap.

Command: `oc get configmap cluster-monitoring-config -n openshift-monitoring -o yaml`

Description: Look inside the `data.config.yaml` section of this ConfigMap for a setting like `enableUserWorkload: true`. If this key exists and is set to true, UWM is enabled. The presence and health of the `openshift-user-workload-monitoring` namespace and its pods is also a strong indicator.

126. Describe how you could query a specific metric directly from the cluster's Prometheus instance.

Method: Use the Prometheus web UI via port-forwarding or a Route (if exposed).

Process:

- ✓ Expose Prometheus:
 - Port-forward: `oc port-forward -n openshift-monitoring svc/prometheus-k8s 9090:9090` (Access via `http://localhost:9090`).
 - (Less common) Create a temporary Route: `oc expose svc/prometheus-k8s -n openshift-monitoring` (Remember to delete it afterwards).
- ✓ Access UI: Open the Prometheus URL (`localhost:9090` or the Route URL) in a browser.
- ✓ Query: Use the "Graph" or "Table" view. Enter a PromQL (Prometheus Query Language) query in the expression bar (e.g., `node_memory_MemAvailable_bytes, sum(rate(container_cpu_usage_seconds_total{namespace="my-app"}[5m])) by (pod)`).
- ✓ Execute: Click "Execute".

Description: Allows direct interaction with the Prometheus query engine to retrieve specific time-series data, test alert rule expressions, or perform advanced analysis beyond the standard Grafana dashboards. Requires understanding PromQL.

127. If cluster logging is installed, how do you typically access the Kibana UI?

Method: Access via its Route.

Command to find URL: `oc get route kibana -n openshift-logging -o jsonpath='{ "https://" }{.spec.host}{ " \n" }'`

Description: The OpenShift Logging stack (based on Elasticsearch, Fluentd, Kibana - EFK) includes Kibana as the web UI for searching, visualizing, and analyzing the aggregated logs. This command finds the Route exposing the Kibana UI. Login is typically via OpenShift credentials.

128. How do you check the status of the Elasticsearch pods used for logging?

Command: `oc get pods -n openshift-logging -l component=elasticsearch` (or similar label depending on deployment method).

Description: Elasticsearch runs as a StatefulSet to store and index log data. This command lists the Elasticsearch pods. Check that they are Running, stable (minimal restarts), and that the desired number of replicas are present. Health issues here impact log storage and search capabilities.

129. How do you check the status of the Fluentd pods responsible for collecting logs from nodes?

Command: `oc get pods -n openshift-logging -l component=fluentd` (or similar label). Check the DaemonSet: `oc get ds fluentd -n openshift-logging`.

Description: Fluentd runs as a DaemonSet, meaning one pod runs on each eligible node in the cluster. These pods collect container and node logs and forward them to Elasticsearch. This command lists these collector pods. Ensure one is Running on each expected node. Problems here mean logs from specific nodes might be missing.

130. How do you check the status of the Kibana pods?

Command: `oc get pods -n openshift-logging -l component=kibana` (or similar label).

Description: This command lists the pod(s) running the Kibana web UI and backend service. Ensure it's Running for users to access the log exploration interface.

131. How would you check the health status (e.g., green, yellow, red) of the Elasticsearch cluster used for logging?

Methods:

- ✓ Kibana UI: Navigate to "Stack Management" -> "Index Management" or "Overview" within Kibana. It usually displays the cluster health status prominently.
- ✓ Direct API Query (via pod exec):

```
oc exec <any_es_pod_name> -n openshift-logging -c elasticsearch -- curl -s -k -u elastic:${(oc get secret elasticsearch -n openshift-logging -o jsonpath='{.data.admin-password}' | base64 -d)} "https://localhost:9200/_cluster/health?pretty"
```

Description: Elasticsearch reports its health as:

- ✓ green: All primary and replica shards are allocated and active. Healthy.
- ✓ yellow: All primary shards are active, but some replica shards are not allocated (e.g., not enough nodes). Cluster is functional but lacks full redundancy.
- ✓ red: Some primary shards are not allocated. Cluster is non-functional, data might be missing, searches will likely fail. Requires immediate investigation.

132. How can you view the logs of the Fluentd log collector running on a particular node?

Process:

- ✓ Find Pod: Identify the Fluentd pod running on the target node: `oc get pods -n openshift-logging -o wide --field-selector spec.nodeName=<node_name> -l component=fluentd`
- ✓ Get Logs: Use `oc logs` with the pod name found: `oc logs <fluentd_pod_name> -n openshift-logging`

Description: Checking the logs of a specific Fluentd pod is essential for troubleshooting log collection issues originating from that particular node. Logs might show errors connecting to Elasticsearch, parsing specific log formats, or reading log files.

133. How would you troubleshoot if logs from applications are not appearing in Kibana?

Troubleshooting Steps:

- ✓ Check Fluentd on Node: Is the Fluentd pod running on the node where the application pod resides (`oc get pods -n openshift-logging -o wide`)? Check its logs (`oc logs ...`) for errors related to that application's logs or connection issues to Elasticsearch.
- ✓ Check Elasticsearch Health: Is the ES cluster healthy (green/yellow)? (`oc exec ... _cluster/health` or Kibana UI). If red/yellow, logs might not be indexing correctly.
- ✓ Check Elasticsearch Disk Space: Is ES running out of disk space? (`oc get pvc -n openshift-logging`). Full disks prevent indexing.
- ✓ Check Kibana Index Pattern: In Kibana -> Stack Management -> Index Patterns, ensure the correct index pattern (e.g., `app-*`, `infra-*`) is configured and includes the relevant indices. Refresh the pattern if needed.
- ✓ Check Time Range in Kibana: Ensure the time filter selected in Kibana covers the period when the logs were generated.
- ✓ Check Application Logging: Is the application actually writing logs to stdout/stderr? (`oc logs <app_pod_name> -n <app_namespace>`). Fluentd primarily collects these standard streams.
- ✓ Check Index Mappings/Templates (Advanced): Occasionally, incorrect index mappings can cause indexing failures for specific log formats.

134. How do you monitor the disk usage of the Elasticsearch cluster?

Methods:

- ✓ Check PVCs: `oc get pvc -n openshift-logging -l component=elasticsearch`. This shows the capacity and status of the Persistent Volume Claims used by ES pods.
- ✓ Kibana UI: Stack Management -> Index Management often shows disk usage per node and shard information.

- ✓ ES API: Use `_cat/allocation?v` or `_cluster/stats` APIs via curl inside an ES pod (similar to health check command) to get detailed node disk usage.
- ✓ Prometheus Metrics: Query metrics like `elasticsearch_filesystem_data_available_bytes` if ES metrics export is configured.

Description: Monitoring Elasticsearch disk usage is critical because it will stop indexing new logs if it runs low on disk space, leading to log loss. Regular monitoring and capacity planning (or index lifecycle management) are essential.

135. What is the role of the node-exporter pods in the monitoring stack? How do you check their status?

Command to Check Status: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=node-exporter`. Check the DaemonSet: `oc get ds node-exporter -n openshift-monitoring`.

Role: node-exporter is an official Prometheus exporter that runs as a DaemonSet on every node in the cluster. Its role is to collect hardware and OS-level metrics from the host node it's running on (CPU usage, memory usage, disk I/O, network statistics, filesystem usage, etc.). Prometheus then scrapes these metrics from each node-exporter pod.

Description: These pods provide the fundamental host-level metrics visible in Grafana dashboards for node performance analysis. Ensuring they are running correctly on all nodes is vital for complete node monitoring coverage.

136. What is the role of the kube-state-metrics pods? How do you check their status?

Command to Check Status: `oc get pods -n openshift-monitoring -l app.kubernetes.io/name=kube-state-metrics`. Check the Deployment: `oc get deployment kube-state-metrics -n openshift-monitoring`.

Role: kube-state-metrics listens to the Kubernetes API server and converts information about the state of Kubernetes objects (like Deployments, Pods, Nodes, Services, PVCs) into metrics that Prometheus can scrape. For example, it generates metrics for the number of desired vs. available replicas in a Deployment, pod statuses, PVC statuses, node conditions, etc.

Description: While node-exporter provides OS/hardware metrics, kube-state-metrics provides metrics about the *state of Kubernetes objects*. Both are essential for comprehensive cluster monitoring dashboards and alerting rules. Ensuring kube-state-metrics is running correctly is vital for alerts based on deployment status, pod failures, etc.

Upgrades & Maintenance

137. How do you check which OpenShift versions are available for upgrade within your configured channel?

Use the `oc adm upgrade` command without any arguments. This command queries the OpenShift Update Service (OSUS) based on your cluster's currently configured update channel. It will display the current cluster version and list any available updates (versions) within that channel, along with their status (e.g., Recommended, Available).

```
oc adm upgrade
```

Key Output: Look for lines like `Updates:` which list available versions, and `Channel:` which shows the currently configured update stream (e.g., `stable-4.12`, `fast-4.13`).

138. What is the command to initiate a cluster upgrade to a specific version or the latest recommended one?

To upgrade to the latest recommended version within the current channel (as shown by `oc adm upgrade`):

```
✓ oc adm upgrade --to-latest
```

To upgrade to a specific available version listed by `oc adm upgrade`:

```
✓ oc adm upgrade --to=<version_number>
```

```
✓ # Example: oc adm upgrade --to=4.12.15
```

Important: Always review the release notes for the target version before initiating an upgrade. Ensure cluster health and prerequisites are met.

139. How can you monitor the real-time progress of an ongoing cluster upgrade?

There are several ways:

- ✓ **oc adm upgrade:** Running this command while an upgrade is in progress will show the target version and often indicate which component (like a specific Cluster Operator or Machine Config Pool) is currently being updated.
- ✓ **oc get clusterversion:** This shows the overall status, the target version (`spec.desiredUpdate`), and the history of applied updates (`status.history`). The `status.conditions` will indicate if the upgrade is Progressing.

```
oc get clusterversion version -o yaml
```

```
# Watch the status: watch oc get clusterversion
```

- ✓ **oc get clusteroperator or oc get co:** Monitor the status of individual operators. During an upgrade, many operators will temporarily enter the `Progressing=True` state. Watch for any operators becoming `DEGRADED=True`.

```
watch oc get co
```

- ✓ **oc get machineconfigpool or oc get mcp:** Monitor the status of node pools. They will show UPDATING=True as nodes within the pool are rebooted with the new configuration. Check the UPDATEDMACHINECOUNT, READYMACHINECOUNT, and MACHINECOUNT columns.

```
watch oc get mcp
```

140. Is it possible to pause an ongoing cluster upgrade? If so, how, and why might you do it?

Yes, it is possible to pause an ongoing cluster upgrade, but it should be done with caution and typically only when troubleshooting a blocking issue.

How: Patch the clusterversion object:

```
oc patch clusterversion version --type=merge -p '{"spec":{"paused": true}}'
```

Why: You might pause an upgrade if:

- ✓ A critical Cluster Operator becomes DEGRADED and blocks progress, requiring investigation and manual intervention.
- ✓ An unexpected issue arises in the infrastructure or critical applications during the upgrade process that needs immediate attention before proceeding.
- ✓ You need to perform emergency maintenance unrelated to the upgrade itself.

Resuming: To resume the upgrade after resolving the issue:

- ✓ `oc patch clusterversion version --type=merge -p '{"spec":{"paused": false}}'`
- ✓ **Caution:** Pausing upgrades for extended periods is generally not recommended as it can leave the cluster in an inconsistent state.

141. What are Machine Config Pools (MCPs), and how do you check their status during an upgrade?

Machine Config Pools (MCPs): Groups of nodes (typically master and worker, but custom pools can exist) that share the same MachineConfig. The Machine Config Operator (MCO) manages updates to nodes within a pool sequentially to apply new configurations (including OS updates delivered via MachineConfigs during an OCP upgrade).

Checking Status During Upgrade: Use `oc get machineconfigpool` or `oc get mcp`.

```
oc get mcp
```

```
# Example Output Columns: NAME CONFIG UPDATED UPDATING DEGRADED
MACHINECOUNT READYMACHINECOUNT UPDATEDMACHINECOUNT
DEGRAEDMACHINECOUNT AGE
```

```
# master rendered-master-... False False False 3 3 3 0
...
```

```
# worker rendered-worker-... True True False 5 5 5 0
...
```

- ✓ Look for the UPDATING column. If True, the pool is actively being updated.
- ✓ Monitor UPDATEDMACHINECOUNT increasing towards MACHINECOUNT.
- ✓ Monitor READYMACHINECOUNT to ensure nodes become ready after rebooting.
- ✓ Check the DEGRADED column for any issues.
- ✓ `oc describe mcp <pool_name>` provides more detailed status and events.

142. How can you see which specific MachineConfig version is currently applied to the nodes in an MCP?

Use `oc describe machineconfigpool <pool_name>`. Look for the `CurrentMachineConfig` field (or `status.configuration.name` in the YAML output). This shows the name of the rendered MachineConfig that the pool's nodes are currently running or attempting to apply.

```
oc describe mcp worker | grep CurrentMachineConfig
```

Or get directly from YAML

```
oc get mcp worker -o jsonpath='{.status.configuration.name}'
```

The name usually follows the pattern `rendered-<pool_name>-<hash>`.

143. How do you monitor the status of individual nodes within an MCP as they are being updated?

List nodes in the pool: Use labels associated with the pool.

```
oc get nodes -l machineconfiguration.openshift.io/role=<pool_name>
```

Example: `oc get nodes -l machineconfiguration.openshift.io/role=worker`

Observe Node Status: During an update, nodes in the pool will be cordoned, drained, rebooted, and uncordoned one by one (or based on `maxUnavailable` settings). Watch the STATUS column in `oc get nodes`. Nodes will transition through `Ready,SchedulingDisabled -> NotReady,SchedulingDisabled -> Ready`.

Check oc get mcp: The counts (READYMACHINECOUNT, UPDATEDMACHINECOUNT) reflect the aggregate status of nodes within the pool.

Check MCD Logs: For detailed progress on a specific node, check the Machine Config Daemon logs (see Q11).

144. What steps would you take if a Cluster Operator becomes DEGRADED and halts the upgrade process?

1. **Identify the Degraded Operator:** Use `oc get co` to find which operator(s) have `DEGRADED=True`.
2. **Describe the Operator:** Use `oc describe co <operator_name>` to get detailed status conditions and messages explaining *why* it's degraded. Pay close attention to the message fields under `status.conditions`.

3. **Check Operator Logs:** Find the operator's deployment/pods (usually in openshift-`<operator_name>` namespace) and check their logs for specific errors: `oc logs deployment/<operator_deployment> -n openshift-<operator_name>`.
4. **Check Operand Logs:** The operator manages other components (operands). Check the logs of the pods related to the operator's function (e.g., for ingress operator, check router pods in openshift-ingress).
5. **Check Related Resources:** The `oc describe co` output often lists related objects. Check the status of these objects (e.g., Deployments, DaemonSets, CRDs managed by the operator).
6. **Consult Documentation/Knowledgebase:** Search Red Hat documentation, knowledgebase articles, or bug reports for the specific error messages or conditions observed.
7. **Consider Pausing (Carefully):** If investigation requires time or intervention, consider pausing the upgrade (`oc patch clusterversion version --type=merge -p '{"spec":{"paused": true}}'`).
8. **Attempt Remediation:** Based on the findings, attempt to fix the underlying issue (e.g., fix a configuration error, address resource constraints, resolve network issues).
9. **Resume Upgrade:** Once the operator becomes healthy (`AVAILABLE=True`, `DEGRADED=False`), resume the upgrade if paused.

145. What checks should you perform before initiating a cluster upgrade?

1. **Read Release Notes:** Thoroughly review the release notes for the target OpenShift version for known issues, prerequisites, deprecated features, and breaking changes.
2. **Check Cluster Health:** Ensure all Cluster Operators are `AVAILABLE=True`, `PROGRESSING=False`, `DEGRADED=False` (`oc get co`).
3. **Check Node Status:** Ensure all nodes are in the Ready state (`oc get nodes`). Address any NotReady nodes.
4. **Check PodDisruptionBudgets (PDBs):** Verify that critical application PDBs allow for sufficient disruptions (`Allowed Disruptions > 0`) so node drains during MCP updates do not stall (`oc get pdb -A`). Misconfigured PDBs are a common cause of upgrade delays.
5. **Check Resource Usage:** Ensure sufficient CPU, memory, and storage resources are available on nodes, especially control plane nodes, to handle the upgrade process.
6. **Backup:** Perform a recent etcd backup and ensure application data backups (PVs, databases) are current. Back up critical CRs/YAMLs.
7. **Check Network Connectivity:** Ensure the cluster can reach required endpoints (Update Service, Quay.io/Registry.redhat.io, or mirror registry).
8. **Review Customizations:** Assess any non-standard configurations (custom MachineConfigs, modified operator configs) for compatibility with the new version.

9. **Check Operator Subscriptions:** Ensure any installed Operators (from OperatorHub) are compatible with the target OpenShift version and their update channels are appropriate.

146. What is the Machine Config Daemon (MCD), and how do you find its pod on a specific node?

Machine Config Daemon (MCD): A DaemonSet managed by the Machine Config Operator (MCO). An MCD pod runs on every node in the cluster. Its primary responsibility is to watch for changes to the desired MachineConfig for the node it's running on, apply those changes (e.g., writing files, modifying systemd units), and report status back to the MCO. It orchestrates the node updates during upgrades or custom config rollouts.

- ✓ **Finding the Pod:** MCD pods run in the openshift-machine-config-operator namespace. Use a field selector to find the pod on a specific node:
- ✓ `oc get pods -n openshift-machine-config-operator -o wide --field-selector spec.nodeName=<node_name>`
- ✓ # Example: `oc get pods -n openshift-machine-config-operator -o wide --field-selector spec.nodeName=worker-0.example.com`

147. How can you check the logs of the MCD to troubleshoot node update issues?

Once you have identified the MCD pod name on the specific node (using the command from the previous question), use `oc logs`:

```
oc logs <mcd_pod_name> -n openshift-machine-config-operator
```

Follow logs in real-time:

```
oc logs -f <mcd_pod_name> -n openshift-machine-config-operator
```

The logs will show details about which MachineConfig it's trying to apply, steps being taken (writing files, reloading services), interactions with rpm-ostree (for RHCOS), drain/cordon operations, and any errors encountered during the update process.

148. What is a MachineConfig object? How are custom node configurations typically applied?

MachineConfig Object: A Kubernetes Custom Resource (CR) used by the Machine Config Operator (MCO) to define the configuration state of nodes (specifically RHCOS nodes) in an OpenShift cluster. They can contain Ignition configuration snippets, systemd units, files, kernel arguments, etc. The MCO combines multiple MachineConfigs (base OS config, cluster-specific settings, custom settings) into a single "rendered" MachineConfig for each pool.

Applying Custom Configs:

1. Create a new MachineConfig YAML file defining your desired change (e.g., adding a kernel argument, creating a file).
2. Assign a label to the MachineConfig that matches the target MachineConfigPool (e.g., `machineconfiguration.openshift.io/role: worker`). If

no label is specified, it typically applies to all pools where its settings don't conflict.

3. Apply the YAML file: `oc apply -f your-custom-machineconfig.yaml`.
4. The MCO detects the new MachineConfig, incorporates it into a new rendered config for the targeted pool(s), and initiates a rolling update of the nodes in that pool via the MCDs.

149. How can you view the final "rendered" MachineConfig for a pool, which combines multiple configuration sources?

1. First, find the name of the current rendered config for the pool using `oc describe mcp <pool_name>` and look for `CurrentMachineConfig` or `status.configuration.name`.
2. Then, use `oc get machineconfig <rendered_config_name> -o yaml` to view the full content.
3. # Example for worker pool
4. `RENDERED_CONFIG_NAME=$(oc get mcp worker -o jsonpath='{.status.configuration.name}')`
5. `oc get machineconfig ${RENDERED_CONFIG_NAME} -o yaml`

This YAML will contain the combined configuration from the base OS, cluster settings, and any applied custom MachineConfigs for that specific pool.

150. Explain the role of PodDisruptionBudgets (PDBs) during node maintenance and upgrades.

PodDisruptionBudgets (PDBs): Kubernetes objects that limit the number of pods of a specific application (identified by labels) that can be voluntarily disrupted simultaneously. Voluntary disruptions include actions like node drains performed during upgrades or maintenance.

Role during Upgrades/Maintenance: When the MCO (via MCD) or an administrator initiates a node drain (`oc adm drain`), the drain process respects PDBs. Before evicting a pod covered by a PDB, the system checks if the eviction would violate the budget (i.e., cause the number of available pods for that application to fall below the PDB's specified minimum available or maximum unavailable count).

Impact: If evicting a pod would violate its PDB, the node drain operation will **block** until the PDB allows the disruption (e.g., after other pods become ready elsewhere). This ensures application availability but can **stall upgrades or maintenance** if PDBs are too restrictive (e.g., `minAvailable: 1` for a single-replica deployment) or if pods cannot be rescheduled successfully. It's crucial to configure PDBs correctly to balance availability with the ability to perform cluster maintenance.

User Management & RBAC

151. How do you list all Users known to the OpenShift cluster?

Use the `oc get users` command. This lists all User objects that have been created in the cluster, typically representing individuals who have logged in at least once via a configured identity provider or have been manually created (less common).

`oc get users`

Example:

```
# Output Columns: NAME          UID          FULL NAME  IDENTITIES
#      developer  xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx Developer  htpasswd_provider:developer
#      kubeadmin  xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx kube:admin kube:admin
```

Note: This only shows users recognized by OpenShift, not necessarily all users defined in an external identity provider (like LDAP) unless they have logged in.

152. How do you list all Groups known to the OpenShift cluster?

Use the `oc get groups` command. This lists Group objects within the cluster. These groups can be synchronized from an external identity provider (like LDAP groups) or created manually within OpenShift using `oc adm groups new`.

`oc get groups`

Example:

```
# Output Columns: NAME          USERS
#      cluster-admins      kubeadmin, user1
#      dedicated-admins
#      openshift-authenticated-ldap user1, user2
```

153. What command adds an existing user to an existing group?

Use the `oc adm groups add-users` command. You need to specify the group name and one or more user names. This modifies the users list within the specified Group object.

`oc adm groups add-users <group_name> <user_name_1> <user_name_2> ...`

Example: Add 'developer' and 'testuser' to the 'app-devs' group

`oc adm groups add-users app-devs developer testuser`

154. How do you remove a user from an OpenShift group?

Use the `oc adm groups remove-users` command. Specify the group name and the user(s) to remove.

`oc adm groups remove-users <group_name> <user_name_1> <user_name_2> ...`

Example: Remove 'testuser' from the 'app-devs' group

`oc adm groups remove-users app-devs testuser`

155. How do you create a new, empty group object in OpenShift?

Use the `oc adm groups new` command followed by the desired group name. This creates a Group resource within OpenShift that you can then add users to or bind roles to.

```
oc adm groups new <new_group_name>
# Example: Create a group named 'qa-team'
oc adm groups new qa-team
```

156. If using the HTPasswd identity provider, what is the process for adding a new user?

Adding a user via HTPasswd involves modifying the `htpasswd` file used by the provider and updating the corresponding secret in OpenShift:

1. Generate/Update HTPasswd Entry: Use the `htpasswd` command-line utility.
 - ✓ To create a new file or add the first user:
`htpasswd -c -B -b /path/to/your/users.htpasswd <username> <password>`
 - ✓ To add subsequent users to an existing file:
`htpasswd -B -b /path/to/your/users.htpasswd <username> <password>`

(Use -B for bcrypt hashing, which is recommended. -b allows password on command line.)
2. Update the Secret: Update the OpenShift secret that holds the `htpasswd` file data (the secret name is defined in the HTPasswd identity provider configuration within the OAuth CR).
 - # Replace 'htpass-secret' with the actual secret name
 - `oc create secret generic htpass-secret --from-file=htpasswd=/path/to/your/users.htpasswd --dry-run=client -o yaml -n openshift-config | oc replace -f -`
 - ✓ This command creates the secret definition locally using the updated file, then replaces the existing secret in the cluster with the new content. The authentication operator will detect the change and reload the configuration.

157. How can you verify the group memberships for a specific user?

Use `oc get user <user_name> -o yaml`. The output YAML will contain a `groups:` field listing all the groups OpenShift recognizes that user as being a member of.

```
oc get user developer -o yaml
```

Look for the `groups:` section in the output. It might be null if the user belongs to no groups recognized by OpenShift.

158. How do you assign the cluster-level cluster-admin role to a specific user? What are the risks?

Command: Use `oc adm policy add-cluster-role-to-user`.
`oc adm policy add-cluster-role-to-user cluster-admin <user_name>`

Risks: Assigning cluster-admin grants unrestricted superuser access to the entire OpenShift cluster. The user can perform *any* action on *any* resource in *any* project, including modifying cluster configurations, managing nodes, deleting projects, viewing all secrets, and changing security settings. This role should be assigned extremely sparingly and only to trusted cluster administrators responsible for the overall health and management of the platform. Accidental or malicious actions by a cluster-admin can have catastrophic consequences.

159. How do you revoke the cluster-admin role from a user?

Use the `oc adm policy remove-cluster-role-from-user` command.

```
oc adm policy remove-cluster-role-from-user cluster-admin <user_name>
```

This removes the ClusterRoleBinding that grants the specified user the cluster-admin ClusterRole.

160. What is the command to create a new project (namespace) with a display name and description?

Use the `oc new-project` command.

```
oc new-project <project_name> --display-name="Your Display Name" --description="Project description here"
```

Example:

```
oc new-project my-app-prod --display-name="My App (Production)" --description="Production environment for My App"
```

This creates the Kubernetes Namespace and associated OpenShift Project object, applying any default templates or configurations defined by the cluster administrator. The user running the command automatically gets the admin role within the new project.

161. How can you inspect the template used to create default resources when a new project is requested?

New projects are typically created based on a cluster-level template. You can inspect this template, usually named `project-request`, located in the `openshift-config` namespace.

```
oc describe template project-request -n openshift-config
```

Or view the full YAML

```
oc get template project-request -n openshift-config -o yaml
```

This template defines default objects like RoleBindings (granting the creator admin rights), LimitRanges, or potentially default NetworkPolicies that are created automatically whenever `oc new-project` is executed.

162. What is a ResourceQuota object used for? How would you apply one to a project?

Purpose: A ResourceQuota object constrains the total amount of compute resources (CPU, memory), storage resources (PVC count, total storage capacity), or object counts (pods, services, secrets) that can be consumed within a specific project (namespace). It helps prevent resource exhaustion and ensures fair usage across different projects or teams.

Applying:

1. Define the quota in a YAML file (e.g., `my-quota.yaml`):


```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-storage-quota
spec:
  hard:
    requests.cpu: "10"    # Total requested CPU across all pods (cores)
```

```

requests.memory: 50Gi  # Total requested Memory across all pods
limits.cpu: "20"       # Total CPU limit across all pods
limits.memory: 100Gi   # Total Memory limit across all pods
pods: "50"             # Max number of pods
persistentvolumeclaims: "20" # Max number of PVCs
requests.storage: 100Gi # Max total storage requested by PVCs

```

2. Apply the YAML to the target project:
`oc apply -f my-quota.yaml -n <project_name>`

163. What is a LimitRange object used for? How would you apply one to a project?

Purpose: A LimitRange object defines constraints on resource *requests* and *limits* for individual Pods or Containers within a project. It can set default request/limit values if not specified by the container, enforce minimum/maximum values, and control the ratio between requests and limits. This helps ensure pods have reasonable resource settings even if not explicitly defined.

Applying:

1. Define the limits in a YAML file (e.g., my-limits.yaml):

```

apiVersion: v1
kind: LimitRange
metadata:
  name: resource-limits
spec:
  limits:
  - type: Container
    default:      # Default limits if none specified
      cpu: "1"
      memory: "1Gi"
    defaultRequest: # Default requests if none specified
      cpu: "100m"
      memory: "256Mi"
    max:          # Max limits allowed per container
      cpu: "4"
      memory: "8Gi"
    min:          # Min requests allowed per container
      cpu: "50m"
      memory: "64Mi"
    maxLimitRequestRatio: # Max ratio of limit to request
      cpu: "10"          # Limit can be 10x request
      memory: "4"        # Limit can be 4x request

```
2. Apply the YAML to the target project:
`oc apply -f my-limits.yaml -n <project_name>`

164. How can you display the authentication token currently used by your oc CLI session?

Use the `oc whoami --show-token` command. This will output the bearer token associated with your current login session. This token is used by `oc` to authenticate subsequent requests to the OpenShift API server.

```
oc whoami --show-token
```

Handle this token carefully as it grants permissions associated with your user account.

165. What command initiates a login using a username and password?

Use the `oc login` command, providing the API server URL and credentials.

```
oc login <api_server_url> -u <username> -p <password>
```

```
# Example: oc login https://api.mycluster.example.com:6443 -u developer -p mysecretpassword
```

You can often omit the password (`-p`) and be prompted securely. This command authenticates against the identity provider configured for the cluster (e.g., `HTPasswd`, `LDAP`).

166. How do you log out of your current oc CLI session?

Use the `oc logout` command. This clears the stored authentication token for the current server from your local `oc` configuration (`~/.kube/config`), effectively ending your session.

```
oc logout
```

167. What command switches your active project context?

Use the `oc project` command followed by the name of the project you want to switch to. This sets the default namespace for subsequent `oc` commands that operate within a project scope (like `oc get pods`, `oc apply`).

```
oc project <project_name>
```

```
# Example: oc project my-app-dev
```

Running `oc project` without arguments displays the currently active project.

168. How do you get a list of all projects your current user has access to?

Use the `oc projects` command. This queries the API server and lists all the projects (namespaces) for which your currently logged-in user has at least view permissions.

```
oc projects
```

Internal Registry & Images

169. How do you check if the internal image registry deployment is running correctly?

Check the Cluster Operator: The image-registry operator manages the registry deployment. Check its status:

- ✓ `oc get co image-registry`
- ✓ `# Ensure AVAILABLE=True, PROGRESSING=False, DEGRADED=False`
- ✓ `oc describe co image-registry # Check for detailed status messages/errors`

Check the Deployment: Verify the image-registry deployment in the openshift-image-registry namespace is available and its pods are running and ready.

- ✓ `oc get deployment image-registry -n openshift-image-registry`
- ✓ `oc get pods -n openshift-image-registry -l docker-registry=default`

170. How would you find the external URL (Route) for the internal registry, if one is configured?

By default, the internal registry is not exposed externally with a Route. If it has been manually exposed:

Check for a Route:

- ✓ `oc get route image-registry -n openshift-image-registry`
- ✓ `# If a route exists, its HOST/PORT column shows the URL.`

Check Operator Configuration: The exposure might be configured via the registry operator's config.

- ✓ `oc get config.imageregistry.operator.openshift.io/cluster -o yaml`
- ✓ `# Look for spec.routes configuration.`

171. What is the standard internal service hostname and port for the OpenShift image registry?

The internal registry is accessible within the cluster using its Kubernetes service name and port:

- ✓ **Hostname:** `image-registry.openshift-image-registry.svc`
- ✓ **Port:** `5000`
- ✓ **Full Internal Address:** `image-registry.openshift-image-registry.svc:5000`
- ✓ Pods within the cluster use this address to push and pull images from the internal registry.

172. How can you check the storage backend configuration (e.g., PVC, S3, filesystem) for the internal registry?

- ✓ Examine the imageregistry.operator.openshift.io cluster configuration resource:
- ✓ `oc get config.imageregistry.operator.openshift.io/cluster -o jsonpath='{.spec.storage}'`
- ✓ # Or view the full YAML for more context:
- ✓ `oc get config.imageregistry.operator.openshift.io/cluster -o yaml`
- ✓ The spec.storage field will show the configured backend, such as pvc, s3, azure, gcs, swift, or emptyDir (not recommended for production). It will also contain specific parameters for the chosen backend (like PVC name/claim, bucket names, credentials secret).

173. If the registry uses persistent storage (PVC), how do you find the associated PVC?

Check Operator Config: Get the storage configuration as shown above (`oc get config.imageregistry... -o yaml`). If spec.storage.pvc is configured, it will contain the claim name.

Get PVC: Use the claim name found in the config to get the PVC details in the openshift-image-registry namespace.

```
# Assuming the claim name found was 'image-registry-storage'
oc get pvc image-registry-storage -n openshift-image-registry
oc describe pvc image-registry-storage -n openshift-image-registry
```

174. Explain the purpose of `oc adm prune images`. What options can control its behavior?

Purpose: The `oc adm prune images` command removes unused image layers and manifests from the internal OpenShift registry to reclaim storage space. It identifies images that are no longer referenced by any ImageStream tags and image layers (blobs) that are not part of any remaining image manifest stored in the registry.

Common Options:

- ✓ `--keep-tag-revisions=N`: Keep the specified number of most recent revisions per image stream tag (default: 3).
- ✓ `--keep-younger-than=DURATION`: Keep images younger than the specified duration (e.g., 60m, 24h).
- ✓ `--prune-over-size-limit`: Prune images exceeding the cluster's image limit settings (less common for manual pruning).
- ✓ `--all`: Prune images even if they are not part of any image stream (use cautiously).
- ✓ `--confirm`: Required to actually delete the images/layers; without it, the command performs a dry run showing what would be pruned.
- ✓ `--registry-url`: Specify registry URL if different from default internal service.
- ✓ `--cacert`, `--token`: Provide CA cert or auth token if needed to access the registry.

175. How can you check if the automated image pruner CronJob is configured and running successfully?

The automated pruner is configured via the `imageregistry.operator.openshift.io` resource (spec.pruning). If enabled, a CronJob is created:

Check CronJob: Look for the image-pruner CronJob in the openshift-image-registry namespace.

✓ `oc get cronjob image-pruner -n openshift-image-registry`

Check Last Schedule/Run: The output shows the schedule (SCHEDULE), suspend status (SUSPEND), last scheduled time (LAST SCHEDULE), and age.

Check Job History: List the jobs created by the CronJob to see recent runs and their completion status.

✓ `oc get jobs -n openshift-image-registry -l image-pruner=true`

Check Job Logs: View the logs of a completed pruner job's pod for details on what was pruned or any errors.

✓ # Find a completed job name (e.g., image-pruner-123456)

✓ `JOB_NAME=$(oc get jobs -n openshift-image-registry -l image-pruner=true --sort-by=.metadata.creationTimestamp -o jsonpath='{.items[-1:].metadata.name}')`

✓ `oc logs job/${JOB_NAME} -n openshift-image-registry`

176. What is an ImageStream in OpenShift? How do you list them in a project?

ImageStream: An OpenShift-specific resource (ImageStream or is) that acts as a pointer or abstraction layer over container images. It provides a stable endpoint (imagestreamtag or istag) within a project that references one or more specific container image digests stored in a registry (internal or external). ImageStreams enable build and deployment triggers based on image updates without needing to directly reference mutable tags like :latest. They track the history of images associated with tags.

Listing: Use `oc get imagestream` or `oc get is`.

✓ `oc get is -n <project_name>`

✓ # Output Columns: NAME IMAGE REPOSITORY TAGS
UPDATED

✓ # my-app image-registry.openshift-image-registry.svc:5000/my-project/my-app latest ...

✓ # ubi8 registry.access.redhat.com/ubi8/ubi 8.5 ...

177. How can you view the different tags within an ImageStream and the image digests they point to?

Use `oc describe imagestream` or `oc describe is`.

`oc describe is my-app -n <project_name>`

- ✓ The output will list each tag (e.g., latest, v1.0, prod) and show the image digest (SHA) it currently points to, along with the registry location and when it was created or updated. It also shows the history of images previously associated with each tag.

178. What command imports an image from an external registry into an OpenShift ImageStream?

Use the `oc import-image` command. This command inspects the external image and updates or creates a tag within the specified ImageStream to point to that external image's digest.

```
oc import-image <imagestream_name_or_imagestreamtag_name> --
from=<external_registry/image:tag> --confirm -n <project_name>
```

Example: Import `nginx:latest` into the 'nginx' imagestream, tagging it as 'latest'

```
oc import-image nginx --from=docker.io/library/nginx:latest --confirm -n my-project
```

Example: Import a specific image and tag it as 'stable' in the 'my-app' imagestream

```
oc import-image my-app:stable --from=quay.io/myorg/my-app:v2.1 --confirm -n my-project
```

- ✓ `--confirm`: Required to actually perform the import; otherwise, it's a dry run.
- ✓ If the ImageStream doesn't exist, this command can create it if you specify `<name>:<tag>`.

179. Where is the cluster's global pull secret stored, and how do you inspect its contents?

Location: The cluster-wide pull secret, containing credentials needed by nodes to pull images (including for OpenShift components from Red Hat registries), is stored as a Secret named `pull-secret` in the `openshift-config` namespace.

Inspection:

- ✓ Get the Secret YAML:
- ✓ `oc get secret pull-secret -n openshift-config -o yaml`
- ✓ The actual credentials are in the `.data[".dockerconfigjson"]` field, base64 encoded. To decode and view the JSON content:
- ✓ `oc get secret pull-secret -n openshift-config -o jsonpath='{.data.\.dockerconfigjson}' | base64 --decode`
 - ✓ This outputs a JSON structure similar to a local Docker `config.json` file, containing authentication tokens for various registries. Handle this output securely.

180. Describe the process for adding credentials for a new private registry to the cluster's global pull secret.

Modifying the global pull secret requires care as it affects the entire cluster.

Get Current Secret: Extract the current decoded `.dockerconfigjson` data into a file:

```
oc get secret pull-secret -n openshift-config -o jsonpath='{.data.\.dockerconfigjson}' |
base64 --decode > current_config.json
```

Prepare New Credentials: Create a temporary `Docker config.json` file containing *only* the credentials for the *new* private registry. You can often generate this by running `podman login <your_private_registry>` or `docker login <your_private_registry>` locally and copying the relevant entry from your local `~/.docker/config.json` or `~/.config/containers/auth.json`. It will look something like:

```
{
  "auths": {
    "my-private-registry.example.com": {
      "auth": "BASE64_ENCODED_USERNAME:PASSWORD",
      "email": "your-email@example.com"
    }
  }
}
```

Merge Credentials: Merge the new registry credentials into the `current_config.json` file downloaded in step 1. You can do this manually by editing the JSON or using a tool like `jq`. Ensure the final structure is correct JSON with multiple entries under `"auths"`.

Example using `jq` (install `jq` if needed)

```
jq -s '.[0] * .[1]' current_config.json new_creds_only.json > merged_config.json
```

Patch the Secret: Update the pull-secret in `openshift-config` with the new merged and base64 encoded content.

```
oc patch secret pull-secret -n openshift-config -p \
{"data\":{\".dockerconfigjson\":\"$(cat merged_config.json | base64 -w0)\"}}
```

The cluster nodes will gradually pick up the updated secret. This process ensures existing credentials (like Red Hat registry access) are preserved.

181. What is an ImageContentSourcePolicy (ICSP), and how is it used for registry mirroring?

ImageContentSourcePolicy (ICSP): A cluster-scoped OpenShift resource (ImageContentSourcePolicy) that allows administrators to redirect image pull requests from a source registry to one or more mirror registries.

Usage for Mirroring: In disconnected or restricted network environments, ICSPs are crucial. They tell nodes: "When you need to pull an image from registry.redhat.io/ubi8/ubi, try pulling it from mymirror.internal:5000/ubi8/ubi instead." This redirects pulls for specific repositories (or entire registries) to a local mirror that contains copies of the required images, avoiding the need for direct internet access from cluster nodes. Multiple mirrors can be specified for redundancy.

182. How do you list the currently configured ICSPs in the cluster?

Use `oc get imagecontentsourcepolicy`.

`oc get imagecontentsourcepolicy`

Or using the short name:

`oc get icsp`

Output Columns: NAME AGE

redhat-mirror 120d

my-app-mirror 55d

- ✓ To see the actual mirroring rules, describe a specific ICSP:
- ✓ `oc describe icsp redhat-mirror`
- ✓ # Or view the YAML:
- ✓ `oc get icsp redhat-mirror -o yaml`

183. How can you check if policies related to image signature verification are configured?

Image signature verification policies are configured in the cluster-wide image configuration resource:

`oc get image.config.openshift.io/cluster -o yaml`

- ✓ Look within the `spec:` section for fields related to policy, such as `policyJson` or references to `ClusterImagePolicy` objects (if using the `image-policy-operator`). The `policyJson` field (if used directly) contains the detailed policy rules defining trusted registries, keys, and enforcement actions (reject, allow). Alternatively, list `ClusterImagePolicy` resources:

`oc get clusterimagepolicy`

Backup, Restore & Disaster Recovery

184. What is the primary method for backing up the etcd cluster state in OpenShift 4?

The primary and recommended method is utilizing the **built-in etcd backup mechanism managed by the etcd Cluster Operator**. This operator automatically takes periodic snapshots of the etcd database, which stores the entire cluster state (all Kubernetes/OpenShift objects like Deployments, Secrets, ConfigMaps, CRDs, etc.).

- ✓ These backups are crucial for disaster recovery scenarios where the etcd cluster becomes corrupted or lost, as etcd holds the definitive state of the cluster.
- ✓ The operator ensures backups are taken consistently across the etcd members. While manual triggering is possible via scripts on the master nodes, relying on the automated backups configured via the operator is the standard practice.

185. Where are the etcd backups typically stored on the master nodes?

By default, the etcd operator stores these backups **on the local filesystem of each master node**. Common default locations include:

- ✓ /etc/kubernetes/static-pod-resources/etcd-backup/
- ✓ /var/lib/etcd-backup/
- ✓ The exact path can be confirmed by inspecting the configuration of the etcd Cluster Operator or the static pod definition for etcd (/etc/kubernetes/manifests/etcd-pod.yaml on the masters).

Crucially for Disaster Recovery: These on-node backups **must be copied off the cluster nodes to a secure, external location** (e.g., remote storage like NFS, S3, or a dedicated backup server). Relying solely on backups stored locally on the masters does not protect against complete node or site failure.

186. What are common strategies for backing up application data stored in Persistent Volumes? Mention OADP/Velero.

Backing up Persistent Volume (PV) data requires considering the storage backend and application consistency needs. Common strategies include:

Storage-Level Snapshots: Many underlying storage systems (SAN, NAS, Cloud Provider Block Storage, ODF/Ceph) offer native snapshot capabilities. These create point-in-time copies of volumes quickly and often efficiently at the block level. Integration might require vendor-specific tools or APIs.

CSI VolumeSnapshots: The Kubernetes Container Storage Interface (CSI) standard includes support for volume snapshots. If your storage driver supports this, you can create Kubernetes VolumeSnapshot objects, which trigger the underlying storage provider to create a snapshot in a vendor-neutral way. This is becoming the preferred method for Kubernetes-integrated volume snapshots.

Application-Level Backups: For stateful applications like databases, simply snapshotting the disk might not guarantee data consistency. Using application-specific tools (pg_dump, mysqldump,

application export features) to create consistent backups is often essential. These backup files can then be stored either *within* another PV or, more commonly, pushed to external backup storage (like S3).

OADP (OpenShift API for Data Protection) / Velero: This is the Red Hat recommended cloud-native solution. OADP, built upon the upstream Velero project, provides a framework for backing up and restoring OpenShift applications.

- ✓ It backs up Kubernetes object definitions (Deployments, PVCs, Services, etc.).
- ✓ It integrates with storage providers (via CSI VolumeSnapshots or vendor-specific plugins) to snapshot PV data *or* can use file-level backup tools (like Restic/Kopia, integrated as 'hooks') to back up data from within pods.
- ✓ Backups (both resource definitions and PV snapshots/data) are typically stored in external object storage (S3, Azure Blob, etc.).
- ✓ OADP provides a holistic approach, managing both the application state (objects) and data (PVs) together.

187. How can you perform a basic backup of OpenShift resource definitions (like Deployments, Services) as YAML files?

You can use the `oc get` command combined with the `-o yaml` output format and shell redirection.

Specific Resource Type in a Project:

```
# Backup all Deployments in 'my-project'
oc get deployment -n my-project -o yaml > my-project-deployments.yaml
# Backup all Services in 'my-project'
oc get svc -n my-project -o yaml > my-project-services.yaml
# Backup a specific ConfigMap
oc get configmap my-config -n my-project -o yaml > my-project-my-config-cm.yaml
```

Multiple Resource Types in a Project:

```
oc get deployment,statefulset,svc,route,pvc,configmap,secret -n my-project -o yaml
> my-project-core-resources.yaml
```

Specific Resource Type Cluster-Wide (e.g., ClusterRoles):

```
oc get clusterroles -o yaml > clusterroles.yaml
```

Limitations:

- ✓ This method requires manual identification of all necessary resource types.
- ✓ It doesn't automatically handle dependencies between resources.
- ✓ Restoring requires applying files in the correct order.
- ✓ It captures the state *at that moment* and doesn't include PV data.
- ✓ Tools like OADP/Velero are generally preferred for comprehensive application backups as they handle these complexities better.

188. How would you back up User and Group definitions?

User and Group objects are cluster-scoped resources in OpenShift. You can export their definitions using `oc get`:

```
oc get users -o yaml > users-backup.yaml
oc get groups -o yaml > groups-backup.yaml
```

Important Considerations:

- ✓ This only backs up the OpenShift representation of the users and groups.
- ✓ It **does not** back up the actual user accounts, passwords, or group memberships stored in your external Identity Provider (IDP) like LDAP, Active Directory, or an HTPasswd file/secret.
- ✓ You **must** have a separate backup strategy for your IDP itself.
- ✓ Restoring just these OpenShift objects without the backing IDP may result in incomplete user/group information or login failures.

189. How would you back up cluster-wide and project-specific RBAC definitions?

Role-Based Access Control (RBAC) definitions include Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.

Cluster-Wide (ClusterRoles, ClusterRoleBindings): These are cluster-scoped.

```
oc get clusterroles -o yaml > clusterroles-backup.yaml
oc get clusterrolebindings -o yaml > clusterrolebindings-backup.yaml
```

Project-Specific (Roles, RoleBindings): These are namespaced. You can back them up per project or across all projects.

```
# For a specific project 'my-app-dev'
oc get roles -n my-app-dev -o yaml > my-app-dev-roles.yaml
oc get rolebindings -n my-app-dev -o yaml > my-app-dev-rolebindings.yaml
# For all projects (may include many default/system bindings)
oc get roles --all-namespaces -o yaml > all-roles-backup.yaml
oc get rolebindings --all-namespaces -o yaml > all-rolebindings-backup.yaml
```

Note: Backing up RBAC is crucial for restoring application permissions correctly. OADP/Velero typically includes relevant RBAC resources when backing up namespaces.

190. If using OADP (Velero), how do you check the status of the Velero pods?

The OADP Operator installs Velero components, usually into the openshift-adp namespace (this can be customized during installation). Check the pods in that namespace:

```
oc get pods -n openshift-adp
```

Look for:

- ✓ The main velero deployment pod(s): Responsible for coordinating backups and restores.
- ✓ The node-agent DaemonSet pods (one per node): Used if employing file-level PV backups (like Restic/Kopia). Not always present if only using CSI snapshots.
- ✓ Plugin pods for specific providers (e.g., velero-plugin-for-aws, velero-plugin-for-vsphere).

Ensure these pods are in the Running state and have their containers ready (e.g., 1/1 or 2/2).

191. How do you trigger an ad-hoc backup using the velero CLI?

Once the velero command-line tool is installed and configured to point to your cluster and backup storage location, use the velero backup create command.

```
# Backup all resources in namespace 'my-app' and associated PVs
velero backup create my-app-backup-$(date +%Y%m%d%H%M) --include-namespaces my-app --snapshot-volumes=true --wait
# Backup resources matching a label, without PV snapshots
velero backup create backend-services --selector service=backend --snapshot-volumes=false
# Set a Time-To-Live (TTL) for the backup
```

```
velero backup create temporary-backup --include-namespaces test-ns --ttl 24h0m0s
```

- ✓ --include-namespaces: Specifies which namespaces to back up.
- ✓ --selector: Filters resources based on labels.
- ✓ --snapshot-volumes: Instructs Velero to attempt PV snapshots (requires compatible storage plugin and setup). Set to false to only back up resource definitions.
- ✓ --wait: Waits for the backup to complete and reports the status.
- ✓ Many other flags exist for fine-tuning (--include-resources, --exclude-resources, --storage-location, etc.). Run velero backup create --help for details.
- ✓ Alternatively, create a Backup Custom Resource definition in YAML and apply it using oc apply -f backup-crd.yaml -n openshift-adp.

192. How do you check the status and details of completed OADP/Velero backups?

Use the velero backup get and velero backup describe commands provided by the Velero CLI.

```
# List all backups, their status, creation time, expiration, etc.
velero backup get
# Show detailed information about a specific backup
velero backup describe <backup_name>
# Example: velero backup describe my-app-backup-202504251100
# Download logs for a specific backup (useful for failures)
velero backup logs <backup_name>
velero backup get shows the PHASE (e.g., Completed, PartiallyFailed, Failed).
```

- ✓ velero backup describe provides comprehensive details, including: start/completion times, TTL, included/excluded resources, PV snapshot info (if applicable), warnings, and critical errors encountered during the backup process.
- ✓ You can also view the Backup Custom Resources using oc: `oc get backups -n openshift-adp`.

193. Describe, at a high level, the process involved in restoring the cluster from an etcd backup. Why is it a DR scenario?

High-Level Process: Restoring from an etcd backup is a critical **Disaster Recovery (DR)** procedure used only when the etcd cluster (which holds the entire cluster state) is corrupt, lost, or otherwise unrecoverable. The general steps are:

1. **Stop Control Plane:** Ensure the Kubernetes API server and other control plane components are stopped on all master nodes to prevent conflicting writes.
2. **Select Backup:** Identify a known-good, consistent etcd snapshot from your external backup location.
3. **Initialize Restore:** On *one* master node, use etcd utilities (etcdctl snapshot restore) or documented OpenShift recovery procedures/scripts to restore the snapshot into a *new* etcd data directory.
4. **Reconfigure Etcd:** Adjust the etcd configuration to reflect the restored state and potentially a single-member initial cluster.
5. **Start Initial Node:** Start the etcd service and potentially the API server on this first restored master.
6. **Clean & Join Other Masters:** On the *other* master nodes, completely remove their old etcd data directories. Configure them to join the etcd cluster hosted by the first restored node.
7. **Verify & Restart:** Once etcd quorum is re-established and stable, restart all control plane components across all masters and verify cluster health. Restart nodes if necessary.

Why DR:

- ✓ **Requires Full Control Plane Outage:** The API server must be down during the core restore process.
- ✓ **Data Loss:** All cluster changes (new applications, configurations, secrets, etc.) made *after* the timestamp of the etcd backup being restored are **permanently lost**. The cluster reverts entirely to the state captured in the backup.
- ✓ **Complexity & Risk:** The procedure is complex, requires deep system-level access to masters, and has significant risk if performed incorrectly. It requires strict adherence to official Red Hat documentation for the specific OpenShift version.

- ✓ **Last Resort:** It's used only when the cluster state database is fundamentally broken and cannot be repaired through normal operator recovery or quorum adjustments.

194. How would you restore application resources if you only had YAML backups?

Restoring from individual YAML files requires careful planning and execution, especially regarding dependencies.

1. **Prepare Target:** Ensure the target project (namespace) exists (oc new-project ... if needed). Verify cluster-level dependencies like required CRDs or StorageClasses are present on the target cluster.
2. **Determine Order:** Identify resource dependencies. Generally, apply resources in an order like this:
 - ✓ Namespaces (if not already existing)
 - ✓ Custom Resource Definitions (CRDs) - less common to back up this way
 - ✓ Service Accounts, Roles, RoleBindings
 - ✓ Secrets, ConfigMaps
 - ✓ PersistentVolumeClaims (PVCs) - *Note: This only creates the claim; it doesn't restore data.*
 - ✓ Deployments, StatefulSets, DaemonSets (these create Pods)
 - ✓ Services
 - ✓ Routes or Ingresses
3. **Apply Resources:** Use `oc apply -f <filename> -n <project_name>` for each YAML file in the determined order. Using `oc apply` is generally safer than `oc create` as it handles existing resources.


```
oc apply -f my-project-rbac.yaml -n my-project
oc apply -f my-project-secrets.yaml -n my-project
oc apply -f my-project-configmaps.yaml -n my-project
oc apply -f my-project-pvcs.yaml -n my-project
oc apply -f my-project-deployments.yaml -n my-project
oc apply -f my-project-services.yaml -n my-project
oc apply -f my-project-routes.yaml -n my-project
```
4. **Verify:** Check the status of restored pods, services, and routes (`oc get pods`, `oc describe pod`, `oc logs`).

Limitations: This method **does not restore PV data**. It can be error-prone due to dependency ordering. Generated resource names might cause issues if not handled correctly.

195. How is PV data typically restored when using storage-level snapshots or CSI VolumeSnapshots?

Storage-Level Snapshots: The exact procedure depends on the storage vendor's tools and capabilities:

1. Identify the required snapshot on the storage system.
2. Use the storage vendor's interface (CLI/GUI) to create a *new volume* cloned from that snapshot. Restoring in-place over the original volume is possible but often riskier.
3. Manually create a Kubernetes PersistentVolume (PV) object definition that points to this newly created storage volume (using its unique ID, path, etc.). Ensure the PV spec matches the original (capacity, access modes).
4. Create a new PersistentVolumeClaim (PVC) in the target namespace that can bind to this manually created PV (matching StorageClass, access modes, size request).
5. Modify the application's Deployment/StatefulSet YAML to mount the *new* PVC containing the restored data. Apply the updated application manifest.

CSI VolumeSnapshots: This provides a Kubernetes-native restore workflow:

6. Ensure a VolumeSnapshot Kubernetes object exists, representing the point-in-time backup you want to restore.
7. Create a *new* PVC manifest. In the spec.dataSource field, reference the VolumeSnapshot object by name, kind (VolumeSnapshot), and apiGroup (snapshot.storage.k8s.io). The PVC's storageClassName, accessModes, and resources.requests.storage should match the original PVC.

Example restored-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-app-data-restored
spec:
  storageClassName: ocs-storagecluster-ceph-rbd # Example
  dataSource:
    name: my-app-data-snapshot-20250425 # Name of VolumeSnapshot
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

8. Apply the new PVC manifest: `oc apply -f restored-pvc.yaml -n <namespace>`.
The CSI driver interacts with the storage backend to provision a new volume from the snapshot and automatically creates/binds the corresponding PV.
9. Update the application's Deployment/StatefulSet to use this newly created PVC (my-app-data-restored).

196. How do you initiate a restore operation using the velero CLI?

Use the `velero restore create` command, referencing the backup you want to restore from.

```
# Basic restore of everything in a backup into the original namespaces
velero restore create my-restore-$(date +%Y%m%d%H%M) --from-backup <backup_name> --wait
# Restore only specific namespaces from a backup
velero restore create restore-app-only --from-backup <backup_name> --include-namespaces my-app-dev
# Restore resources but exclude specific types
velero restore create restore-no-secrets --from-backup <backup_name> --exclude-resources secrets
# Restore into different namespaces
velero restore create restore-remapped --from-backup <backup_name> --namespace-mappings old-ns:new-ns,another-ns:target-ns
# Restore without PVs (if backup included snapshots you don't want to restore now)
velero restore create restore-no-pvs --from-backup <backup_name> --restore-volumes=false
```

--from-backup: Specifies the name of the backup object to use.
 --include-namespaces, --exclude-namespaces, --selector, --include-resources, --exclude-resources: Flags for filtering what gets restored.
 --namespace-mappings: Restores resources from one namespace into another.
 --restore-volumes: Set to true (default) or false to control PV restore from snapshots/backups.
 --wait: Waits for the restore to complete (or fail) and reports status.
 See `velero restore create --help` for all options. You can also create a Restore CRD YAML and apply it via `oc apply`.

197. How do you monitor the progress and check the status of an OADP/Velero restore operation?

Use the Velero CLI commands:

```
# List restores and their current phase
velero restore get
# Describe a specific restore for detailed progress and errors/warnings
velero restore describe <restore_name>
# Example: velero restore describe my-restore-202504251130
# Get the logs for the restore operation
velero restore logs <restore_name>
```

- ✓ `velero restore get` shows the PHASE (e.g., New, InProgress, Completed, PartiallyFailed, Failed).
- ✓ `velero restore describe` is essential for details. It lists the total items to restore, how many have been processed, warnings (e.g., resource already exists), and critical errors that caused failures.
- ✓ Monitor the target namespace(s) directly using `oc get pods`, `oc get events`, etc., to see resources being created and pods starting up.
- ✓ The Restore Custom Resources can also be checked via `oc get restores -n openshift-adp`.

198. Why is it important to keep a copy of the original `install-config.yaml` file?

The `install-config.yaml` file used by the OpenShift installer contains the fundamental, user-provided configuration choices made *before* the cluster existed. Keeping a safe copy is crucial for several reasons:

- ✓ **Disaster Recovery Reference:** If you need to rebuild the cluster or troubleshoot fundamental issues, this file documents the original intent regarding platform (AWS, vSphere, BareMetal, etc.), region, base domain, networking selections (cluster network CIDR, service network CIDR, network type), control plane/compute node configurations (for IPI), and the initial pull secret.
- ✓ **Configuration Audit:** It serves as a record of the initial cluster design decisions.
- ✓ **Adding Nodes (UPI):** For UPI installs, referring back to the networking details can be helpful when adding new nodes manually.
- ✓ **Troubleshooting Installation Failures:** If the initial installation fails, this file is required for analysis.
- ✓ **Cluster Reconfiguration (Limited):** While many settings are immutable post-installation, knowing the initial settings can inform certain reconfiguration attempts (though direct modification of many core settings post-install is unsupported or complex).
- ✓ **Cannot Be Reliably Reconstructed:** While some information can be gleaned from the running cluster's CRDs, the `install-config.yaml` captures the specific inputs provided to the installer, which cannot always be perfectly reconstructed after the fact.

Performance & Tuning

199. How do you identify the pods consuming the most CPU across the entire cluster?

Use the `oc adm top pods` command with the `-A` (or `--all-namespaces`) flag and sort by CPU.

```
oc adm top pods -A --sort-by=cpu
```

- ✓ This command queries the cluster's metrics server (usually deployed by default) and lists pods from all namespaces, ordered by their current CPU consumption (typically shown in millicores). This helps quickly identify potential CPU hotspots.

200. How do you identify the pods consuming the most memory across the entire cluster?

Similar to CPU, use `oc adm top pods` with the `-A` flag and sort by memory.

```
oc adm top pods -A --sort-by=memory
```

- ✓ This lists pods from all namespaces ordered by their current memory consumption (typically shown in MiB or GiB). This helps identify pods that might be causing memory pressure on nodes.

201. How can you check the configured CPU/Memory requests and limits for a specific running container?

Use the `oc describe pod` command and inspect the Resources section for the specific container.

```
oc describe pod <pod_name> -n <project_name>
```

- ✓ In the output, navigate to the Containers: section, find the relevant container name, and look under its Resources: subsection. This will show the configured Requests (amount guaranteed) and Limits (maximum allowed) for both cpu and memory. If not explicitly set, defaults from a LimitRange might apply, or they might be unset.

202. What is the most common way to check if a pod was terminated due to exceeding its memory limit (OOMKilled)?

The most common way is to use `oc describe pod`.

```
oc describe pod <pod_name> -n <project_name>
```

Look in two places in the output:

1. **Container Status:** Under the State: or Last State: (if it terminated) of the relevant container, the Reason: field will often show OOMKilled.
2. **Events:** The Events section at the bottom might show events related to the pod being killed due to OOM, often indicating which node it occurred on.

OOMKilled means the container used more memory than its configured limit, and the Linux kernel terminated the process.

203. How can you investigate if a container is being CPU throttled?

CPU throttling occurs when a container tries to use more CPU time than its configured limit allows over a period. You can investigate this using:

1. **Metrics:** Query the cluster's Prometheus instance (via Grafana dashboards or direct query) for metrics like:
 - ✓ `container_cpu_cfs_throttled_periods_total`: A counter of the total number of periods where the container was throttled.
 - ✓ `container_cpu_cfs_throttled_seconds_total`: The total time the container spent throttled. Increases in these metrics indicate throttling.

2. **oc adm top pod --containers:** While this primarily shows current usage, consistently high usage near the limit *might* correlate with throttling, although metrics are more definitive.
3. `oc adm top pod <pod_name> -n <project_name> --containers`

204. What is the purpose of the Performance Addon Operator and PerformanceProfiles?

Purpose: The Performance Addon Operator is designed to optimize OpenShift nodes for high-performance, low-latency workloads, often required in fields like Telco (NFV), High-Performance Computing (HPC), and real-time financial applications.

PerformanceProfile: This is a Custom Resource managed by the operator. Administrators create PerformanceProfile objects to define a set of performance-related tunings to apply to a group of nodes (selected via node labels). These tunings can include:

- ✓ Reserving specific CPUs exclusively for high-priority workloads (CPU affinity/isolation).
- ✓ Configuring hugepages.
- ✓ Applying specific tuned profiles (e.g., realtime, network-latency).
- ✓ Adjusting kernel parameters and IRQ affinities.

In essence, it automates complex node tuning tasks required for demanding applications.

205. How can you verify the tuned profile currently active on a node?

The tuned daemon applies system tuning profiles. To check the active profile on an RHCOS node:

1. Start a debug session on the node: `oc debug node/<node_name>`
2. Inside the debug pod, access the node's host environment and run the `tuned-adm` command:
3. `chroot /host tuned-adm active`

This will output the name of the currently active tuned profile (e.g., `openshift-node`, `realtime`, or a custom profile applied by the Performance Addon Operator).

206. How do you check the status of the Performance Addon Operator components?

The operator typically runs in the `openshift-performance-addon-operator` namespace (or similar, check operator installation details).

```
# Check the operator deployment
oc get deployment -n openshift-performance-addon-operator
# Check the operator pods
oc get pods -n openshift-performance-addon-operator
# Check the status of applied PerformanceProfiles
oc get performanceprofiles
oc describe performanceprofile <profile_name> # Check conditions/events
```

- ✓ Ensure the operator deployment is available and its pods are running. Check the status conditions of any applied PerformanceProfile CRs for errors.

207. Explain how Linux hugepages can benefit certain workloads.

Linux manages memory in units called "pages" (typically 4KB on x86_64). The CPU uses a Translation Lookaside Buffer (TLB) to cache mappings between virtual and physical memory addresses for faster lookups.

Benefit: For applications that manage large amounts of memory (like databases, JVMs, scientific computing), using the standard small page size can lead to frequent TLB misses, as the TLB can only hold a limited number of mappings. Hugepages (typically 2MB or 1GB) allow single TLB entries to map much larger memory regions. This significantly reduces TLB misses, improving memory access performance and overall application throughput for memory-intensive workloads.

Trade-offs: Hugepages must usually be pre-allocated, reducing memory flexibility. Not all applications benefit.

208. How are hugepages typically configured on OpenShift nodes?

Configuring hugepages on RHCOS nodes usually involves modifying the node's kernel arguments and potentially sysctl parameters, typically managed via MachineConfigs:

1. **MachineConfig:** Create a custom MachineConfig targeting the desired node pool (e.g., worker).
2. **Kernel Arguments:** Add kernel arguments to reserve hugepages at boot time. Common arguments include:
 - `hugepagesz=<size>` (e.g., `hugepagesz=1G` or `hugepagesz=2M`)
 - `hugepages=<count>` (e.g., `hugepages=16`)
 - `default_hugepagesz=<size>`
3. **Apply MachineConfig:** Apply the custom MachineConfig. The MCO will roll out the change to the nodes in the pool, requiring node reboots.
4. **Pod Specification:** Applications needing hugepages must request them in their pod spec's `resources.limits` section (e.g., `hugepages-1Gi: 8Gi`).

The Performance Addon Operator can also automate hugepage configuration as part of a PerformanceProfile.

209. How do you check the number of hugepages configured and available on a node?

Use `oc describe node <node_name>` and look at the Capacity and Allocatable sections in the output.

```
oc describe node <node_name> | grep hugepages
```

The output will show lines like:

```
hugepages-1Gi: 16Gi (Capacity - total configured)
hugepages-1Gi: 16Gi (Allocatable - available for pods)
```

You can also check `/proc/meminfo` inside a node's debug pod (`chroot /host cat /proc/meminfo | grep -i huge`).

210. What Prometheus metrics would you check to assess etcd performance, particularly disk latency?

Monitoring etcd performance is critical for cluster stability. Key Prometheus metrics related to disk latency include:

- ✓ `etcd_disk_wal_fsync_duration_seconds_bucket`: Histogram of WAL (Write Ahead Log) fsync durations. High latencies here indicate slow disk writes for transaction logging, which severely impacts performance. Check the higher percentile buckets (e.g., `le="0.1"`, `le="0.5"`).
- ✓ `etcd_disk_backend_commit_duration_seconds_bucket`: Histogram of backend commit durations (writing state to disk). High latencies indicate slow disk performance for persisting the main database.
- ✓ `etcd_server_leader_changes_seen_total`: Frequent leader changes can indicate network instability or performance issues.
- ✓ General etcd health metrics (`etcd_server_has_leader`, `etcd_server_health_success`, `etcd_server_health_failures`).
- ✓ These are often visualized in the default OpenShift etcd Grafana dashboard.

211. What Prometheus metric helps measure Kubernetes API server request latency?

The primary metric for API server request latency is:

- ✓ `apiserver_request_duration_seconds_bucket` (or `apiserver_request_latencies_bucket` in older versions).
- ✓ This is a histogram metric that tracks the end-to-end latency of requests processed by the API server, broken down by verb (GET, POST, PUT, DELETE), resource (pods, nodes, deployments), and subresource. Analyzing the higher percentile buckets (e.g., `quantile="0.99"`) helps identify tail latency issues.
- ✓ Other related metrics like `apiserver_request_total` (request count) and `apiserver_current_inflight_requests` are also useful. These are typically visualized in the default API Server Grafana dashboards.

212. How would you monitor the performance of the OpenShift Ingress Controllers?

OpenShift Ingress Controllers (routers) are typically based on HAProxy and expose HAProxy metrics that can be scraped by Prometheus. Key aspects to monitor include:

- ✓ **Request Rate**: Total number of requests per second (`haproxy_frontend_sessions_total`, `haproxy_backend_sessions_total`).
- ✓ **Latency**: Backend connection times, response times (`haproxy_backend_connect_time_average_seconds`, `haproxy_backend_response_time_average_seconds`).
- ✓ **Connections**: Current active connections (`haproxy_frontend_current_sessions`, `haproxy_backend_current_sessions`).
- ✓ **Error Rates**: HTTP response codes (4xx, 5xx) (`haproxy_backend_http_responses_total{code="5xx"}`). Check router pod logs for specific errors.

- ✓ **Resource Usage:** CPU and Memory usage of the router pods (oc adm top pod ... -n openshift-ingress).
- ✓ These metrics are usually available in the default HAProxy Grafana dashboard provided by OpenShift monitoring.

213. What are some strategies to optimize container image pull times within the cluster?

Slow image pulls delay application startup and scaling. Strategies include:

Optimize Images:

- ✓ Use smaller base images (e.g., UBI minimal, Alpine, distroless).
- ✓ Employ multi-stage builds to discard build tools and intermediate layers from the final runtime image.
- ✓ Minimize the number of layers in the Dockerfile/Containerfile.

Registry Proximity/Mirroring:

- ✓ Use a geographically close registry mirror.
- ✓ Deploy an **internal OpenShift registry mirror** (using oc adm release mirror or oc-mirror) for frequently used images, especially in disconnected/restricted environments. Configure ICSPs to redirect pulls.

Network: Ensure nodes have high-bandwidth, low-latency network connectivity to the relevant registries (internal or external).

Node Configuration:

- ✓ Increase concurrent pulls: Modify the Kubelet configuration (via MachineConfig) to increase maxParallelImagePulls (default is often 5). Be mindful of node/registry load.
- ✓ Ensure sufficient disk space and IOPS on nodes for image storage (/var/lib/containers).

ImageStream Pre-pulling (Less common): For critical images, potentially use DaemonSets or CronJobs to explicitly pull specific ImageStreamTags onto nodes ahead of time, though this adds complexity.

Troubleshooting Common Infra Issues

214. A node is frequently switching between Ready and NotReady. What are the first things you would investigate?

Node flapping indicates instability. Investigate systematically:

1. **Check Node Conditions:** `oc describe node <node_name>`. Look at the Conditions section (e.g., MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable, KubeletReady). Note the LastTransitionTime and Reason/Message for clues.
2. **Check Kubelet Logs:** This is often the most critical step. Use `oc debug node/<node_name>` to get a shell, then `chroot /host journalctl -u kubelet -f --since "10 minutes ago"`. Look for errors related to PLEG (Pod Lifecycle Event Generator), communication with the API server, resource pressure, CNI issues, or certificate problems.
3. **Check Node Resource Usage:** Use `oc adm top node <node_name>` to check real-time CPU/Memory usage. Use `chroot /host df -h` (in debug pod) for disk usage, especially on `/var/lib/containers` or `/var/log`. High usage can cause instability.
4. **Check Network Connectivity:** From the node (via debug pod), try pinging/curling the API server internal endpoint (`api-int.<cluster_name>.<base_domain>`). Check DNS resolution (`chroot /host resolvectl status`). Check node network interface status (`chroot /host ip a`).
5. **Check CRI-O Logs:** Use `chroot /host journalctl -u crio -f --since "10 minutes ago"` to check for container runtime issues.
6. **Check Recent Events:** `oc get events --field-selector involvedObject.kind=Node,involvedObject.name=<node_name> --sort-by='.lastTimestamp'`.
7. **Check Underlying Infrastructure:** If on a VM or cloud instance, check the hypervisor/cloud provider console for infrastructure-level issues (e.g., host problems, network interruptions).

215. After starting a cluster upgrade, several Cluster Operators go into a DEGRADED state. What is your troubleshooting approach?

This indicates problems applying the new version or configuration for those components.

1. **Identify Degraded Operators:** `oc get co`. Note which specific operators are `DEGRADED=True`.
2. **Prioritize Core Operators:** Focus first on fundamental operators if degraded (e.g., etcd, kube-apiserver, kube-controller-manager, network, machine-config). Issues here often cause cascading failures.
3. **Describe Degraded Operators:** `oc describe co <operator_name>`. Read the `status.conditions` carefully, especially the message for the Degraded condition. This often points directly to the problem.

4. **Check Operator Logs:** `oc logs deployment/<operator_deployment> -n openshift-<operator_name>`. Look for errors related to applying manifests, connecting to dependencies, or version mismatches.
5. **Check Operand Logs:** Check logs of the components managed by the operator (e.g., for etcd operator, check etcd pods in openshift-etcd; for ingress operator, check router pods in openshift-ingress).
6. **Check Related Resources:** `oc describe co` lists related objects. Check their status (`oc get deployment/daemonset/...`).
7. **Check Upgrade Progress:** Run `oc adm upgrade` again. It might provide specific blocking messages.
8. **Consult Release Notes:** Re-check the target version's release notes for known upgrade issues related to the specific operators.
9. **Consider Pausing:** If the issue isn't immediately obvious, consider pausing the upgrade (`oc patch clusterversion version --type=merge -p '{"spec":{"paused": true}}'`) to prevent further changes while investigating.

216. A user reports their pod is stuck in the Pending state. What are the most likely causes you would check first?

Pending means the scheduler cannot place the pod onto a suitable node.

1. **Describe the Pod:** `oc describe pod <pod_name> -n <project_name>`. The Events section is crucial here. Look for messages like:
 - **FailedScheduling:** This indicates scheduler issues. The message will often specify the reason:
 - ✓ 0/X nodes are available: X Insufficient cpu/memory. (Not enough node resources requested).
 - ✓ 0/X nodes are available: X node(s) didn't match node selector/affinity. (Pod requires specific node labels/features not available).
 - ✓ 0/X nodes are available: X node(s) had taints that the pod didn't tolerate. (Pod cannot run on available nodes due to taints).
 - ✓ PodToleratesNodeTaints predicate failed... (Similar to above).
 - ✓ Volume binding predicate failed... (Storage-related, see Q12).
2. **Check Resource Requests:** Does the pod request more CPU/memory than any single node can provide (`oc describe node <node>` shows Allocatable resources)?
3. **Check Node Availability:** Are there enough nodes in the Ready state (`oc get nodes`)? Are worker nodes cordoned (`SchedulingDisabled`)?
4. **Check Taints/Tolerations:** Do available nodes have taints (`oc describe node <node> | grep Taints`) that the pod doesn't tolerate (`oc describe pod <pod>` shows Tolerations)?

5. **Check Node Selectors/Affinity:** Does the pod spec have nodeSelector or nodeAffinity rules (oc describe pod <pod>) that don't match any available node labels (oc get node --show-labels)?
6. **Check PVC Status:** If the pod mounts a PVC, is the PVC Bound (oc get pvc -n <project>)? If the PVC is also Pending, troubleshoot the storage issue first (see Q12).
7. **Check Quotas:** Has the project hit its resource quota limits (oc describe resourcequota -n <project>) for pods, CPU, or memory?

217. A pod is stuck in ContainerCreating. What are potential reasons and how would you diagnose them?

ContainerCreating means the node's Kubelet is trying to start the container but is encountering problems before the container process itself begins.

1. **Describe the Pod:** oc describe pod <pod_name> -n <project_name>. Check the Events section first. Common event messages include:

Failed to create pod sandbox: Often indicates CNI (networking) issues. Check SDN/OVN pod logs on the node.

FailedMount: Problems mounting a requested volume.

- ✓ Check if the referenced ConfigMap or Secret exists in the same namespace (oc get configmap/secret <name> -n <project>).
- ✓ Check if the PVC is Bound and the underlying PV is available/correctly configured. Check CSI driver logs on the node.
- ✓ Check permissions if using hostPath or specific volume types.

FailedAttachVolume: Problems attaching the underlying storage volume to the node (check CSI driver/storage backend).

Error syncing pod: Generic error, often accompanied by more specific messages.

2. **Check Node Status:** Is the node healthy (oc get node <node_name>)? Check disk space (df -h via debug pod), especially /var/lib/containers.
3. **Check Image:** Although ImagePullBackOff is distinct, sometimes image issues manifest here. Verify the image exists and can be pulled manually (podman pull ... on a node or bastion). Check pull secrets.
4. **Check Security Context/SCCs:** While less common for ContainerCreating, sometimes restrictive SCCs might prevent actions needed *before* container start (like setting up certain volume types). Check pod's securityContext and allowed SCCs.
5. **Check Kubelet/CRI-O Logs:** Use oc debug node/<node_name> and chroot /host journalctl -u kubelet or chroot /host journalctl -u crio for low-level errors related to container/sandbox creation or volume mounting.

218. Pods are failing with ImagePullBackOff errors. List the potential causes and checks you would perform.

ImagePullBackOff means the Kubelet failed repeatedly to pull the container image.

1. **Describe the Pod:** `oc describe pod <pod_name> -n <project_name>`. Check Events for messages like Failed to pull image ..., rpc error: code = Unknown desc = ..., manifest unknown, unauthorized.
2. **Verify Image Name/Tag:** Double-check the image: field in the pod spec/deployment YAML for typos in the registry, repository name, or tag. Does the specified tag actually exist in the registry?
3. **Check Registry Connectivity:** Can the node where the pod is scheduled reach the image registry?
 - ✓ Use `oc debug node/<node_name>` and try `curl <registry_url>` or `ping <registry_host>`.
 - ✓ Check firewalls, proxies (`oc get proxy cluster`), and DNS resolution (`resolvectl` status in debug pod).
4. **Check Pull Secret:**
 - ✓ If pulling from a private registry, does the pod's Service Account reference the correct image pull secret (`oc describe sa <sa_name>`)?
 - ✓ Does the referenced secret exist (`oc get secret <secret_name>`)?
 - ✓ Does the secret contain valid, non-expired credentials for the registry (`oc get secret <secret_name> -o jsonpath='{.data.\.dockerconfigjson}' | base64 -d`)?
 - ✓ For cluster components/operators, check the global pull-secret in openshift-config.
5. **Check ICSP / Mirroring:** If using mirrors, are the ImageContentSourcePolicy resources correct (`oc get icsp`)? Is the mirror registry accessible and does it contain the required image digest?
6. **Check Registry Status:** Is the target registry itself operational? Check its status page if external, or check internal registry components (`oc get co image-registry`, `oc get pods -n openshift-image-registry`) if internal.
7. **Check Image Manifest:** Sometimes the error indicates issues with the image manifest itself (e.g., manifest for platform linux/arm64 requested on an amd64 node).

219. You are unable to connect to an RHCOS node using oc debug node. What could be wrong?

Failure to start a debug session can stem from various issues:

1. **Node Not Ready/Unreachable:** Is the target node in the Ready state (`oc get node <node_name>`)? If NotReady or unreachable from the API server, the debug pod cannot be scheduled or started. Troubleshoot the node status first.
2. **API Server Issues:** Is the OpenShift API server responsive (`oc cluster-info`)? `oc debug` needs to communicate with the API.
3. **RBAC Permissions:** Does your user account have the necessary permissions? Running `oc debug node` requires privileges typically granted by the cluster-admin role or a custom role allowing pod creation with hostPath mounts and privileged security contexts (often needing the privileged SCC). Use `oc auth can-i use scc privileged` and `oc auth can-i create pods --subresource=debug -n default` to check.
4. **Scheduling Failure:** The debug pod itself might fail to schedule onto the target node. Check for pending pods in the default namespace (or specified namespace) on that node: `oc get pods -n default -o wide --field-selector spec.nodeName=<node_name>`. Describe the pending debug pod (`oc describe pod <debug_pod_name>`) to see why it failed scheduling (e.g., resource constraints, taints).
5. **Debug Image Issues:** Is the default debug image (`registry.redhat.io/ocp/4.x/tools-rhel8`) pullable? Check global pull secret and registry connectivity if issues are suspected. You can specify a different image with `oc debug node/<node_name> --image=<alternative_image>`.
6. **Network Policies:** Could a NetworkPolicy be preventing the API server or other components from interacting correctly with the debug pod once scheduled? (Less common for session initiation).

220. Applications within the cluster are experiencing DNS resolution failures. How do you troubleshoot this?

DNS issues can be tricky. Follow these steps:

1. **Verify CoreDNS/Cluster DNS:**
 - ✓ Check the status of the dns Cluster Operator: `oc get co dns`.
 - ✓ Check the status of the CoreDNS pods (usually named `dns-default-xxxxx`) in the `openshift-dns` namespace: `oc get pods -n openshift-dns`. Ensure they are Running and Ready.
 - ✓ Check CoreDNS pod logs for errors: `oc logs -n openshift-dns -l dns.operator.openshift.io/daemonset-dns=default`.
2. **Check Pod's DNS Configuration:**
 - ✓ Exec into an affected pod: `oc exec <pod_name> -n <project_name> -- cat /etc/resolv.conf`.

- ✓ Verify the nameserver points to the ClusterIP of the dns-default service (oc get svc dns-default -n openshift-dns -o jsonpath='{.spec.clusterIP}').
- ✓ Check the search domains – they should include relevant suffixes like <namespace>.svc.cluster.local, svc.cluster.local, cluster.local.

3. Test Resolution from Pod:

Exec into the pod: `oc exec <pod_name> -n <project_name> -- /bin/sh (or bash).`

Try resolving different types of names using dig or getent hosts (install bind-utils or getent if needed):

- ✓ Internal Service (same namespace): `dig <service_name>`
- ✓ Internal Service (different namespace): `dig <service_name>.<other_namespace>.svc.cluster.local`
- ✓ External Hostname: `dig google.com`
- ✓ Specify cluster DNS directly: `dig google.com @<cluster_dns_ip>`

4. **Check Network Policies:** Ensure NetworkPolicies are not blocking DNS traffic (UDP/TCP port 53) from the application pods to the CoreDNS pods/service IP in the openshift-dns namespace.
5. **Check Node DNS:** Use `oc debug node/<node_name>` and check the node's `/etc/resolv.conf` (`chroot /host cat /etc/resolv.conf`) and test resolution from the node itself (`chroot /host dig ...`). Node issues can affect pod DNS.
6. **Check Upstream/Forwarding:** If CoreDNS forwards to external resolvers, check connectivity from CoreDNS pods to those external resolvers. Check the CoreDNS ConfigMap (`oc get configmap dns-default -n openshift-dns -o yaml`) for forwarding configuration.

221. The etcd Cluster Operator becomes DEGRADED, reporting an unhealthy cluster. What common issues would you investigate?

Etcd is the distributed brain of the cluster; its health is paramount.

1. **Describe the Operator:** `oc describe co etcd`. Read the Degraded condition message carefully. It often indicates quorum loss, slow requests, or member health issues.
2. **Check Etcd Pod Logs:** `oc logs -n openshift-etcd -l k8s-app=etcd`. Look for errors related to peer communication, leader election failures, slow disk writes (wal), database corruption, or snapshot issues.
3. **Check Etcd Member Health (via debug pod):**
 - `oc debug node/<master_node>`
 - `chroot /host etcdctl endpoint health --cluster` (Requires `etcdctl` and certs configured, often easier via `oc exec` if pods are running).
4. **Check Master Node Resources:** Are master nodes under high CPU, memory, or disk I/O pressure (`oc adm top nodes`, `iostat` via debug pod)? Etcd is sensitive to resource starvation.

5. **Check Disk Performance:** Etcd requires low-latency disk writes. Use tools like fio (via debug pod) to benchmark disk performance on /var/lib/etcd mount points if slow writes are suspected based on logs/metrics.
6. **Check Network Connectivity:** Verify stable, low-latency network connectivity *between* all master nodes on the etcd peer ports (usually 2380). Use ping or other tools between master debug pods. Network partitioning is a common cause of quorum loss.
7. **Check Clock Skew:** Ensure time is synchronized accurately across all master nodes using NTP. Significant clock skew can disrupt etcd. Check chronyc sources via debug pod.
8. **Check Etcd Metrics:** Look at Prometheus metrics (see Performance section) for disk latency, leader changes, etc.

222. Users report the Kubernetes API server is slow or timing out. What areas would you check? API server performance issues impact all cluster interactions.

1. **Check API Server Operator/Pods:**
 - ✓ oc get co kube-apiserver. Check for Degraded/Progressing status.
 - ✓ oc describe co kube-apiserver. Check conditions.
 - ✓ oc get pods -n openshift-kube-apiserver. Ensure pods are Running/Ready. Check restarts.
 - ✓ oc logs -n openshift-kube-apiserver -l app=openshift-kube-apiserver. Look for errors, slow request logs, or throttling messages.
2. **Check Etcd Health:** The API server relies heavily on etcd. If etcd is slow or unhealthy (see Q8), the API server will be impacted. Troubleshoot etcd first if it shows issues.
3. **Check Master Node Resources:** Are the master nodes hosting the API server pods overloaded (CPU, Memory)? Use oc adm top nodes.
4. **Check API Server Metrics:** Query Prometheus/Grafana for:
 - ✓ apiserver_request_duration_seconds_bucket: High latency, especially in upper percentiles.
 - ✓ apiserver_request_total: Abnormally high request rate?
 - ✓ apiserver_current_inflight_requests: High number of concurrent requests?
5. **Check Network:** Verify connectivity from clients (oc CLI, web console, controllers) to the API server endpoints (external api.*, internal api-int.*). Check load balancers if applicable.
6. **Identify Problematic Clients:** Are specific users, controllers, or applications making excessive or inefficient API calls? Audit logs (oc get clusterrolebinding ... audit config) can sometimes help identify sources of high load, though parsing can be complex.
7. **Check Underlying Storage/Network (Masters):** Ensure the master nodes have adequate disk and network performance.

223. Traffic is not reaching an application exposed via a Route. How would you troubleshoot the Ingress path?

Troubleshoot layer by layer from outside-in:

1. **DNS Resolution:** Does the Route hostname (`oc get route <route_name> -o jsonpath='{.spec.host}'`) resolve correctly (using `dig` or `nslookup` from *outside* the cluster) to the public IP address of the OpenShift router/Load Balancer?
2. **External Connectivity/Firewall:** Can you reach the router's public IP on the correct port (usually 80/443) from outside? Check external firewalls, security groups, Load Balancer health checks. Use `curl -v http(s)://<route_host>`.
3. **Ingress Controller (Router) Status:**
 - ✓ Are the router pods running and ready in openshift-ingress (`oc get pods -n openshift-ingress`)?
 - ✓ Check router pod logs for errors related to the specific route or backend connections (`oc logs <router_pod> -n openshift-ingress`).
 - ✓ Check the ingress Cluster Operator status (`oc get co ingress`).
4. **Route Configuration:**
 - ✓ Does the Route exist (`oc get route <route_name> -n <project>`)?
 - ✓ Does it point to the correct Service (`spec.to.name`)?
 - ✓ Is TLS configured correctly (termination type, certificates - `oc describe route <route_name>`)? Check certificate validity if using HTTPS.
5. **Service Configuration:**
 - ✓ Does the target Service exist (`oc get svc <service_name> -n <project>`)?
 - ✓ Does the Service have active Endpoints (`oc get endpoints <service_name> -n <project>`)? If not, the pods matching the Service selector are not ready or don't exist.
6. **Pod Status:** Are the application pods targeted by the Service running, ready, and passing readiness probes (`oc get pods -l <service_selector> -n <project>`)?
7. **Network Policies:** Is there a NetworkPolicy blocking traffic from the openshift-ingress namespace to the application pods in the target project on the required port?
8. **Application Logs:** Check the application pod logs (`oc logs <app_pod> -n <project>`) to see if requests are reaching the application but failing internally.

224. During a node update managed by the Machine Config Operator, a node gets stuck and doesn't update. How do you investigate?

Node updates involve cordoning, draining, applying config, and rebooting. Issues can occur at any stage.

1. **Check MCP Status:** `oc get mcp`. Note the status of the pool the node belongs to (UPDATING, DEGRADED). Check MACHINECOUNT vs READYMACHINECOUNT vs UPDATEDMACHINECOUNT.
2. **Check Node Status:** `oc get node <node_name>`. Is it Ready, NotReady, SchedulingDisabled?
3. **Check MCD Logs:** This is crucial. Find the Machine Config Daemon pod on the stuck node (`oc get pods -n openshift-machine-config-operator -o wide --field-selector spec.nodeName=<node_name>`) and check its logs (`oc logs <mcd_pod> -n openshift-machine-config-operator`). Look for errors related to:
 - ✓ Draining the node (often PDB violations).
 - ✓ Applying the new MachineConfig (rpm-ostree errors, file writing errors).
 - ✓ Rebooting the node.
 - ✓ Post-reboot checks failing.
4. **Check Drain Failures:** If logs suggest drain issues, check PDBs (`oc get pdb -A`). Are any applications preventing eviction? Describe the pods that failed to drain (`oc describe pod <pod_name>`).
5. **Check Node oc describe:** `oc describe node <node_name>`. Look at recent Events for drain failures, CNI errors, Kubelet issues.
6. **Check Console Access:** If possible (e.g., VM console, BMC), check the node's console during boot/runtime for kernel panics or systemd errors.
7. **Check Rendered Config:** Did the MCO successfully create the target rendered MachineConfig (`oc get mc <rendered_config>`)?
8. **Check MCO Logs:** Check the Machine Config Operator logs (`oc logs deployment/machine-config-operator -n openshift-machine-config-operator`) for higher-level errors about managing the pool update.

225. A user cannot create a PVC; it remains Pending. What storage-related issues might be the cause?

Pending PVCs usually mean the storage provisioner cannot fulfill the request.

1. **Describe the PVC:** `oc describe pvc <pvc_name> -n <project_name>`. The Events section is key. Look for messages like:
 - ✓ ProvisioningFailed: The storage provisioner encountered an error.
 - ✓ No persistent volumes available for this claim...: No static PVs match, and dynamic provisioning might be failing or disabled for the requested StorageClass.
 - ✓ storageclass.storage.k8s.io "<class_name>" not found: The specified StorageClass doesn't exist.

2. Check StorageClass:

- ✓ Does the PVC specify a StorageClass (spec.storageClassName)?
- ✓ Does that StorageClass exist (oc get sc <class_name>)?
- ✓ If no class is specified, is there a default StorageClass configured (oc get sc)?
- ✓ Is the provisioner listed in the StorageClass (oc describe sc <class_name>) healthy?

3. **Check Provisioner Pods:** Find the pods for the relevant storage provisioner (e.g., CSI driver pods in openshift-cluster-csi-drivers, ODF pods in openshift-storage). Check their logs for errors related to volume creation.
4. **Check Underlying Storage:** Is the backend storage system (SAN, NAS, Cloud Provider, Ceph) healthy and does it have sufficient capacity? Check the storage system's console/logs.
5. **Check Quotas:** Has the project hit its persistentvolumeclaims or requests.storage quota (oc describe resourcequota -n <project_name>)?
6. **Check PV Availability (Static Provisioning):** If expecting to bind to a pre-created PV, does a suitable PV exist (oc get pv) with matching accessModes, capacity, and potentially storageClassName or labels, and is its status Available?

226. You notice gaps in metrics data in Grafana or alerts aren't firing as expected. How do you troubleshoot the monitoring stack?

Issues in the monitoring pipeline can cause data loss or alert failures.

1. **Check Monitoring Operator:** oc get co monitoring. Ensure it's Available, not Degraded or Progressing. oc describe co monitoring for details.
2. **Check Prometheus:**
 - ✓ Are Prometheus pods running/ready (oc get pods -n openshift-monitoring -l app.kubernetes.io/name=prometheus)? Check restarts.
 - ✓ Check Prometheus pod logs for errors (scraping failures, storage issues, configuration reload problems).
 - ✓ Check Prometheus PVCs (oc get pvc -n openshift-monitoring -l app.kubernetes.io/name=prometheus). Are they Bound? Is there sufficient disk space on the underlying PVs?
 - ✓ Access Prometheus UI (via port-forward or route) and check Targets page for scrape errors. Check Alerts page for rule evaluation errors.
3. **Check Alertmanager:**
 - ✓ Are Alertmanager pods running/ready (oc get pods -n openshift-monitoring -l app.kubernetes.io/name=alertmanager)? Check restarts.
 - ✓ Check Alertmanager pod logs for errors (configuration issues, notification failures, peer communication problems).
 - ✓ Check Alertmanager configuration secret (alertmanager-main in openshift-monitoring) for syntax errors.

- ✓ Check Alertmanager UI (via route) to see active alerts, silences, and receiver integrations.
- 4. **Check Exporters:** Are the metrics sources (node-exporter, kube-state-metrics, application-specific exporters) running correctly? Check their respective pods/logs.
- 5. **Check Network:** Is there network connectivity between Prometheus and its scrape targets? Between Prometheus and Alertmanager? Between Alertmanager and notification receivers? Check NetworkPolicies.
- 6. **Check Grafana:** If dashboards are failing, check Grafana pods (oc get pods -n openshift-monitoring -l app.kubernetes.io/name=grafana) and logs. Check data source configuration within Grafana UI.

227. Application logs are missing from Kibana or the logging stack reports errors. What steps would you take?

Troubleshoot the logging pipeline (Fluentd -> Elasticsearch -> Kibana).

1. **Check Logging Operator:** oc get co logging (if using Red Hat OpenShift Logging). Ensure Available/not Degraded. oc describe co logging.
2. **Check Fluentd:**
 - ✓ Are Fluentd pods running on all nodes (oc get pods -n openshift-logging -l component=fluentd)?
 - ✓ Check Fluentd pod logs on nodes *where logs are missing*. Look for errors connecting to Elasticsearch, buffer overflows, parsing errors, or permission issues reading container logs (/var/log/pods/...).
3. **Check Elasticsearch:**
 - ✓ Are ES pods running/ready (oc get pods -n openshift-logging -l component=elasticsearch)? Check StatefulSet status.
 - ✓ Check ES cluster health (via curl in ES pod or oc describe co logging). Look for red or yellow status.
 - ✓ Check ES pod logs for errors (shard allocation failures, disk watermark issues, configuration errors).
 - ✓ Check ES PVCs/disk usage (oc get pvc -n openshift-logging ...). Is the cluster running out of disk space?
4. **Check Kibana:**
 - ✓ Are Kibana pods running/ready (oc get pods -n openshift-logging -l component=kibana)?
 - ✓ Check Kibana pod logs for errors connecting to Elasticsearch.
 - ✓ In Kibana UI, verify the correct index patterns (e.g., app-*, infra-*) are configured and exist in Elasticsearch. Check the time range selected.
5. **Check Application:** Is the application actually generating logs to stdout/stderr? Use oc logs <app_pod> to confirm.

6. **Check Network:** Verify network connectivity between Fluentd pods and the Elasticsearch service. Check NetworkPolicies.

228. How would you identify which specific pods are causing consistently high resource usage on a particular node?

Use oc adm top pods with Node Selector: Filter the top pods output to show only pods running on the specific node, then sort by the resource of interest (CPU or memory).

```
# Top CPU consumers on 'worker-3.example.com'
oc adm top pods -A --sort-by=cpu --node-selector=kubernetes.io/hostname=worker-3.example.com
# Top Memory consumers on 'worker-3.example.com'
oc adm top pods -A --sort-by=memory --node-selector=kubernetes.io/hostname=worker-3.example.com
```

List Pods and Check Individually: Get all pods on the node and then check usage individually if needed.

```
# Get pod names and namespaces on the node
oc get pods -A -o wide --field-selector spec.nodeName=<node_name> | awk '{if(NR>1) print "-n \"$1\" \"$2\"}'
```

Then check specific pods if needed (less efficient for finding top consumers)

```
# oc adm top pod <pod_name> -n <namespace> --containers
```

Use Monitoring Dashboards: Grafana dashboards often have views that allow filtering by node and sorting pods by resource consumption, providing a visual way to identify top consumers over time. Look for dashboards related to "Node Details" or "Pod Resources".

Miscellaneous Infra Tasks

229. How can you determine if an OpenShift cluster was installed using IPI (Installer-Provisioned Infrastructure) or UPI (User-Provisioned Infrastructure)?

Several indicators help determine the installation type:

Infrastructure Resource: Check the cluster-level Infrastructure object. The `status.platformStatus.type` field indicates the underlying platform (AWS, vSphere, BareMetal, etc.), but not directly IPI/UPI. However, IPI installations typically populate more fields under `status.platformStatus`.

```
oc get infrastructure cluster -o yaml
```

Machine API Resources: The most reliable indicator is the presence and active use of Machine API resources (Machines, MachineSets). IPI heavily relies on these to manage cluster nodes. UPI installations *can* optionally use them but often manage nodes externally.

```
oc get machinesets -A
```

```
oc get machines -A
```

If multiple MachineSets exist and correspond to your control plane and worker nodes, it's almost certainly an IPI installation. If these namespaces/resources are mostly empty or absent, it's likely UPI.

install-config.yaml: If available, the original install config clearly defines the platform and implies the method (IPI usually has more platform-specific automation fields).

230. How do you configure cluster-wide HTTP/HTTPS proxy settings for outbound traffic?

Cluster-wide proxy settings are configured using the Proxy cluster object named `cluster`.

1. **Edit the Proxy Object:**
2. `oc edit proxy cluster`
3. **Modify spec:** Add or update the following fields within the `spec` section:
 - ✓ `httpProxy`: URL of the HTTP proxy (e.g., `http://user:password@proxy.example.com:8080`).
 - ✓ `httpsProxy`: URL of the HTTPS proxy (often the same as HTTP proxy URL).
 - ✓ `noProxy`: Comma-separated list of domains, CIDRs, or IPs that should *not* use the proxy (e.g., `.cluster.local,.svc,.example.com,192.168.1.0/24`). It's crucial to include internal cluster domains (`.svc`, `.cluster.local`), API server endpoints, and any internal registries/services.
4. **Save Changes:** The Cluster Network Operator watches this object and propagates the proxy environment variables (`HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY`) to relevant cluster components (like operator pods) and newly created pods (via admission webhook). Existing pods generally need to be recreated to pick up the new settings.

231. What is the process for adding a custom CA certificate bundle to be trusted by cluster components and workloads?

To make cluster components (like operators pulling images) and potentially workloads trust internal CAs or proxies performing TLS inspection:

1. **Prepare CA Bundle:** Concatenate all necessary CA certificates (in PEM format) into a single file (e.g., custom-ca-bundle.crt).
2. **Create ConfigMap:** Create a ConfigMap in the openshift-config namespace containing this bundle. The key within the ConfigMap *must* be ca-bundle.crt.
3. `oc create configmap custom-ca --from-file=ca-bundle.crt=./custom-ca-bundle.crt -n openshift-config`
4. **Patch Cluster Proxy:** If the CA is needed for trusting the configured HTTP/HTTPS proxy:
5. `oc patch proxy cluster --type=merge --patch='{"spec":{"trustedCA":{"name":"custom-ca"}}}'`
6. **Patch Image Config:** If the CA is needed for trusting image registries (internal or external mirrors):
7. `oc patch image.config.openshift.io cluster --type=merge --patch='{"spec":{"additionalTrustedCA":{"name":"custom-ca"}}}'`
8. **Propagation:** Cluster operators and node services (like CRI-O) will detect these changes and update their trust stores. Node updates might involve Machine Config Operator rollouts. Pods generally need to be recreated to mount the updated trust bundles (often mounted via openshift-service-ca.crt ConfigMap which gets updated).

232. How do you check the status of the Machine API Operator and its associated pods?

Check Cluster Operator:

```
oc get co machine-api
oc describe co machine-api # Check conditions for errors
```

Check Pods: The operator components run in the openshift-machine-api namespace.

```
oc get pods -n openshift-machine-api

# Look for pods related to: machine-api-operator, machine-api-controllers,
potentially provider-specific controllers (e.g., vsphere-machine-controllers)

Ensure the Cluster Operator is Available and pods are Running/Ready.
```

233. What is a MachineSet, and how do you list the ones defined in the cluster?

MachineSet: A Machine API resource (similar concept to ReplicaSet for Pods) that ensures a specified number of Machine objects exist for a given configuration. It defines a template for creating new Machines (specifying instance type, image, availability zone, user data, etc.). If a Machine managed by a MachineSet is deleted or fails health checks, the MachineSet controller creates a new one to maintain the desired replica count. MachineSets are primarily used in IPI environments to manage worker node scaling.

Listing:

```
oc get machinesets -n openshift-machine-api
```

```
# Output Columns: NAME      DESIRED  CURRENT  READY  AVAILABLE  AGE
#      cluster-abc-w-a  2        2        2        2        150d
#      cluster-abc-w-b  1        1        1        1        150d
```

234. In an IPI environment, how do you scale the number of worker nodes using MachineSets?

Use the `oc scale` command, targeting the specific MachineSet you want to adjust in the `openshift-machine-api` namespace.

```
oc scale machineset <machineset_name> --replicas=<desired_number> -n openshift-machine-api
```

Example: Scale the machineset in zone 'a' to 3 replicas

```
oc scale machineset cluster-abc-w-a --replicas=3 -n openshift-machine-api
```

- ✓ Increasing replicas causes the MachineSet controller to create new Machine objects based on its template. The platform-specific controllers then provision the corresponding infrastructure (e.g., EC2 instances, vSphere VMs).
- ✓ Decreasing replicas causes the controller to delete excess Machine objects, triggering infrastructure deprovisioning.

235. How can you monitor the provisioning status of new Machines created by a MachineSet?

1. **List Machines:** Filter machines potentially owned by the MachineSet (labels often help, or check `ownerReferences`).
2. `oc get machines -n openshift-machine-api` # Look for newly created ones
3. **Check Machine Phase:** The `PHASE` column in `oc get machines` shows the status (e.g., Provisioning, Provisioned, Running, Deleting, Failed).
4. **Describe Machine:** Get detailed status and events for a specific machine:
5. `oc describe machine <machine_name> -n openshift-machine-api`
 - ✓ Look at `status.phase`, `status.errorMessage` (if Failed), and the `Events` section for detailed provisioning steps and errors.

6. **Check Corresponding Node:** Once a Machine reaches the Running phase, a corresponding Node object should appear (oc get nodes). It might take time for the node to fully register and become Ready.

236. How do you find the underlying cloud provider instance ID (e.g., AWS EC2 instance ID, vSphere VM name) associated with an OpenShift Node object in an IPI cluster?

The spec.providerID field on the Node object usually holds this information.

```
oc get node <node_name> -o jsonpath='{.spec.providerID}'
```

Example Output (AWS): aws:///us-east-1a/i-0123456789abcdef0

Example Output (vSphere): vsphere:///<vcenter>/<datacenter>/vm/<folder>/<vm_uuid>

- ✓ You can also often find this information on the corresponding Machine object in the openshift-machine-api namespace (oc get machine <machine_name> -n openshift-machine-api -o jsonpath='{.spec.providerID}').

237. What is the Node Tuning Operator used for? How do you check its status?

Purpose: The Node Tuning Operator manages the tuned daemon on RHCOS nodes. It allows administrators to apply custom system-level performance tunings (beyond the defaults) to groups of nodes based on labels. It uses Tuned Custom Resources to deliver these profiles, which can adjust kernel parameters, CPU affinities, disk schedulers, etc., often for specific workload requirements (like low latency or high throughput).

Checking Status:

Operator Pods:

```
oc get pods -n openshift-cluster-node-tuning-operator
```

Tuned DaemonSet: Check the tuned DaemonSet pods running on each node:

```
oc get pods -n openshift-cluster-node-tuning-operator -l openshift-app=tuned
```

Applied Tuned CRs: List the Tuned objects:

```
oc get tuned -n openshift-cluster-node-tuning-operator
```

```
oc describe tuned <tuned_profile_name> -n openshift-cluster-node-tuning-operator # Check status/conditions
```

238. How can you list any custom Tuned profiles applied in the cluster?

List the Tuned Custom Resources in the operator's namespace. Custom profiles are typically created by administrators in addition to the default rendered profiles managed by other operators (like the Performance Addon Operator).

```
oc get tuned -n openshift-cluster-node-tuning-operator
```

- ✓ Inspect the YAML of non-default Tuned objects to see the custom configurations being applied (`oc get tuned <custom_tuned_name> -n openshift-cluster-node-tuning-operator -o yaml`).

239. If using MetalLB for bare metal LoadBalancer services, how do you check the status of its components?

MetalLB typically runs components in the metallb-system namespace.

```
oc get pods -n metallb-system
```

Look for:

- ✓ controller Deployment pod(s): Manages IP address assignments.
- ✓ speaker DaemonSet pods (one per node): Announce service IPs using BGP or L2 protocols.

Ensure these pods are Running and Ready. Check their logs for any configuration or announcement errors.

240. How are IP address pools typically configured for MetalLB?

IP address pools that MetalLB can assign to LoadBalancer services are configured via:

ConfigMap (Older Method): Editing the config ConfigMap in the metallb-system namespace. Define address-pools within the data.config section.

- ✓ `oc edit configmap config -n metallb-system`

CRDs (Operator Method - Recommended): If installed via the MetalLB Operator, use Custom Resources like MetalLB, AddressPool, BGPAdvertisement, L2Advertisement. Create/edit AddressPool CRs to define ranges of IPs MetalLB can use.

- ✓ `oc get addresspool -n metallb-system`
- ✓ `oc edit addresspool <pool_name> -n metallb-system`

241. If using the Local Storage Operator, how do you check the status of its pods?

The Local Storage Operator components usually run in the openshift-local-storage namespace.

```
oc get pods -n openshift-local-storage
```

Look for:

- ✓ The local-storage-operator deployment pod.
- ✓ The local-storage-diskmaker DaemonSet pods (run on nodes to discover local disks).
- ✓ The local-storage-provisioner DaemonSet pods (if using the local volume provisioner).

Check logs for discovery or provisioning errors.

242. How do you list the LocalVolume resources managed by the Local Storage Operator?

The operator creates LocalVolume Custom Resources representing the discovered storage devices on nodes that match the operator's configuration.

```
oc get localvolume -n openshift-local-storage
```

- ✓ This shows the discovered volumes, their capacity, node affinity, and status. These are then used to provision Persistent Volumes with node affinity.

243. If using OpenShift Data Foundation (ODF), how do you check the status of its core component pods?

ODF (formerly OpenShift Container Storage/OCS) deploys its components primarily in the openshift-storage namespace.

```
oc get pods -n openshift-storage
```

Look for pods related to:

- ✓ **Ceph:** rook-ceph-osd-*, rook-ceph-mon-*, rook-ceph-mgr-*, csi-rbdplugin-*, csi-cephfsplugin-* (OSDs handle data, MONs handle quorum, MGRs provide management/metrics, CSI plugins handle PV mounting).
- ✓ **NooBaa:** noobaa-operator-*, noobaa-core-*, noobaa-db-* (for S3 object storage).
- ✓ **Rook Operator:** rook-ceph-operator-* (manages Ceph deployment).
- ✓ **ODF Operator:** ocs-operator-*, odf-operator-controller-manager-* (manages ODF deployment).

Ensure key pods (operator, MONs, OSDs, CSI drivers) are Running and Ready.

244. How do you quickly check the health status of the underlying Ceph cluster managed by ODF?

Check CephCluster CR: The CephCluster resource provides a high-level health summary.

```
oc get cephcluster -n openshift-storage -o jsonpath='{.items[0].status.ceph.health}'  
# Example output: HEALTH_OK, HEALTH_WARN, HEALTH_ERR
```

Use Ceph Tools Pod: For detailed status, exec into the Rook Ceph tools pod and run ceph status.

```
# Find tools pod  
TOOLS_POD=$(oc get pods -n openshift-storage -l app=rook-ceph-tools -o  
jsonpath='{.items[0].metadata.name}')
```

```
# Exec and run command  
oc exec -it ${TOOLS_POD} -n openshift-storage -- ceph status
```

This provides detailed health checks, MON/OSD status, pool status, PGs (Placement Groups) status, IO activity, etc.

245. How can you check the overall storage capacity and usage within ODF?

Ceph Status: The ceph status command (run via the tools pod as above) shows overall capacity (SIZE), used space (USED), and available space (AVAIL).

Ceph Block Pools: Check capacity and usage per storage pool (often backing StorageClasses).

```
oc exec -it ${TOOLS_POD} -n openshift-storage -- ceph df  
# Or check the CRs  
oc get cephblockpool -n openshift-storage
```

ODF Dashboards: The OpenShift Console often includes ODF-specific dashboards (under Storage) that visualize capacity, usage, performance (IOPS, throughput), and health. Grafana dashboards for Ceph are also usually available via cluster monitoring.

Prometheus Metrics: Query Ceph-related metrics exposed to Prometheus (e.g., ceph_cluster_total_bytes, ceph_cluster_total_used_bytes).

246. What are two ways to find the URL for the OpenShift web console?

oc whoami --show-console: If logged in via oc, this command directly outputs the console URL.

```
oc whoami --show-console
```

oc get route console -n openshift-console: Get the Route object for the console and extract the hostname.

```
oc get route console -n openshift-console -o jsonpath='{ "https://" }{.spec.host}{ "n" }'
```

247. How do you check the status of the OpenShift Console Operator and its pods?**Check Cluster Operator:**

```
oc get co console
oc describe co console # Check conditions
```

Check Pods: Console components run in the openshift-console namespace.

```
oc get pods -n openshift-console
# Look for 'console-*' pods (main UI) and 'downloads-*' pods (serving CLI tools etc.)
```

Ensure the operator is Available and pods are Running/Ready.

248. How can the appearance (e.g., login page, branding) of the OpenShift Console be customized?

Customizations are applied by editing the Console cluster resource named cluster.

```
oc edit console cluster
```

Modify fields under spec.customization::

- ✓ branding: Set to okd or openshift (influences default logo/theme).
- ✓ documentationBaseUrl: Point to custom documentation.
- ✓ customProductName: Set a custom name displayed in the console.
- ✓ customLogoFile: Reference a ConfigMap (in openshift-config) containing a custom logo image (ca-bundle.crt key convention often used, but file key can be specified).

Refer to the official documentation for specific fields and ConfigMap structure for logos.

249. What command lists all Custom Resource Definitions (CRDs) installed in the cluster?

Use `oc get crd`.

```
oc get crd
# Or use the full name
oc get customresourcedefinitions
```

This lists all the custom resource types (beyond core Kubernetes types like Pods, Services) that have been defined in the cluster, often installed by Operators.

250. How can you check if etcd encryption at rest is enabled and what mode is used?

Check the APIServer cluster resource named cluster.

```
oc get apiserver cluster -o jsonpath='{.spec.encryption.type}'
```

The output will show the encryption type currently configured. Common values:

- ✓ identity: No encryption at rest is enabled (keys are stored unencrypted).
- ✓ aescbc: AES-CBC encryption is enabled. This is the standard mode.
- ✓ aesgcm: AES-GCM encryption is enabled (generally preferred if supported by your version/platform).

An empty output or absence of the spec.encryption field usually implies identity (no encryption).

251. Describe the high-level process for rotating etcd encryption keys.

Rotating etcd encryption keys is a sensitive operation performed to enhance security. It involves generating new keys and migrating existing data to be encrypted with them.

1. **Trigger Rotation:** Patch the APIServer cluster resource (spec.encryption.keyRotationStrategy) to signal the desire for rotation (e.g., set a new reason). The specific mechanism might vary slightly by OCP version, consult docs. Often involves setting spec.unsupportedConfigOverrides.encryption.reason.
2. **Operator Action:** The kube-apiserver-operator detects the trigger. It generates a *new* encryption key (e.g., encryption-key-... secret in openshift-config-managed).
3. **API Server Reconfiguration:** The operator updates the API server configuration to use *both* the old and new keys for decryption but *only* the new key for encrypting *new* data. API servers are rolled out with this new config.
4. **Data Migration:** The operator initiates a background process where the API server reads all resources from etcd, decrypts them (using old or new key), and rewrites them encrypted with the *new* key. This happens gradually.
5. **Monitoring:** Monitor the APIServer resource (status.encryption.writeKey), etcd encryption secrets (oc get secrets -n openshift-config-managed -l apiserver.openshift.io/encryption-key=true), and operator logs to track migration progress.
6. **Finalization:** Once migration is complete, the operator may automatically (or via another trigger) update the API server config again to *only* use the new key, effectively retiring the old key.

Important: This process requires the cluster to be healthy and should be done during a maintenance window, following official documentation precisely.

252. How can you check which instance of a scaled control plane component (like kube-controller-manager) holds the leader election lease?

Core Kubernetes control plane components use a leader election mechanism (usually based on Leases or Endpoints) to ensure only one instance is active at a time.

Identify Namespace: Find the namespace where the component runs (e.g., openshift-kube-controller-manager).

Get Lease/Endpoint: Check for a Lease object (newer Kubernetes versions) or an Endpoint object (older versions) often named after the component itself within that namespace.

```
# Newer method using Leases

oc get lease kube-controller-manager -n openshift-kube-controller-manager -o yaml

# Older method using Endpoints annotation

oc get endpoints kube-controller-manager -n openshift-kube-controller-manager -o
yaml
```

Inspect Holder Identity: Look for fields like holderIdentity (in Leases) or an annotation like control-plane.alpha.kubernetes.io/leader (in Endpoints). The value typically contains the hostname or pod name of the current leader instance.

253. How would you verify the NTP server configuration being used by an RHCOS node?

Use `oc debug node` and the `chronyc` command:

1. Start debug session: `oc debug node/<node_name>`
2. Inside the debug pod, run:
3. `chroot /host chronyc sources -v`

This command queries the `chronyd` daemon running on the node. The output lists the configured NTP sources (servers), their status (e.g., `^*` indicates the current sync source), stratum, poll interval, and offset/jitter details.

254. How do you check if the `chronyd` service is running and synchronized on a node?

Use `oc debug node` and `systemctl / chronyc`:

1. Start debug session: `oc debug node/<node_name>`
2. Check service status:
3. `chroot /host systemctl status chronyd`
 - Ensure it's active (running).
4. Check synchronization status:
5. `chroot /host chronyc tracking`

Look at Reference ID (should point to the sync source server), Stratum (should be reasonable, e.g., 2, 3, 4), Last offset (should be small, close to zero), and Leap status (should be Normal).

255. How can you inspect the effective Kubelet configuration arguments being used on a node?

The Kubelet configuration comes from multiple sources (files, MachineConfigs).

1. **Check Kubelet Config File:** The primary config file is often referenced by the systemd unit.
 - ✓ `oc debug node/<node_name>`
 - ✓ `chroot /host cat /etc/kubernetes/kubelet.conf` (Common location, might vary)
2. **Check MachineConfig:** Find the rendered MachineConfig applied to the node's pool (`oc get node <node_name> -o jsonpath='{.metadata.annotations.machineconfiguration.openshift.io/currentConfig}'`). Then get the MachineConfig YAML (`oc get mc <rendered_mc_name> -o yaml`) and look for the Kubelet configuration snippet within `ignition.config.systemd.units` or related sections.
3. **Check Running Process (Less reliable):** `chroot /host ps aux | grep kubelet` might show some command-line arguments, but many settings are loaded from files.

256. How can you inspect the effective CRI-O configuration settings on a node?

CRI-O settings are primarily defined in configuration files.

1. **Check Main Config File:**
 - ✓ `oc debug node/<node_name>`
 - ✓ `chroot /host cat /etc/crio/crio.conf`
2. **Check Drop-in Files:** Configuration can be overridden or extended by files in `/etc/crio/crio.conf.d/`.
 - ✓ `chroot /host ls /etc/crio/crio.conf.d/`
 - ✓ `chroot /host cat /etc/crio/crio.conf.d/<filename>.conf`
3. **Check MachineConfig:** As with Kubelet, the applied rendered MachineConfig might contain CRI-O configuration snippets under `ignition.config.storage.files` or related sections.

257. How do you check the configured maximum number of pods allowed to run on a specific node?

1. **Node Status:** The node object reports its capacity.
2. `oc get node <node_name> -o jsonpath='{.status.capacity.pods}'`
3. **Kubelet Configuration:** The ultimate source is the Kubelet's `--max-pods` setting. Check the Kubelet config file or effective arguments (see Q27). If not explicitly set, Kubernetes calculates a default based on resources or uses a platform default (often 110 or 250).

258. What is the purpose of the `oc adm must-gather` command?

`oc adm must-gather` is a diagnostic tool used to collect comprehensive information about an OpenShift cluster (or specific components) for troubleshooting purposes, typically when working with Red Hat Support. It runs a collector image that gathers:

- ✓ Cluster Operator status and logs.
- ✓ Core component logs (API server, etcd, controllers).
- ✓ Node logs (kubelet, crio).
- ✓ Resource definitions (YAMLs for nodes, pods, CRDs, etc.).
- ✓ Network information.
- ✓ Configuration files.
- ✓ It packages all this data into a compressed archive that can be uploaded for analysis, providing a snapshot of the cluster's state and recent activity.

259. If using the Compliance Operator, how do you check the status of its pods?

The Compliance Operator typically runs in the `openshift-compliance` namespace.

```
oc get pods -n openshift-compliance
```

Look for the main operator deployment pod(s) and potentially pods related to specific scans or remediations (e.g., `ocp4-cis-scanner-*`).

260. How do you list the results of compliance scans run by the Compliance Operator?

The operator uses several CRDs to manage scans and results:

- ✓ `ComplianceScan`: Represents a request to scan nodes against a profile.
- ✓ `ComplianceSuite`: Groups scans and remediations.
- ✓ `ComplianceCheckResult`: Stores the outcome (Pass, Fail, Info) of individual checks within a scan.
- ✓ `ComplianceRemediation`: Represents actions to fix failed checks.

```
oc get compliancescan -n openshift-compliance
```

```
oc get compliancesuite -n openshift-compliance
```

```
oc get compliancecheckresult -n openshift-compliance # Can be numerous
```

```
oc get complianceremediation -n openshift-compliance
```

Describe specific objects for details, especially `ComplianceCheckResult` for reasons why a check failed.

261. If using the File Integrity Operator, how do you check the status of its pods?

The File Integrity Operator usually runs in the `openshift-file-integrity` namespace.

```
oc get pods -n openshift-file-integrity
```

Look for the operator deployment pod(s) and the `aide-daemon-*` DaemonSet pods (one per node) which perform the integrity checks using AIDE (Advanced Intrusion Detection Environment).

262. How do you view the results of file integrity checks performed on nodes?

The operator stores results in the FileIntegrity Custom Resource, typically one per node pool.

```
oc get fileintegrity -n openshift-file-integrity
```

```
# Example: oc get fileintegrity worker-fileintegrity -n openshift-file-integrity -o yaml
```

- ✓ Inspect the status field of the relevant FileIntegrity object. It shows the overall status (Phase: Pending, Active, Re-initializing, Failed) and detailed results, including counts of added/removed/changed files detected during the last scan compared to the baseline database.

263. Describe a method to test network latency between two cluster nodes.

Use ping from within debug pods running on the source and target nodes.

1. Start debug pods on both nodes:
2. `oc debug node/<node1_name> --image=registry.redhat.io/rhel8/support-tools -n default -- bash -c 'sleep infinity' &`
3. `oc debug node/<node2_name> --image=registry.redhat.io/rhel8/support-tools -n default -- bash -c 'sleep infinity' &`

(Using support-tools image as it likely contains ping)

4. Find the debug pod names and the *internal IP* of node 2 (`oc get node <node2_name> -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}'`).
5. Exec into the debug pod on node 1:
6. `oc exec -it <debug_pod_on_node1> -n default -- /bin/bash`
7. Run ping inside the pod:
8. `ping <node2_internal_ip>`

Observe the round-trip time (RTT) values. Consistent, low RTT (e.g., <1-2ms within same DC/AZ) is expected. High or variable latency indicates network issues.

264. Describe a method to test network bandwidth between two cluster nodes.

Use the iperf3 tool within debug pods.

Start debug pods on both nodes, ensuring the image contains iperf3 (e.g., a custom image or potentially registry.redhat.io/rhel8/support-tools might have it).

```
# Assuming image has iperf3
```

```
oc debug node/<node1_name> --image=<image_with_iperf3> -n default -- bash -c 'sleep infinity' &
```

```
oc debug node/<node2_name> --image=<image_with_iperf3> -n default -- bash -c 'sleep infinity' &
```

```
Find debug pod names and the internal IP of node 1.
```

Exec into the debug pod on node 1 (Server):

```
oc exec -it <debug_pod_on_node1> -n default -- /bin/bash
```

```
iperf3 -s # Start iperf3 server
```

Exec into the debug pod on node 2 (Client):

```
oc exec -it <debug_pod_on_node2> -n default -- /bin/bash
```

```
iperf3 -c <node1_internal_ip> # Connect client to server
```

The client will report the measured bandwidth between the two nodes. Run multiple times for consistency.

265. How do you inspect the certificate currently being used by the default Ingress Controller?

The default Ingress Controller uses a certificate stored in a secret, typically named router-certs, within the openshift-ingress namespace.

1. Get the secret:
2. `oc get secret router-certs -n openshift-ingress -o yaml`
3. The certificate data is in `.data."tls.crt"`, base64 encoded. Decode it and pipe to `openssl` to view details:
4. `oc get secret router-certs -n openshift-ingress -o jsonpath='{.data.tls\.crt}' | base64 -decode | openssl x509 -noout -text`

This shows the Issuer, Subject (Common Name, SANs), Validity period (Not Before, Not After), etc.

266. What is the general process for replacing the default Ingress certificate with a custom one?

1. **Prepare Custom Certificate:** Obtain your custom certificate and private key files (PEM format). Ensure the certificate covers the necessary wildcard domain (`*.apps.<cluster_name>.<base_domain>`) and potentially other specific hostnames. Include any necessary intermediate CA certificates in the certificate file (server cert first, then intermediates).
2. **Create/Update Secret:** Create a new TLS secret in the openshift-ingress namespace containing your custom certificate and key.
3. `oc create secret tls custom-ingress-cert --cert=path/to/your.crt --key=path/to/your.key -n openshift-ingress`
4. # Or 'oc replace secret tls router-certs ...' if overwriting the default (less common)
5. **Patch IngressController:** Edit the default IngressController resource in the openshift-ingress-operator namespace to reference your new secret.
6. `oc patch ingresscontroller default -n openshift-ingress-operator --type=merge --patch='{ "spec": { "defaultCertificate": { "name": "custom-ingress-cert" } } }'`

7. **Rollout:** The Ingress Operator will detect the change and roll out updates to the router pods, which will start using the new certificate. Monitor router pods (`oc get pods -n openshift-ingress -w`).

267. How do you inspect the certificate authority used for signing the API server's serving certificate? The API server's serving certificate is typically signed by an internal CA managed by the cluster. The CA certificate is often stored in secrets within operator namespaces. A common one to check is the kube-apiserver's client CA, used for aggregation:

```
# Check the CA bundle used by kube-apiserver to verify clients/aggregation

oc get configmap kube-apiserver-client-ca -n openshift-kube-apiserver -o jsonpath='{.data.ca-bundle\.crt}' | openssl x509 -noout -text -inform PEM

# Check the CA that signs the serving cert itself (often managed internally)

# This might be in openshift-kube-apiserver-operator or openshift-config-managed namespaces

# Example: Check the secret referenced by the Kube API Server operator status

oc get secret serving-ca -n openshift-kube-apiserver-operator -o jsonpath='{.data.tls\.crt}' | base64 --decode | openssl x509 -noout -text -inform PEM
```

The exact secret name might vary slightly depending on the OCP version and configuration.

268. How are internal certificates for services typically managed in OpenShift 4, and how could you check their validity?

Management: Internal service certificates (used for secure communication between pods within the cluster) are primarily managed automatically by the **Service CA Operator**. When a Service is annotated with `service.beta.openshift.io/serving-cert-secret-name: <secret_name>`, this operator automatically generates a TLS certificate and key, signed by a cluster-internal CA, and stores them in the specified secret `<secret_name>` within the service's namespace. Applications mount this secret to use the certificate. The operator also handles automatic rotation of these certificates before they expire.

Checking Validity:

1. Identify the secret name from the Service annotation (`oc get svc <service_name> -o yaml`).
2. Get the secret from the service's namespace (`oc get secret <secret_name> -n <namespace> -o yaml`).
3. Decode the certificate (`.data."tls.crt"`) and check its validity period using `openssl`:
4. `oc get secret <secret_name> -n <namespace> -o jsonpath='{.data.tls\.crt}' | base64 --decode | openssl x509 -noout -text | grep -A 2 Validity`
5. # Or just check expiry date
6. `oc get secret <secret_name> -n <namespace> -o jsonpath='{.data.tls\.crt}' | base64 --decode | openssl x509 -noout -enddate`

269. How would you find the process ID (PID) of the kubelet process running on a node?

Use oc debug node and standard Linux process tools:

```
oc debug node/<node_name>
Inside the debug pod:
chroot /host pgrep -o kubelet # '-o' shows the oldest/original process if multiple
match
# Or more detailed:
chroot /host ps aux | grep '/usr/bin/kubelet'
```

270. How would you find the process ID (PID) of the main cri-o process running on a node?

Use oc debug node and standard Linux process tools:

```
oc debug node/<node_name>
Inside the debug pod:
chroot /host pgrep -o cri-o # '-o' shows the oldest/original process
# Or more detailed:
chroot /host ps aux | grep '/usr/bin/crio'
```

271. How do you check the current SELinux enforcement mode (Enforcing, Permissive, Disabled) on an RHCOS node?

Use oc debug node and SELinux tools:

```
oc debug node/<node_name>
Inside the debug pod:
chroot /host getenforce
# Or for more detail:
chroot /host sestatus
```

OpenShift nodes *must* run in Enforcing mode for proper operation and security.

272. How can you view the active firewall rules (iptables or nftables) on an RHCOS node?

Use oc debug node and the appropriate firewall command:

1. oc debug node/<node_name>
2. Inside the debug pod:

If using iptables (older OCP versions or specific configs):

```
chroot /host iptables -L -n -v
chroot /host iptables -t nat -L -n -v
```

If using nftables (default in newer RHEL/RHCOS):

```
chroot /host nft list ruleset
```

These commands display the complex rules managed by components like kube-proxy and the CNI plugin to handle pod/service networking.

273. How do you display the IP routing table configured on an RHCOS node?

Use oc debug node and the ip command:

```
oc debug node/<node_name>
Inside the debug pod:
chroot /host ip route show
# Or 'ip r' for short
```

This shows how the node routes traffic to different destinations, including default gateways, pod networks, and service networks.

274. How do you view the kernel ring buffer messages (dmesg) on an RHCOS node?

Use oc debug node and the dmesg command:

```
oc debug node/<node_name>
Inside the debug pod:
chroot /host dmesg -T
# '-T' adds human-readable timestamps
```

This is useful for diagnosing low-level hardware, driver, or kernel-related issues.

275. How might you check CPU affinity settings if performance tuning has been applied?

CPU affinity restricts processes to specific CPU cores.

1. **Check PerformanceProfile/Tuned:** If using the Performance Addon Operator or Node Tuning Operator, examine the applied PerformanceProfile or Tuned CR YAML for `isolated_cores` or affinity settings.
2. **Check Pod Spec:** Some high-performance pods might have CPU manager policies set (static) and request specific exclusive CPUs (`resources.limits.cpu matching resources.requests.cpu`).
3. **Check Running Process (Requires tools):**
 - ✓ Exec into the container: `oc exec <pod_name> -c <container_name> -- /bin/bash`
 - ✓ Find the process ID (PID) of the application.
 - ✓ Use taskset (if available in the container image): `taskset -cp <PID>` - This shows the current CPU affinity mask for the process.

276. How do you check if Transparent Huge Pages (THP) are enabled or disabled on a node?

Use oc debug node and check sysfs entries:

```
oc debug node/<node_name>
Inside the debug pod:
# Check if THP is enabled (always, madvise, never)
chroot /host cat /sys/kernel/mm/transparent_hugepage/enabled
# Check if background defragmentation for THP is enabled
chroot /host cat /sys/kernel/mm/transparent_hugepage/defrag
```


For many performance-sensitive workloads (especially databases), disabling THP (never) is often recommended. This is typically done via MachineConfig/tuned.

277. How can you determine the I/O scheduler being used for a specific block device on a node?

Use oc debug node and check sysfs:

```
oc debug node/<node_name>
```

Identify the block device name (e.g., sda, nvme0n1) using chroot /host lsblk.

Inside the debug pod, check the scheduler file:

```
chroot /host cat /sys/block/<device_name>/queue/scheduler
```

```
# Example: chroot /host cat /sys/block/sda/queue/scheduler
```

The output shows the available schedulers, with the active one enclosed in square brackets (e.g., [mq-deadline] kyber bfq none). Common options include mq-deadline, bfq, kyber, none.

278. How do you check the disk space usage of the systemd journal on a node?

Use oc debug node and journalctl:

```
oc debug node/<node_name>
```

Inside the debug pod:

```
chroot /host journalctl --disk-usage
```

This reports the current disk space occupied by archived and active journal files. Configuration in /etc/systemd/journald.conf (e.g., SystemMaxUse=) controls size limits.

279. How do you check the status of the kube-proxy pods running on the cluster nodes?

kube-proxy runs as a DaemonSet managed by the cluster-network-operator in the openshift-kube-proxy namespace.

```
oc get pods -n openshift-kube-proxy -o wide
```

Ensure a pod is running on each relevant node and is in the Running state with ready containers. Check logs (oc logs <kube-proxy-pod> -n openshift-kube-proxy) if issues are suspected (e.g., errors applying firewall rules).

280. How do you check the status of the dns-operator and its pods?

Operator Status: oc get co dns, oc describe co dns

Operator Pods:

```
oc get pods -n openshift-dns-operator
```

The dns-operator manages the CoreDNS deployment (dns-default) in the openshift-dns namespace.

281. How do you check the status of the authentication operator and its pods?

Operator Status: `oc get co authentication, oc describe co authentication`

Operator Pods:

`oc get pods -n openshift-authentication-operator`

This operator manages authentication components like the internal OAuth server and the OAuth API server.

282. How do you check the status of the oauth-apiserver pods?

The OAuth API server provides OAuth-related API endpoints.

`oc get pods -n openshift-oauth-apiserver`

Check the deployment status and pod readiness/logs.

283. How do you check the status of the internal oauth-openshift server pods?

This is the built-in OAuth server that handles token issuance and interaction with configured Identity Providers.

`oc get pods -n openshift-authentication`

Look for pods named `oauth-openshift-*`. Check deployment status and pod readiness/logs.

284. How do you check the status of the etcd-operator pods?

Operator Status: `oc get co etcd, oc describe co etcd`

Operator Pods:

`oc get pods -n openshift-etcd-operator`

This operator manages the lifecycle (deployment, backups, scaling) of the etcd cluster itself, whose pods run in `openshift-etcd`.

285. How do you check the status of the kube-storage-version-migrator operator and pods?

This operator handles the migration of stored Kubernetes objects when their storage version changes between Kubernetes releases.

Operator Status: `oc get co kube-storage-version-migrator, oc describe co kube-storage-version-migrator`

Operator Pods:

`oc get pods -n openshift-kube-storage-version-migrator-operator`

Also check the `StorageVersionMigration` resources: `oc get storageversionmigration`.

286. What command lists all MachineConfig objects (base and rendered)?

Use `oc get machineconfig` or its short name `oc get mc`.

`oc get mc`

This lists all MachineConfigs, including:

- ✓ Base configs (e.g., 00-worker, 01-master-kubelet).
- ✓ Custom configs created by administrators.
- ✓ Rendered configs applied to pools (e.g., rendered-worker-`<hash>`).

287. What are ControllerRevisions used for in Kubernetes/OpenShift?

ControllerRevision resources are primarily used internally by controllers like StatefulSet and DaemonSet to manage and track revisions of their pod templates. When you update the pod template within a StatefulSet or DaemonSet, the controller creates a new ControllerRevision containing the updated template hash. This allows for rollback capabilities – you can tell the controller to revert to a previous ControllerRevision, effectively rolling back the pod template used by the workload. They provide an immutable record of past configurations.

288. How do you list all PodDisruptionBudgets (PDBs) configured across all projects?

Use `oc get poddisruptionbudgets --all-namespaces` or the short name `oc get pdb -A`.

`oc get pdb -A`

This lists all PDBs defined cluster-wide, showing the minimum available/maximum unavailable pods allowed for the associated application during voluntary disruptions.

289. How can you determine if a PDB is currently preventing pods from being evicted (e.g., during a node drain)?

Use `oc describe pdb <pdb_name> -n <project_name>`.

`oc describe pdb my-app-pdb -n my-app`

Look at the Status: section, specifically the Allowed Disruptions field. If this value is 0, it means evicting another pod covered by this PDB would violate the budget (minAvailable or maxUnavailable), and therefore, voluntary evictions (like those during a node drain) for these pods are currently blocked. The drain process will wait until Allowed Disruptions becomes greater than 0.

290. How would you get a rough estimate of the total CPU and memory resources requested by all pods currently running in the cluster?

There isn't a single built-in oc command for this exact sum. Methods include:

Monitoring Dashboards: Grafana dashboards often have panels summarizing total cluster resource requests and limits based on Prometheus metrics scraped from kube-state-metrics. This is usually the easiest way. Look for cluster overview or capacity planning dashboards.

Scripting oc get pods: You can write a script to iterate through all pods in all namespaces, extract their container resource requests (spec.containers[*].resources.requests), and sum them up.

```
# Conceptual example (requires parsing JSON/YAML output)

# oc get pods -A -o jsonpath='{.items[*].spec.containers[*].resources.requests}' |
<script_to_parse_and_sum>
```

oc describe nodes: Summing the Allocated resources across all nodes (oc describe node <node> shows allocated requests per node) gives an approximation, though it might include terminated pod resources temporarily.

291. Explain the role of the Service CA operator.

The Service CA Operator is a cluster-level operator responsible for **automatically generating and managing TLS certificates for internal cluster services**. Its primary function is to watch for Services annotated with service.beta.openshift.io/serving-cert-secret-name: <secret_name>. When it sees such an annotation, it:

1. Generates a TLS private key and certificate.
2. The certificate's Common Name (CN) is typically set to the service's internal DNS name (<service_name>.<namespace>.svc).
3. The certificate is signed by a cluster-internal Certificate Authority (the "Service CA").
4. It stores the key (tls.key), certificate (tls.crt), and the CA certificate (ca.crt) in the specified secret (<secret_name>) within the service's namespace.
5. It automatically rotates the certificates before they expire.

This allows pods to easily mount these secrets and establish secure TLS communication with other internal services, trusting the Service CA.

292. How does the Machine Config Operator (MCO) apply changes to nodes? Describe the flow.

The MCO orchestrates node configuration updates using MachineConfigs:

1. **Detection:** The Machine Config Controller (part of MCO) watches for changes to MachineConfig objects.
2. **Rendering:** When changes are detected (new MC created, existing one modified/deleted), the Controller combines all relevant MachineConfigs (base OS, cluster, custom) for a specific pool (e.g., worker) into a new "rendered" MachineConfig (e.g., rendered-worker-<new_hash>).
3. **Pool Update:** The Controller updates the MachineConfigPool object for that pool, pointing its spec.configuration.name to the new rendered config.
4. **MCD Notification:** The Machine Config Daemon (MCD) running on each node within the pool watches its corresponding MachineConfigPool object. It sees the desired configuration has changed.
5. **Node Cordon & Drain:** The MCO (often via the MCD coordinating) selects a node to update (respecting maxUnavailable). It cordons the node (oc adm cordon) and then drains it (oc adm drain), evicting pods gracefully (respecting PDBs).
6. **Apply Config:** Once drained, the MCD on the node applies the changes defined in the new rendered MachineConfig (e.g., writes files, potentially runs rpm-ostree commands for RHCOS updates).
7. **Reboot:** If the changes require it (e.g., kernel update, OS update), the MCD triggers a node reboot.
8. **Uncordon & Verify:** After the node reboots and the Kubelet reports Ready, the MCD verifies the update and the MCO uncordons the node (oc adm uncordon), making it available for scheduling again.
9. **Repeat:** The process repeats for the next node in the pool until all nodes are updated to the new rendered config.

293. What is the purpose of the oc adm must-gather command and when would you use it?

Purpose: oc adm must-gather is a diagnostic tool designed to collect a comprehensive snapshot of cluster state, configuration, and logs. It gathers information from various sources (Cluster Operators, nodes, resource definitions, events) relevant to troubleshooting complex cluster issues.

When to Use: It's typically used when:

- ✓ Engaging with Red Hat Support for a cluster problem. Support engineers will often request must-gather output for analysis.
- ✓ Performing deep troubleshooting of cluster-level failures (e.g., upgrade issues, operator degradation, networking problems) where individual component logs might not provide the full picture.

- ✓ Capturing the state of the cluster at a specific point in time when an intermittent issue occurs.

It packages the collected data into a compressed archive, making it easier to share for offline analysis.

294. Describe the difference between scaling a Deployment/DeploymentConfig and scaling a MachineSet.

Scaling Deployment/DeploymentConfig: This controls the number of **Pods** running for a specific application *within* the existing cluster nodes. `oc scale deployment my-app --replicas=5` tells Kubernetes to ensure 5 identical Pods for my-app are running, scheduling them onto available worker nodes that meet the pod's requirements. It manages the application's runtime instances.

Scaling MachineSet (IPI Clusters): This controls the number of **Nodes** (physical or virtual machines) belonging to a specific pool (e.g., worker nodes in a particular availability zone). `oc scale machineset my-cluster-worker-us-east-1a --replicas=3 -n openshift-machine-api` tells the Machine API Operator (and underlying cloud provider) to ensure 3 actual machine instances matching the MachineSet's template exist. It manages the cluster's infrastructure capacity itself. Scaling a MachineSet adds or removes nodes from the cluster.

295. Why is it generally discouraged to manually modify resources in openshift-* namespaces unless following specific documentation?

Resources in namespaces prefixed with openshift- (e.g., openshift-kube-apiserver, openshift-ingress, openshift-machine-api, openshift-monitoring) are typically **managed by Cluster Operators**.

- ✓ **Operator Reconciliation:** Operators continuously watch the resources they manage and try to reconcile their state back to a desired configuration defined by the operator logic or its Custom Resource.
- ✓ **Overwritten Changes:** If you manually edit a resource managed by an operator (e.g., a Deployment, ConfigMap, Secret), the operator is likely to **overwrite your changes** during its next reconciliation loop, reverting it back to what the operator expects.
- ✓ **Unexpected Behavior:** Manual changes can conflict with the operator's logic, potentially leading to unexpected behavior, instability, or preventing the operator from functioning correctly (making it DEGRADED).
- ✓ **Configuration Method:** Configuration for these components should almost always be done via the mechanisms provided by the operator, usually by editing the operator's own Custom Resource (e.g., `oc edit ingresscontroller default -n openshift-ingress-operator`) or cluster-level configuration objects (like `oc edit proxy cluster`). Only modify resources directly in these namespaces if explicitly instructed by official OpenShift documentation for a specific task.

296. How can you identify which nodes belong to the 'master' pool vs. a 'worker' pool?

Nodes have labels indicating their role.

Check Node Labels: Use `oc get nodes --show-labels`. Look for labels like:

- ✓ `node-role.kubernetes.io/master`: Indicates a control plane node.
- ✓ `node-role.kubernetes.io/worker`: Indicates a worker node (may also be present on masters unless they are tainted to prevent workloads).

Filter by Label:

List master nodes

```
oc get nodes -l node-role.kubernetes.io/master
```

List worker nodes (that aren't also masters, if masters have worker role)

```
oc get nodes -l node-role.kubernetes.io/worker,!node-role.kubernetes.io/master
```

Or simply list all workers if masters don't have the worker role label

```
oc get nodes -l node-role.kubernetes.io/worker
```

Check MachineConfigPools: The default MCPs are usually named master and worker. You can list nodes associated with an MCP label:

```
oc get nodes -l machineconfiguration.openshift.io/role=master
```

```
oc get nodes -l machineconfiguration.openshift.io/role=worker
```

297. What considerations are important when choosing a Persistent Volume Reclaim Policy?

The `persistentVolumeReclaimPolicy` field in a `PersistentVolume` (PV) or `StorageClass` determines what happens to the underlying storage volume when the corresponding PVC is deleted. Key considerations:

Delete:

- ✓ **Pros:** Automatically cleans up the underlying storage volume when the PVC is deleted. Prevents orphaned volumes and associated costs. Simple workflow for dynamically provisioned volumes where data persistence beyond the PVC lifecycle isn't needed.
- ✓ **Cons:** **Data is permanently lost** if the PVC is accidentally deleted. Not suitable for critical data that needs to survive application deletion/recreation unless backups are robust.

Retain:

- ✓ **Pros:** Protects against accidental data loss via PVC deletion. The underlying storage volume persists even after the PVC is gone. Allows data recovery or re-attachment to a new PV/PVC later. Suitable for critical data.

- ✓ **Cons:** Requires **manual cleanup** of the underlying storage volume and the PV object (which enters Released state) after the PVC is deleted. Failure to clean up leads to orphaned volumes, consuming resources and potentially costs. More operational overhead.
- ✓ **Choice:** Use Delete for ephemeral or easily reproducible data, or where dynamic provisioning and deletion are frequent. Use Retain for critical, irreplaceable data where manual intervention for cleanup is acceptable to prevent accidental loss. The default often depends on the StorageClass provisioner.

298. Explain the potential impact of incorrect Network Policy rules.

Network Policies control traffic flow between pods and namespaces. Incorrect rules can have significant negative impacts:

- ✓ **Blocking Legitimate Traffic:** Overly restrictive ingress or egress rules can block necessary communication between application tiers (e.g., frontend to backend), connections to databases, access to cluster services (like DNS, API server, monitoring), or outbound connections to external services. This leads to application malfunction or complete failure.
- ✓ **Allowing Unintended Traffic:** Overly permissive rules (or the absence of policies, resulting in default-allow) can negate security segmentation. A compromised pod could potentially access sensitive services or data in other pods/namespaces that it shouldn't be able to reach, increasing the blast radius of a security breach.
- ✓ **DNS Failures:** Incorrectly configured policies might block pods from reaching CoreDNS (port 53 UDP/TCP) in the openshift-dns namespace, causing application failures due to inability to resolve service names or external hosts.
- ✓ **Troubleshooting Difficulty:** Debugging connectivity issues caused by complex or incorrect Network Policies can be challenging, requiring careful examination of selectors and rules across multiple policies.
- ✓ **Operator/Platform Issues:** Blocking traffic needed by OpenShift operators or platform components can lead to operator degradation or cluster instability.

299. Why is running containers as root discouraged, and how do SCCs help enforce this?

Why Discouraged: Running container processes as the root user (UID 0) poses significant security risks:

- ✓ **Increased Blast Radius:** If an attacker compromises a process running as root within the container, they gain root privileges *within that container's namespace*. While container isolation helps, vulnerabilities in the kernel or misconfigurations could potentially allow escape to the host node with elevated privileges.
- ✓ **Principle of Least Privilege:** Applications rarely need full root privileges to function. Running as root violates the principle of granting only the minimum necessary permissions.
- ✓ **Filesystem Permissions:** Root processes can modify any file within the container's writable layers, potentially damaging the container image or other processes.

How SCCs Help: Security Context Constraints (SCCs) enforce restrictions on pods and containers, including user ID control:

RunAsUser Strategy: SCCs define strategies like MustRunAsNonRoot or MustRunAsRange.

- ✓ **MustRunAsNonRoot:** Explicitly prevents containers from starting if they try to run as UID 0. Pods must either specify a non-root runAsUser in their securityContext or the container image must define a non-root USER.
- ✓ **MustRunAsRange:** Requires the container's UID to fall within a specific range allocated to the namespace (often enforced by default in OpenShift 4 using the restricted-v2 or restricted SCC), preventing arbitrary UID selection including root.

Default SCCs: OpenShift applies restrictive default SCCs (like restricted-v2) to standard users, which typically enforce MustRunAsNonRoot or MustRunAsRange, preventing root execution unless explicitly granted access to a more permissive SCC (like anyuid or privileged).

300. What are some key differences in managing an OpenShift 4 cluster compared to managing a standard Kubernetes cluster?

While OpenShift is built on Kubernetes, it adds layers of opinionation, automation, and integrated components, leading to management differences:

- ✓ **Operator Lifecycle Management (OLM):** OpenShift heavily relies on the Operator pattern for managing core components (via Cluster Operators) and applications (via OperatorHub). Upgrades and configuration are often managed via Operator CRs, unlike potentially manual component management in standard Kubernetes.
- ✓ **Immutable Infrastructure (RHCOS):** OpenShift 4 primarily uses Red Hat Enterprise Linux CoreOS, an immutable, container-optimized OS managed via MachineConfigs and the MCO, rather than traditional package management on nodes.
- ✓ **Integrated Components:** OpenShift includes built-in, tightly integrated solutions for routing (Routes/Ingress Controller), image registry, monitoring (Prometheus/Grafana), logging (EFK/Loki), developer tooling (Builds, ImageStreams, Console), and authentication (OAuth Server) that might require separate installation and integration in standard Kubernetes.
- ✓ **Security Context Constraints (SCCs):** OpenShift's SCCs provide a more granular and restrictive security model by default compared to Kubernetes' Pod Security Policies (deprecated) or Pod Security Admission (newer).
- ✓ **Machine API:** IPI installations use the Machine API for declarative node management, abstracting underlying infrastructure provisioning.
- ✓ **oc vs kubectl:** While kubectl works, the oc CLI includes additional OpenShift-specific commands for managing Routes, Builds, Projects, ImageStreams, oc adm tasks, etc.

- ✓ **Projects vs Namespaces:** OpenShift uses Project as an annotation layer on top of Kubernetes Namespace, adding features like default user permissions upon creation.
- ✓ **Update Process:** Cluster updates are managed centrally via the Cluster Version Operator (CVO) and update channels, providing a more automated and controlled upgrade experience for the entire platform stack.

Get one to one assistance for OpenShift Hands on labs (50 labs).
WhatsApp Dhinesh +91 9444410227 and get started today!